INSTITUT TEKNOLOGI DEL

Real-time Automation System for DDoS Based on Anomaly Detection and Route Nullification

TUGAS AKHIR
KELOMPOK 13

13323013 Immanuel Manurung
13323023 Enjelita Sitorus
13323030 Ivan Gunawan Simanjuntak

FAKULTAS VOKASI
PROGRAM STUDI DIPLOMA III TEKNOLOGI KOMPUTER
INSTITUT TEKNOLOGI DEL
2025

INSTITUT TEKNOLOGI DEL

Real-time Automation System for DDoS Based on Anomaly Detection and Route Nullification

TUGAS AKHIR
KELOMPOK 13

Diajukan sebagai salah satu syarat untuk memperoleh gelar A.Md.T

13323013 Immanuel Manurung
13323023 Enjelita Sitorus
13323030 Ivan Gunawan Simanjuntak

FAKULTAS VOKASI
PROGRAM STUDI DIPLOMA III TEKNOLOGI KOMPUTER

DAFTAR ISI

DAFTAR ISI 2
DAFTAR TABEL 5
DAFTAR GAMBAR 6
BAB I PENDAHULUAN 8
1.1 Latar Belakang 8
1.2 Rumusan Masalah 10
1.3 Tujuan 10
1.4 Ruang Lingkup 10
1.5 Ekspektasi Hasil 11
1.6 Sistematika Penyajian 12
BAB II TINJAUAN PUSTAKA 13
2.1 *Distributed Denial of Service (DDoS)* 13
2.1.1 Defenisi dan Konsep Dasar *Distributed Denial of Service (DDoS)* 13
2.1.2 Cara Kerja Serangan *Distributed Denial of Service (DDoS)* 14
2.1.3 Jenis – Jenis Serangan *Distributed Denial of Service (DDoS)* 16
2.1.4 Jenis Serangan yang Diuji 18
2.1.5 Perbedaan Trafik Legal dan Trafik Ilegal 20
2.1.6 Dampak Serangan *DDoS* 21
2.2 Metode Mitigasi Serangan *DDoS* 23
2.2.1 Rate Limiting 23
2.2.2 IP Filtering 24
2.2.3 Load Balancing 25
2.2.4 Intrusion Detection /Prevention System (IDS/IPS) 26
2.2.5 Anomaly-based Mitigation 27
2.3 *Anomaly Detection* 28
2.3.1 Defenisi *Anomaly Detection* 28
2.3.2 Proses *Anomaly Detection* 29
2.3.3 Pendekatan *Anomaly Detection* 30
2.4 *Black Hole Routing* 32
2.4.1 Defenisi Black Hole Routing 33
2.4.2 Strategi Penanganan Black Hole Routing 34

2.4.3 Tantangan Implementasi Black Hole Routing 35
2.4.4 Integrasi *Black Hole Routing* dengan Wazuh *Active Response* 36
2.5 Penggunaan *IPTables Rules* untuk Mitigasi DDoS 37
2.6 Otomatisasi Serangan DDoS 38
2.6.1 Komponen Utama Otomatisasi Respons DDoS 39
2.6.2 Konsep Otomatisasi dalam Mitigasi Serangan 40
2.7 Integrasi *Anomaly Detection* dan Black Hole Routing 40
2.8 Tools (Perangkat Lunak) 41
2.8.1 Wazuh Platform 41
2.8.2 Ubuntu Server 22.04 47
2.8.3 Kali Linux sebagai Mesin Penyerang 47
2.8.4 Hping3 48
2.9 Infrastruktur (Perangkat Keras dan Jaringan) 49
2.9.1 VMWare Workstation (Lingkungan Instalasi) 49
2.9.2 Oracle VirtualBox 50
2.9.3 Modem Telkomsel Orbit 51
2.10 Related Works 52
2.11 Jadwal Penelitian 54
BAB III ANALISIS DAN DESAIN 55
3.1 Analisis 55
3.1.1 Analisis Masalah 55
3.1.2 Analisis Pemecahan Masalah 56
3.1.3 Batasan Lingkungan *Small-Scale Network* 57
3.2 Analisis Kebutuhan Sistem 58
3.2.1 Analisis Kebutuhan Perangkat Keras (Hardware) 58
3.2.2 Analisis Kebutuhan Perangkat Lunak (Software) 60
3.2.3 Analisis Kebutuhan Keamanan Sistem 61
3.3 Model Perancangan Sistem 62
3.3.1 Diagram Blok Sistem 63
3.3.2 Diagram Arsitektur 65
3.3.3 Flowchart 67
3.4 Skenario Pengujian 69
3.4.1 Skenario Kondisi Awal (Pengujian Tanpa Serangan) 69

3.4.2 Skenario Pengujian Serangan *DDoS* 71
3.4.3 Kriteria Keberhasilan Pengujian 73
BAB IV IMPLEMENTASI DAN HASIL PENGUJIAN 75
4.1 Instalasi dan Konfigurasi 75
4.1.1 Instalasi dan Konfigurasi pada *Virtual Machine* I (Web Server) 75
4.1.2 Instalasi dan Konfigurasi pada *Virtual Machine* II (Linux Gateway) 79
4.1.3 Konfigurasi pada *Virtual Machine* III (Wazuh Server) 85
4.1.4 Instalasi dan Konfigurasi pada *Virtual Machine* IV (*Client*) 89
4.1.5 Instalasi dan Konfigurasi pada *Virtual Machine* IV (*Attacker*) 92
4.2 Pengujian Sistem 93
4.2.1 Pengujian Trafik Normal dan Konektivitas Sistem 94
4.2.2 Pengujian Serangan DDoS (*Multi Attacker*) 99
BAB V KESIMPULAN DAN SARAN 105
5.1 Kesimpulan 105
5.2 Saran 105
DAFTAR PUSTAKA 107

DAFTAR TABEL

Tabel 2. 1 Perbedaan Trafik Legal dan Trafik Ilegal 21
Tabel 3. 1 Analisis Kebutuhan Perangkat Keras (Hardware) 59
Tabel 3. 2 Analisis Kebutuhan Perangkat Lunak (Software) 60
Tabel 3. 3 Analisis Kebutuhan Keamanan Sistem 61
Tabel 3. 4 Arsitektur VM Topologi 66
Tabel 3. 5 Kriteria Keberhasilan Pengujian 73
Tabel 4. 1 Konfigurasi Network Interface 80
Tabel 4. 2 Arsitektur dan Pembagian VM Attacker 93

DAFTAR GAMBAR

Gambar 2. 1 Cara Kerja *Distributed Denial of Service* 16
Gambar 2. 2 Wazuh Agent..... 43
Gambar 2. 3 Wazuh Indexer 44
Gambar 2. 4 Wazuh Dashboard 46
Gambar 2. 5 VMWare Workstation..... 50
Gambar 2. 6 Oracle VirtualBox 51
Gambar 2. 7 Jadwal Penelitian..... 54
Gambar 3. 1 Diagram Blok Sistem 64
Gambar 3. 2 Diagram Arsitektur 65
Gambar 3. 3 Flowchart 68
Gambar 3. 4 Skenario Pengujian Tanpa Serangan..... 70
Gambar 3. 6 Skenario Pengujian Serangan DDoS..... 72
Gambar 4. 1 Status layanan Nginx 76
Gambar 4. 2 Halaman default Nginx 77
Gambar 4. 3 Konfigurasi ossec.conf pada Wazuh Agent 78
Gambar 4. 4 Status Layanan Wazuh Agent 79
Gambar 4. 5 Konfigurasi IP statis..... 80
Gambar 4. 6 Hasil konfigurasi netplan 81
Gambar 4. 7 Aktivasi IP Forwarding..... 81
Gambar 4. 8 Konfigurasi ossec.conf pada Wazuh Agent 83
Gambar 4. 9 Status layanan Wazuh Agent 84
Gambar 4. 10 Verifikasi Konfigurasi pada Linux Gateway 85
Gambar 4. 11 Status layanan Wazuh-Indexer..... 86
Gambar 4. 12 Status layanan Wazuh-Manager..... 87
Gambar 4. 13 Status layanan Wazuh-Dasboard 88
Gambar 4. 14 Dashboard Platform Wazuh 88
Gambar 4. 15 Status layanan Filebeat..... 89
Gambar 4. 16 Konfigurasi Jaringan Client 90
Gambar 4. 17 Konfigurasi ossec.conf pada Wazuh Agent 91
Gambar 4. 18 Status layanan Wazuh Agent 92
Gambar 4. 19 Verifikasi instalasi hping3 93
Gambar 4. 20 Hasil Pengujian Konektivitas Client ke Gateway 94

Gambar 4. 21 Pengujian Konektivitas Gateway ke Web Server 95
Gambar 4. 22 Pengujian Konektivitas Client ke Web Server..... 95
Gambar 4. 23 Hasil pengujian akses server target..... 96
Gambar 4. 24 Performa Web Server..... 97
Gambar 4. 25 Status Agent pada Menu Agents..... 98
Gambar 4. 26 Log Event dari Agent Client 99
Gambar 4. 27 Eksekusi serangan DDoS 100
Gambar 4. 28 Dampak Serangan pada Web Server..... 101
Gambar 4. 29 Deteksi Anomali Trafik oleh Wazuh berdasarkan Threshold PPS 102
Gambar 4. 30 Pengujian akses Web Server 103

BAB I
PENDAHULUAN

Bab ini berisi uraian mengenai latar belakang masalah, rumusan masalah, tujuan, manfaat, serta batasan masalah yang menjadi dasar dari pelaksanaan Tugas Akhir. Selain itu, pada bab ini juga dijelaskan metode pendekatan yang digunakan untuk menyelesaikan permasalahan yang diangkat. Dengan demikian, Bab 1 memberikan gambaran menyeluruh tentang arah penelitian dan dasar pemikiran yang melandasi penyusunan Tugas Akhir ini.

1.1 Latar Belakang

Ancaman *Distributed Denial of Service (DDoS)* merupakan salah satu serangan paling berbahaya dalam keamanan jaringan. Serangan ini bekerja dengan membanjiri server atau infrastruktur jaringan menggunakan lalu lintas berlebihan dari banyak sumber sekaligus, sehingga kapasitas sistem menjadi penuh dan layanan tidak dapat diakses oleh pengguna sah. Dampaknya dapat berupa penurunan kinerja layanan, kerugian finansial, hingga lumpuhnya infrastruktur yang bergantung pada ketersediaan layanan jaringan.

Permasalahan utama dalam penanganan serangan *DDoS* adalah lambatnya proses deteksi dan mitigasi. Pada banyak kasus, administrator jaringan harus menganalisis pola lalu lintas dan menentukan tindakan mitigasi secara manual, sementara serangan terus berlangsung. Penelitian terdahulu menunjukkan bahwa mekanisme tradisional seperti *firewall* dan *Intrusion Detection System (IDS)* berbasis *signature* sering kali tidak cukup efektif menghadapi serangan *DDoS* berskala besar dan variatif [1] [2] [3].

Untuk mengatasi keterbatasan tersebut, penelitian terkini banyak mengusulkan pendekatan *anomaly detection*. Sistem deteksi berbasis metode ringan (*lightweight anomaly detection*) terbukti mampu membedakan trafik normal dan trafik serangan secara efisien, terutama pada jaringan dengan keterbatasan sumber daya [4]. Metode berbasis *Deep Neural Network (DNN)* juga menawarkan akurasi tinggi untuk pendeteksian serangan secara *real-time*, meskipun membutuhkan komputasi yang lebih besar [5]. Selain itu, pendekatan berbasis *Digital Twin* memungkinkan pemodelan perilaku jaringan secara dinamis dan memberikan kemampuan deteksi serangan *DDoS* secara lebih adaptif [3]. Dari sisi arsitektur, *Software-Defined Networking (SDN)* turut dimanfaatkan untuk mempercepat proses deteksi dan mitigasi melalui pengendalian terpusat yang lebih fleksibel [6].

Dalam *anomaly detection*, penentuan *threshold* menjadi aspek yang sangat penting. Banyak penelitian terdahulu masih mengandalakan *threshold* statis, seperti jumlah paket per detik atau jumlah koneksi *TCP half-open* sebagai indikator anomali [7] [8]. *Threshold* berbasis *half-open connection* juga digunakan untuk mendeteksi serangan *SYN Flood* [9], sementara *threshold* berbasis *bandwidth* diterapkan untuk mendeteksi serangan *UDP Flood* pada jaringan berskala kecil. Namun, *threshold* statis memiliki kelemahan seperti meningkatnya *false-positive* pada kondisi trafik padat serta risiko *false-negative* saat menghadapai serangan *slow-rate.*

Tren penelitian terbaru menunjukkan pergeseran menuju *threshold* dinamis yang menyesuaikan kondisi trafik secara *real-time*. Beberapa studi mengembangkan metode adaptive *threshold* yang mampu menyesuaikan nilai ambang bedasarkan perilaku trafik aktual sehingga lebih efektif dalam mendeteksi serangan berintensitas rendah (low-rate *DDoS*) [8] [9]. Dengan demikian, *threshold* tidak lagi berupa nilai numerik tetap, tetapi fleksibel mengikuti pola jaringan yang berubah-ubah.

Namun, deteksi saja tidak cukup tanpa mekanisme mitigasi yang cepat dan efektif. Salah satu Teknik mitigasi yang banyak digunakan adalah *blackhole routing* atau *route nullification*. Teknik ini bekerja dengan mengalihkan trafik yang terindikasi sebagai berbahaya menuju *null route* agar paket langsung dibuang sebelum mencapai server target. Penelitian menunjukkan bahwa *blackhole routing* dapat mencegah beban berlebih pada server, meskipun membutuhkan integrasi deteksi yang akurat agar trafik sah tidak ikut terblokir [10] [11].

Urgensi mitigasi otomatis semakin terlihat melalui berbagai insiden besar seperti serangan *DDoS* 2,3 Tbps terhadap Amazon Web Server (AWS) pada tahun 2020 dan serangan terhadap DNS provider Dyn pada tahun 2016 yang membuat layanan besar seperti Twitter, Netflix, dan Reddit tidak dapat diakses. Di Indonesia, serangan *DDoS* juga beberapa kali menargetkan layanan publik dan perbankan, sehingga mendorong perlunya sistem mitigasi otomatis yang cepat dan adaptif.

Dalam konteks penelitian ini, otomatisasi respons *DDoS* diterapkan pada lingkungan small-scale network berbasis *Virtual Machine* (VM). Sistem memanfaatkan Wazuh sebagai platform log-based monitoring yang memiliki kemampuan mendeteksi anomali secara *real-time* dan menjalankan *Active Response* secara otomatis [12]. Wazuh membaca log trafik

pada gateway mitigasi, titik dimana seluruh trafik penyerang dan trafik sah melewati jalur yang sama untuk mendeteksi pola anomali berbasis *threshold*. Ketika anomali terdeteksi, Wazuh memicu Tindakan *blackhole routing* untuk menjatuhkan seluruh trafik berbahaya sehingga server target tetap dapat diakses oleh *client* sah.

Dengan mengintegrasikan *anomaly detection* berbasis *threshold* dan mekanisme *blackhole routing* otomatis, sistem ini diharapkan mampu mendeteksi dan memitigasi serangan secara cepat tanpa campur tangan manual. Fokus penelitian ini mencakup tiga jenis serangan *DDoS* berbasis trafik yang paling umum, yaitu *SYN Flood*, *UDP Flood*, dan *ICMP Flood* [13]. Berdasarkan uraian tersebut, penelitian ini menekankan perancangan dan implementasi sistem otomatisasi respons *DDoS* secara *real-time* yang tidak hanya mampu mendeteksi serangan, tetapi juga langsung mengeksekusi mitigasi untuk mencegah gangguan layanan

1.2 Rumusan Masalah

Permasalahan utama dalam penelitian ini adalah bagaimana merancang sebuah sistem yang mampu mendeteksi dan merespons serangan *Distributed Denial of Service (DDoS)* secara otomatis dan *real-time*.

1.3 Tujuan

Tujuan utama penelitian ini adalah merancang dan mengimplementasikan sistem deteksi dan mitigasi otomatis terhadap sernagan *DDoS* menggunakan Wazuh dan mekanisme *blackhole routing*. Sistem ini diharapkan mampu mendeteksi anomali trafik secara *real-time* dan secara otomais membuang trafik berbahaya, sehingga ketersediaan layanan tetap terjaga tanpa mengganggu trafik sah.

1.4 Ruang Lingkup

Penelitian ini difokuskan pada perancangan dan implementasi sistem otomatisasi respons serangan *DDoS* pada lingkungan small-scale network berbasis simulasi menggunakan *Virtual Machine* (VM). Sistem deteksi dan monitoring menggunakan Wazuh, yang berfungsi untuk mengumpulkan log, mendeteksi anomali trafik, dan menjalankan tindakan otomatis melalui fitur *Active Response*.

Proses deteksi dibatasi pada metode *anomaly detection* berbasis *threshold* yang ditentukan berdasarkan pola trafik jaringan selama pengujian. Untuk mitigasi, penelitian ini

menerapkan mekanisme *black hole routing* pada *Linux Gateway*, yaitu Teknik pengalihan trafik berbahaya ke *null route* agar tidak membebani server target.

Seluruh implementasi dilakukan pada jaringan berskala kecil menggunakan VM sehingga penelitian tidak melibatkan router fisik, perangkat jaringan berkapasitas besar, maupun konfigurasi kompleks seperti *BGP blackholing*.

Pengujian serangan difokuskan pada tiga jenis serangan *DDoS* berbasis trafik, yaitu *SYN Flood*, *UDP Flood*, dan *ICMP Flood*, karena ketiganya merupakan serangan umum yang relevan untuk simulasi *DDoS* pada jaringan kecil.

1.5 Ekspektasi Hasil

Hasil yang diharapkan dari penelitian ini adalah tersusunnya arsitektur sistem yang mampu mengintegrasikan proses monitoring, deteksi anomali, dan mitigasi otomatis pada lingkungan jaringan berskala kecil. Sistem monitoring menggunakan platform Wazuh diharapkan dapat mengumpulkan log trafik dari *Linux Gateway*, menganalisis pola aktivitas jaringan, serta mendeteksi anomali berbasis *threshold* yang ditentukan dari perilaku trafik selama pengujian. Ketika terdeteksi adanya pola serangan *DDoS*, sistem diharapkan dapat menjalankan mekanisme mitigasi otomatis melalui fitur *Active Response* dengan mengeksekusi perintah *black hole routing* pada *Linux Gateway* sehingga trafik berbahaya dialihkan ke *null route* dan tidak mencapai server target.

Seluruh proses deteksi dan mitigasi ini diharapkan berlangsung secara *real-time* sehingga layanan pada server tetap dapat diakses oleh pengguna sah meskipun terjadi serangan. Lingkungan uji coba dirancang dalam skala kecil menggunakan *Virtual Machine* yang terdiri dari server target, node penyerang, node pengguna normal, serta *Linux Gateway* sebagai modul mitigasi utama. Dari hasil pengujian, diharapkan diperoleh evaluasi yang menunjukkan efektivitas sistem dalam membedakan trafik normal dan trafik berbahaya, serta kemampuan mitigasi untuk menjaga ketersediaan layanan. Seluruh proses mulai dari perancangan, implementasi, hingga pengujian sistem akan terdokumentasikan secara sistematis dalam laporan Tugas Akhir ini.

1.6 Sistematika Penyajian

Laporan Tugas Akhir ini disusun secara sistematis agar memudahkan pembaca dalam memahami keseluruhan isi penelitian. Adapun sistematika penyajian materi dalam laporan ini adalah sebagai berikut:

1. **BAB I PENDAHULUAN**

   Bab ini berisi latar belakang, rumusan masalah, tujuan, lingkup kajian, batasan penelitian, hasil yang diharapkan, serta sistematika penyajian laporan. Bab ini memberikan gambaran awal mengenai dasar pemikiran, arah, dan tujuan penelitian yang dilakukan.

2. **BAB II TINJAUAN PUSTAKA**

   Pada bab ini membahas teori-teori yang mendasari penelitian, meliputi konsep dasar serangan *Distributed Denial of Service (DDoS)*, teknik *anomaly detection*, *black hole routing*, serta penelitian-penelitian terdahulu yang relevan. Bab ini menjadi landasan konseptual dalam mengembangkan sistem yang diusulkan.

3. **BAB III METODOLOGI PENELITIAN**

   Bab ini menjelaskan strategi, pendekatan, serta tahapan penelitian yang digunakan, termasuk perancangan arsitektur sistem, skenario simulasi, dan metode pengujian yang dilakukan.

4. **BAB IV HASIL DAN PEMBAHASAN**

   Pada bab ini menyajikan hasil implementasi sistem, hasil pengujian dengan simulasi serangan *DDoS*, serta analisis efektivitas mitigasi yang dilakukan. Pada bab ini juga dibahas interpretasi hasil dan kaitannya dengan tujuan penelitian.

5. **BAB V KESIMPULAN DAN SARAN**

   Bab ini berisi rangkuman hasil penelitian yang telah dicapai, keterbatasan penelitian, serta saran untuk pengembangan lebih lanjut di masa mendatang.

Dengan sistematika tersebut, diharapkan laporan ini dapat tersaji secara runtut dan memberikan pemahaman yang jelas mengenai proses penelitian dari awal hingga akhir.

BAB II
TINJAUAN PUSTAKA

Bab ini menyajikan landasan teori dan kajian literatur yang mendukung penelitian mengenai *Real-time Automation System for DDoS Based on Anomaly Detection and Route Nullification*. Isi dari bab ini meliputi uraian tentang konsep serangan *DDoS,* metode deteksi berbasis *anomaly detection*, teknik mitigasi menggunakan *black hole routing*, serta penelitian terdahulu yang relevan.

2.1 *Distributed Denial of Service (DDoS)*

Pada sub bab ini akan dijelaskan lebih spesifik mengenai defenisi, cara kerja, jenis serangan, cara kerja, dampak, dan mitigasi dari *DDoS (Distributed Denial of Service)*

2.1.1 Defenisi dan Konsep Dasar *Distributed Denial of Service (DDoS)*

Distributed Denial of Service (DDoS) adalah jenis serangan siber yang bertujuan membuat suatu layanan, aplikasi, atau infrastruktur jaringan tidak dapat diakses oleh pengguna sah dengan cara membanjiri target menggunakan lalu lintas berlebihan dari banyak sumber terdistribusi. Serangan ini berasal dari kumpulan perangkat yang terlah dikompromikan (botnet) dan dikendalikan oleh penyerang untuk menghasilkan trafik dalam jumlah besar dalam bersamaan. Menurut [1], karakteristik utama *DDoS* adalah penggunaan pola trafik anomali yang sering menyerupai trafik normal sehingga sulit dibedakan oleh mekanisme keamanan tradisional.

Serangan *DDoS* merupakan ancaman nyata bagi layanan digital modern. Serangan terhadap Dyn DNS pada Oktober 2016 menyebabkan plaform besar seperti Twitter, Netflix, dan Spotify tidak dapat diakses selama berjam-jam. Selanjutnya, Amazon Web Service (AWS) pada tahun 2020 melaporkan serangan *DDoS* terbesar yang pernah tercatat dengan intensitas mencapai 2.3 Tbps. Insiden-insiden ini menunjukkan bahwa *DDoS* bukan hanya gangguan teknis, tetapi dapat menimbulkan kerugian besar pada sektor pemerintahan, finansial, hingga layanan publik.

Secara konseptual, serangan *DDoS* terdiri dari 3 komponen utama:

1. Attcker/Master, yang mengendalikan eksekusi serangan
2. Handler/Command Server, yang memberikan instruksi kepada agent

3. Agent/Bot, yaitu perangkat yang menjalankan serangan dengan mengirimkan trafik berbahaya ke target.

Arsitektur terdistribusi ini membuat serangan sulit dihentikan, karena sumber trafik berasal dari banyak IP yang berbeda dan tersebar secara global. Menurut [12], serangan *DDoS* menjadi lebih kompleks ketika trafik botnet dikaburkan agar menyerupai trafik sah, sehingga mekanisme seperti *firewall* statis dan *signature-based IDS* sering gagal mengenalinya.

Penelitian [14] menambahkan bahwa serangan *DDoS* semakin berbahaya pada ekosistem Internet of Things (IoT), karena perangkat IoT umumnya memiliki keamanan yang rendah sehingga mudah diambil alih dan dijadikan botnet dalam skala besar. Hal ini meningkatkan kapasitas serangan dan memperluas permukaan ancaman.

Dampak utama serangan *DDoS* berada pada aspek availability atau ketersediaan layanan. Seperti ditegaskan oleh [15], serangan *DDoS* menyebabkan server mengalami overload, meningkatkan latensi, menurunkan throughput, dan pada kondisi ekstrem membuat layanan berhenti beroperasi sepenuhnya. Serangan ini juga menimbulkan efek lanjutan berupa paket antrean yang menumpuk, konsumsi *CPU* dan memori berlebihan, hingga kerusakan pada perangkat jaringan.

Dengan demikian, *DDoS* bukan hanya sekedar serangan berbasis trafik dalam jumlah besar, tetapi merupakan serangan kompleks yang memanfaatkan struktur jaringan terdistribusi, penyamaran pola trafik, dan perangkat botnet global untuk melumpuhkan target secara berkelanjutan. Kompleksitas inilah yang menjadikan *DDoS* sebagai salah satu ancaman paling sulit ditangani pada infrastruktur jaringan modern.

2.1.2 Cara Kerja Serangan *Distributed Denial of Service (DDoS)*

Serangan *Distributed Denial of Service (DDoS)* bekerja dengan membanjiri target menggunakan lalu lintas berbahaya dalam jumlah besar yang berasal dari banyak perangkat secara terdistribusi. Untuk memahami bagaimana serangan dilakukan, diperlukan penjelasan mengenai tahapan ekrja utama *DDoS* sebagai berikut:

1. **Rekrutmen dan Pembentukan Botnet**

   Tahap awal serangan *DDoS* adalah pembentukan botnet, yaitu kumpulan perangkat yang telah terinfeksi malware dan dapat dikendikan oleh penyerang. Botnet dapat terdiri dari computer, server, router, hingga perangkat Internet of Things (IoT). Menurut [16], perangkat IoT merupakan sasaran empuk karena memiliki kemampuan keamanan yang rendah, konfigurasi sederhana, dan sering tidak diperbarui, sehingga mudah direkrut sebagai bagian dari botnet berskala besar.

2. **Koordinasi Melalui Command and Control (C&C)**

   Setelah botnet terbentuk, penyerang mengelola perangkat-perangkat tersebut menggunakan infrastruktur Command and Control (C&C). Melalui C&C, penyerang dapat mengirimkan instruksi kepada seluruh perangkat zombie (bot) untuk melakukan serangan secara serentak. Struktur ini memungkinkan serangan terdistribusi dalam skala masif, sehingga trafik berbahaya yang dihasilkan sulit dibedakan dari trafik sah.

3. **Eksekusi Serangan: Pengiriman Lalu Lintas Berlebihan**

   Ketika komando serangan diberikan, ribuan hingga jutaan bot mulai mengirimkan trafik ke target secara bersamaan. Bentuk trafik yang dikirimkan dapat bervariasi sesuai teknik serangan, antara lain:

   - *Volumetric Attacks (UDP Flood, ICMP Flood)*

     Menyebabkan konsumsi *bandwidth* ekstrem sehingga jalur komunikasi penuh
   - *Protocol Exploitation (SYN Flood)*

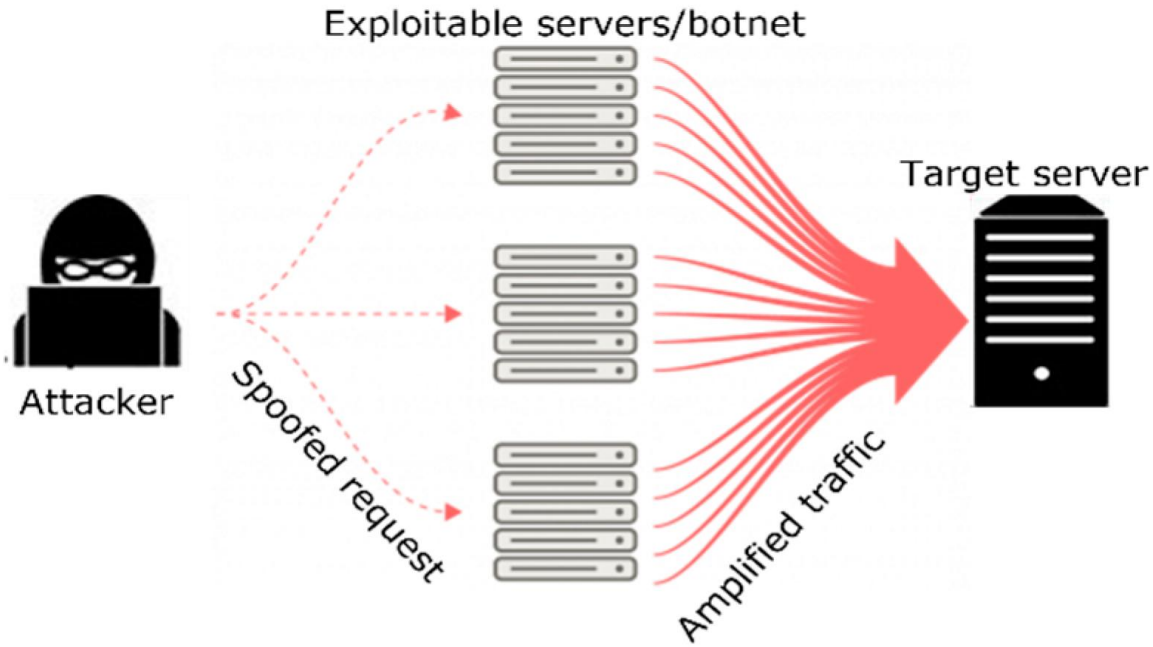
     Mengekspoitasi kelemahan protokol *TCP* dengan membuka koneksi setelah jadi (half-open), menghabiskan sumber daya server.
   - *Application Layer Attacks (HTTP GET/POST Flood)*

     Menyerang layer 7 dengan mengirimkan permintaan *HTTP* palsu menggunakan pola mirip pengguna sah [17].

   Jenis-jenis trafik ini dirancang agar tidak hanya membebani jaringan, tetapi juga mempersulit sistem keamanan dalam mengidentifikasi serangan secara cepat.

4. **Target Mengalami Gangguan hingga Tidak Tersedia**

   Ketika jumlah trafik yang masuk melebihi kapasitas prmprosesan atau *bandwidth*, layanan yang diserang mulai mengalami penurunan kinerja, seperti peningkatan latensi, konksi terputus, hingga downtime total. Mekanisme keamanan tradisional seperti *firewall* statis atau *signature-based IDS* sering gagal mengenali serangan *DDoS* secara *real-time* karena pola trafik serangan sering menyamar sebagai trafik normal.



Gambar 2. 1 Cara Kerja *Distributed Denial of Service*
(sumber:[18])

Dengan memahami tahapan tersebut, dapat disimpulkan bahwa *DDoS* merupakan serangan yang kompleks karena memanfaatkan arsitektur terdistribusi, botnet berskala besar, dan pola trafik yang sulit dibedakan dari tarfik normal. Hal ini membuat deteksi dan mitigasi *DDoS* membutuhkan pendekatan yang lebih adaptif dan otomatis.

2.1.3 Jenis – Jenis Serangan *Distributed Denial of Service (DDoS)*

Serangan *Distributed Denial of Service (DDoS)* dapat dibagi menjadi beberapa kategori berdasarkan teknik yang digunakan untuk menghabiskan sumber daya target. Masing-masing jenis serangan memanfaatkan kelemahan berbeda pada jaringan, server, atau aplikasi. Beberapa jurnal menyebutkan bahwa serangan *DDoS* modern semakin sulit dibedakan dari trafik normal karena pola serangannya semakin variatif [1] [3].

Secara umum, serangan *DDoS* diklasifikasian menjadi empat kategori utama:

1. **Volumetric Attacks (Serangan Berbasis Volume)**

   Serangan ini bertujuan untuk menghabiskan *bandwidth* dengan mengirimkan lalu lintas dalam jumlah sangat besar. Botnet akan membanjiri jaringan target sehingga tidak ada kapasitas yang tersisa untuk pengguna sah.

   Contoh serangan volumetrik:

   - *UDP Flood*, mengirimkan UDP besar ke port acak yang memaksa target mengirimkan respons "port unreachable", membebani sumber daya jaringan [8].
   - *ICMP Flood* (Ping Flood), mengirimkan echo request secara massif sehingga kapasitas jaringan penuh [9].

   Penelitian menunjukkan bahwa serangan volumetric menjadi favorit botnet karena skalanya dapat diperbesar dengan mudah melalui perangkat IoT yang telah terinfeksi.

2. **Protocol Attcks (Serangan Berbasis Protokol)**

   Serangan ini mengekspoitasi kelemahan protokol jaringan untuk menghabiskan sumber daya seperti tabel koneksi, memori, atau *CPU*. Serangan protokol cenderung tidak terlalu besar secara *bandwidth*, tetapi sangat mematikan bagi server.

   Contoh serangan protokol:

   - *SYN Flood*, mengirimkan banyak paket *SYN* tanpa menyelesaikan handshake TCP sehingga tabel koneksi menjadi penuh (half-open synchronization) [9].
   - Smurf Attack, menggunakan spoofing dan broadcast address agar banyak host mengrimkan balasan ke target [1].

3. **Application Layer Attacks (Serangan Lapisan Aplikasi)**

   Serangan ini menargetkan layer 7 (lapisan aplikasi), yaitu bagian yang berhubungan langsung dengan layanan pengguna. Serangan terjadi melalui permintaan yang tampak sah tetapi dikirimkan dalam jumlah besar atau dengan startegi perlambatan (*slow-rate*).

   Contoh serangan aplikasi:

   - *HTTP* GET/POST, mengirim permintaan *HTTP* berulang hingga server kewalahan memproses permintaan

- Slowloris, menjaga koneksi tetap terbuka dengan mengirimkan header tidak lengkap sehingga server menghabiskan resource untuk menunggu koneksi [5].

Serangan lapisan aplikasi sangat berbahaya karena pola trafiknya sangat mirim dengan permintaan sah dan sulit dideteksi menggunakan *firewall* atau *IDS* berbasis *signature* [4] [5].

4. **Amplification Attacks (Serangan Amplifikasi)**

   Jenis serangan ini menggunakan server pihak ketika (reflector) untuk memperbesar volume serangan. Penyerang mengirim permintaan kecil dengan alamat IP target (spoofing), kemudian server akan mengirim respons jauh lebih besar kepada target. Contoh Aplifikasi:

   - DNS Amplification
   - NTP Amplification
   - SSDP/CLDAP Amplification

   Penelitian menunjukkan bahwa serangan amplifikasi berbahaya karena mampu menghasilkan serangan dalam skala puluhan hingga ratusan gigabit per detik, bahkan mencapai terabit pada serangan besar seperti AWS 2020 [3].

2.1.4 Jenis Serangan yang Diuji

Pengujian dalam penelitian ini difokuskan pada tiga jenis serangan serangan *DDoS* berbasis trafik, yaitu *SYN Flood*, *UDP Flood*, dan *ICMP Flood*. Ketiganya dipilih karena merupakan serangan paling umum pada jaringan tradisional maupun modern, serta digunakan sebagai skenario pengujian dalam studi *DDoS* karena struktur serangannya generik dan mudah direplikasi untuk simulasi [13]. Selain itu, ketiga serangan ini mampu menggambarkan variasi serangan volumetric dan protokol yang relevan untuk pengujian di lingkungan jaringan berskala kecil.

1. ***SYN Flood***

   SYN Flood termasuk dalam kategori protocol attacks karena mengeksploitasi proses TCP three-way handshake, yang merupakan mekanisme standar untuk membangun koneksi antara *client* dan server.

   Proses handshake normal terdiri dari:

   - *SYN*, *client* meminta membuka koneksi

- *SYN-ACK*, server membalas sebagai persetujuan koneksi
- *ACK*, *client* mengonfirmasi, koneksi terbentuk

Dalam serangan *SYN Flood*, penyerang mengirimkan banyak paket *SYN* menggunakan alamat IP palsu (spoofed) dan tidak pernah mengirimkan ACK, sehingga koneksi tetap dalam kondisi half-open. Akibatnya:

- Tabel koneksi server penuh
- Memori server habis untuk menunggu koneksi yang tidak pernah selesai
- Pengguna sah tidak dapat membuka koneksi baru

Penelitian menunjukkan bahwa *SYN Flood* merupakan salah satu serangan paling efektif untuk menonaktifkan layanan berbasis TCP karena memanfaatkan kelemahan protokol secara langsung [2] [8].

2. UDP Flood

UDP Flood adalah serangan volumetric, yaitu menyerang target dengan paket UDP dalam jumlah besar menuju port acak atau port tertentu. Serangan ini memanfaatkan karakteristik UCP yang connectionless, sehingga:

- Penyerang dapat mengirim paket dengan sangat cepat tanpa perlu membangun sesi
- Server dipaksa merespons setiap paket menuju port tertutup dengan pesan *ICMP* Destination Unreachable
- Hai ini menyebabkan konsumsi *bandwidth* dan *CPU* meningkat drastic

Dalam banyak penelitian, *UDP Flood* sering dipilih sebagai skenario dasar pengujian *DDoS* karena dapat dengan cepat menghasilkan trafik besar menggunakan botnet atau perangkat IoT yang terinfeksi [8].

3. ICMP Flood

ICMP Flood atau Ping Flood merupakan serangan yang mengirimkan paket *ICMP* Echo Request dalam jumlah besar ke target. Setiap permintaan *ICMP* normalnya akan memicu target untuk mengirim *ICMP* Echo Reply. Ketika jumlah permintaan sangat besar:

- *Bandwidth* jaringan akan penuh
- *CPU* dan memori server meningkat drastic
- Terjadi network congestion yang membuat layanan tidak responsive

Serangan *ICMP Flood* sering digunakan dalam penelitian karena sederhana tetapi efektif untuk mengevaluasi ketahanan jaringan terhadap serangan berbasis *bandwidth*.

2.1.5 Perbedaan Trafik Legal dan Trafik Ilegal

Dalam konteks keamanan jaringan, trafik legal *(legitimate traffic)* adalah lalu lintas data yang dihasilkan oleh pengguna sah sesuai kebutuhan layanan, sedangkan trafik ilegal (attack legal) adalah tarfik yang menyimpang dari pola normal dan bertujuan mengganggu, mengeksploitasi, atau melumpuhkan layanan. Kemampuan membedakan kedua jenis trafik ini merupakan dasar dari mekanisme deteksi anomali dan mitigasi *DDoS.*

Trafik legal umumnya memiliki pola teratur, bersumber dari pengguna dengan IP sah, serta menggunakan protokol secara wajar sesuai perilaku pengguna. Pola permintaan (request pattern) cenderung bervariasi mengikuti jam sibuk, tetapi tetap berada dalam rentang kapasitas layanan yang normal.

Sebaliknya, trafik ilegal banyak ditemukan pada serangan *Distributed Denial of Service (DDoS).* Trafik ini sering bersifat abnormal, seperti lonjakan paket per detik yang tiba-tiba, volume koneksi yang tidak wajar, atau penggunaan protokol tertentu secara berlebihan. Penelitian ini menunjukkan bahwa berbagai serangan *DDoS* memanfaatkan celah protokol seperti *TCP*, *UDP*, dan *ICMP* untuk membanjiri target dengan request palsu sehingga layanan menjadi tidak responsive [8] [9]. Studi terkini juga menekankan bahwa perbedaan mendasar antara trafik legal dan ilegal dapat diidentifikasi melalui parameter statistic seperti packet rate, jumlah connection attempts, serta karakteristik payload [5] [7] [9].

Dalam sistem *anomaly detection*, karakteristik baseline trafik normal digunakan sebagai acuan. Lalu lintas yang menyimpang signifikan dari pola normal akan dikategorikan sebagai trafik ilegal atau indikasi serangan [19]. Dengan demikian, pemisahan tarfik legal dan ilegal sangat penting agar sistem tidak salah mengklasifikasikan pengguna sah (false positive) maupun gagal mendeteksi serangan (false negative).

Tabel 2. 1 Perbedaan Trafik Legal dan Trafik Ilegal

Aspek	Trafik Legal	Trafik Ilegal
Sumber Trafik	Pengguna sah dengan IP sah, autentik	Botnet, IP Spoofing, perangkat terinfeksi malware
Pola Distribusi	Fluktuatif sesuai perilaku pengguna & jam sibuk	Seragam, massif, meningkat tiba-tiba (burst)
Frekuensi dan Volume	Sesuai kapasitas layanan, tidak berlebihan	Dominan satu jenis paket: *SYN*, *UDP*, *ICMP* dalam jumlah abnormal
Tujuan Trafik	Mengakses layanan secara sah	Membanjiri, mengeksploitasi atau melumpuhkan target
Dampak ke sistem	Sistem stabil, resource terkendali	*CPU*, memori, dan *bandwidth* overload, layanan terganggu

Dengan demikian, perbedaan antara trafik legal dan illegal menjadi landasan penting dalam perancangan sistem deteksi *DDoS* berbasis *anomaly detection*. Sistem yang mampu mengidentifikasi penyimpanan pola tarfik secara akurat akan meningkatkan efektivitas mitigasi tanpa mengganggu akses pengguna sah.

2.1.6 Dampak Serangan *DDoS*

Serangan *Distributed Denial of service (DDoS)* menimbulkan dampak luas tidak hanya pada aspek teknis, tetapi juga pada ekonomi, reputasi, dan kemanan infrastruktur. Dampak ini dapat terjadi pada organisasi kecil hingga infrastruktur nasional bersifat kritis. Beberapa dampak spesifik dari serangan *DDoS* berdsarkan penelitian sebagai berikut:

1. **Gangguan Ketersediaan Layanan *(Service Unavailability)***

   Dampak paling langsung dari serangan *DDoS* adalah tidak tersedianya layanan bagi pengguna sah. Lonjakan lalu lintas yang melebihi kapasitas menyebabkan:

   - Server tidak mampu merespons permintaan
   - Aplikasi menjadi lambat atau tidak responsif
   - Layanan benar-benar berhenti beroperasi (total outage)

Penelitian [1] menegaskan bahwa trafik anomali dalam jumlah besar membuat server kewalahan memproses koneksi baru. Hal ini sejalan dengan temuan [14] yang

mendokumentasikan gangguan pada sistem layanan publik di Indonesia akibat serangan *DDoS*, sehingga efektivitas pemerintahan dan pelayanan masyarakat ikut terhenti.

2. **Penurunan Performa dan Stabilitas Jaringan**

   DDoS tidak hanya menyebabkan layanan tidak tersedia, tetapi juga mengacaukan stabilitas jaringan secara keseluruhan. Dampaknya mencakup:

   - Meningkatnya latensi
   - Hilangnya paket (packet loss)
   - Kemacetan (congestion)
   - Turunnya throughput komunikasi

Penelitian [19] menunjukkan bahwa kelebihan trafik dapat menyebabkan jaringan memasuki kondisi anomali, bahkan memicu *blackhole routing* tidak terencana, sehingga paket sah ikut terbuang. Studi lain [9] memperlihatkan bahwa dalam jaringan militer, serangan *DDoS* mampu menurunkan reliabilitas jaringan hingga memengaruhi operasi penting.

3. **Kerugian Finansial dan Operasional**

   Serangan *DDoS* memiliki dampak ekonomi yang besar, terutama pada organisasi yang menyediakakan layanan berbasis online. Kerugian finansial ini muncul dari:

   - Hilangnya pendapatan selama downtime
   - Biaya pemulihan pasca-serangan
   - Biaya tambahan untuk meningkatkan kapasitas keamanan

Menurut [5], downtime layanan menyebabkan kerugian ekonomi lansgsung bagi bisnis digital. Hal ini diperkuat oleh [11] yang menekankan bahwa dalam lingkungan cloud dan IoT, serangan *DDoS* dapat mengganggu operasional bisnis dan menyebabkan kerugian jangka panjang karena layanan tidak dapat digunakan oleh pelanggan.

4. **Kerusakan Reputasi dan Penurunan Kepercayaan Pengguna**

   Organisasi yang tidak mampu menangani serangan *DDoS* secara efektif berisiko mengalami penurunan reputasi. Dampaknya mencakup:

   - Menurunnya tingkat kepercayaan pelanggan
   - Kekhawatiran terhadap keamanan layanan
   - Citra perusahaan menurun di mata publik

2.2 Metode Mitigasi Serangan *DDoS*

Serangan *DDoS* sudah menjadi ancaman serius terhadap ketersediaan layanan jaringan. Dampak serangan ini sangat merugikan karena dapat menghentikan layanan, merugikan ekonomi, bahkan mengganggu reputasi penyedia layanan. Oleh karena itu, berbagai metode mitigasi telah dikembangkan, mulai dari teknik sederhana berbasis aturan statis hingga pendekatan yang memanfaatkan *machine learning*.

Menurut [20], mitigasi *DDoS* pada dasarnya bertujuan untuk menjaga availability dalam CIA *Triad* dengan cara meminimalkan dampak serangan sekaligus mempertahankan akses trafik sah. Namun, karena pola serangan *DDoS* bersifat dinamis, metode mitigasi harus adaptif, bukan sekedar memanfaatkan pendekatan [5].

2.2.1 Rate Limiting

Rate Limiting adalah teknik mitigasi yang membatasi jumlah permintaan *(request)* atau paket dari suatu sumber dalam waktu tertentu. Tujuan utamanya adalah untuk mencegah server mengalami kelebihan beban akibat lonjakan trafik, baik yang bersifat sah maupun serangan.

Dengan menerapkan *rate limiting*, paket atau permintaan yang melebihi ambang batas (*threshold*) yang ditentukan akan ditolak atau ditunda, sehingga server memiliki kapasitas untuk merespons permintaan sah. Teknik ini umumnya digunakan pada *firewall*, API gateway, load balancer, dan web server.

Kelebihan Rate Limiting

- Implementasi sederhana, dapat diterapkan pada server, *firewall*, atau router
- Efektif terhadap serangan volumetrik skala kecil hingga menengah , terutama jika datang dari sumber tunggal
- Mengurangi beban server secara langsung dengan menahan atau membuang trafik berlebih

Kelemahan Rate Limiting

- Berpotensi memblokir trafik sah jika ambang batas ditentukan terlalu rendah, menyebabkan false positive
- Tidak efektif menghadapi serangan *DDoS* terdistribusi, karena trafk berbahaya tersebar ke banyak sumber sehingga melewati batas rate limiter [5]
- Tidak mampu mendeteksi pola serangan kompleks, karena hanya mengandalkan batas numerik statis tanpa memahami perilaku trafik [8]

Penelitian modern menegaskan bahwa meskipun rate limiting efektif sebagai pertahanan awal, metode ini perlu dikombinasikan dengan teknik yang lebih adaptif seperti anomaly-based detection atau routing-based mitigation agar mampu menghadapi serangan *DDoS* berskala besar dan multi-verktor [5] [8].

2.2.2 IP Filtering

IP Filtering metode mitigasi *DDoS* yang dilakukan dengan menyaring dan memblokir trafik berdasarkan alamat IP sumber. Teknik ini bekerja dengan menolak paket yang berasal dari alamat IP tertentu yang telah diindikasi sebagai sumber serangan, sehingga hanya trafik dari alamat IP yang dianggap sah yang diperbolehkan mencapai target. Metode ini merupakan salah satu pendekatan paling sederhana dan umum digunakan pada *firewall*, router, dan perangkat keamanan jaringan dasar.

Dalam implementasinya, IP Filtering dapat dilakukan melalui daftar control akses (Access Control List/ACL), *firewall* rules, atau mekanisme penapisan paket lainnya. Ketika sebuah IP terindikasi menghasilkan trafik abnormal, perangkat kemanan dapat segera menambahkan aturan pemblokiran untuk mencegah trafik tersebut mencapai target.

Namun, pendekatan ini memiliki sejumlah kelemahan signifikan, terutama ketika menghadapai serangan *DDoS* modern. Serangan *DDoS* berskala besar biasanya melibatkan ribuan hingga jutaan sumber trafik yang berasal dari botnet terdistribusi. Selain itu, banyak penyerang menggunakan teknik IP spoofing, yaitu memalsukan alamat IP sumber agar terlihat seperti alamat IP sah. Dalam kondisi seperti ini, sistem sulit membedakan antar pengguna sah dan paket berbahaya, sehingga filtering menjadi tidak efektif.

Menurut [19], kelemahan utama IP Filtering terletak pada sifatnya yang reaktif, yaitu hanya dapat memblokir alamat IP setelah serangan terjadi dan sumbernya teridentifikasi. Pendekatan ini tidak adaptif terhadap pola serangan yang dinamis dan tidak mampu menganalisis perilaku trafik secara komprehensif. Penelitian lain juga menunjukkan bahwa filtering berbasis IP cenderung menghasilkan false positive, terutama ketika banyak IP sah yang memiliki pola penggunaan serupa dengan trafik penyerang [8].

Dengan demikian, IP Filtering hanya efektif pada serangan kecil atau serangan dari sumber tunggal. Pada serangan *DDoS* terdistribusi yang berskala besar, teknik ini perlu

dikombinasikan dengan metode mitigasi lain seperti anomaly-based detection atau balckhole routing untuk memberikan perlindungan yang efektif.

2.2.3 Load Balancing

Load balancing metode yang digunakan untuk mendistribusikan beban trafik ke beberapa server atau sumber daya secara merata agar tidak terjadi penumpukan pada satu server saja. Dengan membagi beban kerja ke banyak server, sistem dapat mempertahankan ketersediaan layanan meskipun terjadi peningkatan trafik yang tinggi. Strategi ini banyak digunakan pada data center, layanan cloud, dna aristektur web modern untuk menjaga stabilitas layanan.

Dalam konteks serangan *DDoS*, load balancing berfungsi sebagai pendekatan resiliency, yaitu meningkatkan ketahanan jaringan terhadap lonjakan trafik. Dengan mendistribusikan trafik ke beberapa node, serangan menjadi sulit melumpuhkan seluruh sitsem karena tidak satu server yang menjadi single point of failure.

Namun dekimian, load balancing bukanlah solusi mitigasi langsung terhadap *DDoS*, melainkan solusi arsitektural. Laod balancing tidak membedakan apakah trafik masuk adalah trafik sah atau trafik berbahaya. Jika volume serangan sangat besar dan terdisribusi, maka:

- Seluruh server di belakang load balancer tetap dapat mengalami kelebihan beban
- Kapasitas total sistem dapat tetap di lampaui
- Laod balancer sendiri dapat menjadi target dan mengalami bottleneck
- Tidak ada proses pemfilteran atau pembuangan trafik berbahaya

Selain itu, penelitian [21] menyebutkan bahwa load balancing memerlukan infrastruktur tambahan, seperti beberapa server backend dan perangkat load balancer, yang tidak selalu tersedia pada jaringan berskala kecil seperti pada penelitian ini. Biaya implementasi juga cukup tinggi dan biasanya hanya digunakan pada arsitektur enterprise atau cloud.

Dengan demikian, meskipun load balancing dapat meningkatkan ketahanan sistem terhadap peningkatan trafik, metode ini tidak menggantikan mekanisme deteksi dan mitigasi *DDoS*, serta harus dikombinasikan dengan teknik lain seperti anomaly-based detection atau *blackhole routing* agar lebih efektif dalam menghadapi serangan besar dan terdistribusi.

2.2.4 Intrusion Detection /Prevention System (IDS/IPS)

Intrusion Detection System (IDS) dan *Intrusion Prevention System (IPS*) merupakan bagian penting dalam mekanisme pertahanan jaringan modern. *IDS* berfungsi untuk mendeteksi aktivitas mencurigakan atau berbahaya, sedangkan *IPS* tidak hanya mendeteksi tetapi juga mengambil tindakan otomatis untuk mencegah serangan mencapai target.

Dalam konteks mitigasi *DDoS*, *IDS*/*IPS* tradisional umumnya menggunakan metode *signature-based*, yaitu mencocokkan pola paket dengan basis data tanda serangan *(signature)*. Pendekatan ini efektif untuk mendeteksi serangan yang telah dikenal, namun memiliki keterbatasan ketika menghadapi pola serangan baru yang tidak terdapat dalam basis data *signature*. Penelitian [5] menegaskan bahwa mekanisme *signature-based* tidak cukup untuk mendeteksi serangan *DDoS* yang bersifat dinamis dan berubah-ubah, terutama pada jaringan modern yang memiliki variasi trafik tinggi.
Selain itu, *IDS*/*IPS* sering mengalami kesulitan ketika menghadapi serangan *DDoS* berskala besar karena:

- volume trafik yang sangat tinggi dapat membebani modul inspeksi,
- paket serangan sering menyerupai trafik sah,
- sumber serangan berasal dari banyak alamat IP terdistribusi,
- pola serangan sulit dideteksi tanpa analisis perilaku yang lebih canggih [3] [8].

Beberapa studi menunjukkan bahwa *IDS*/*IPS* hanya efektif sebagai lapisan pertahanan awal, tetapi tidak memadai untuk mitigasi *DDoS* secara menyeluruh. Oleh karena itu, banyak penelitian merekomendasikan penggunaan *IDS* yang lebih adaptif, seperti sistem deteksi berbasis *anomaly detection*, *machine learning*, atau *threshold* dinamis [5] [8].

Wazuh, yang digunakan dalam penelitian ini, termasuk dalam kategori *Host-based Intrusion Detection System (HIDS)*. Wazuh mendeteksi anomali melalui analisis log, mendukung pembuatan aturan kustom, dan mampu menjalankan tindakan otomatis melalui mekanisme *Active Response*, yang sangat relevan untuk memicu mitigasi seperti *blackhole routing*. Dengan demikian, *IDS*/*IPS* tetap menjadi bagian penting dalam pertahanan berlapis, tetapi perlu dikombinasikan dengan mekanisme adaptif dan respons otomatis untuk menangani serangan *DDoS* secara efektif.

2.2.5 Anomaly-based Mitigation

Anaomaly-based Mitigation merupakan pendekatan mitigasi serangan *DDoS* yang mengandalkan deteksi perilaku trafik yang menyimpang dari pola normal (baseline) sebagai pemicu tindakan mitigasi otomatis. Tidak seperti metode tradisional yang hanya bergantung pada *signature* atau aturan statis, pendekatan ini menmungkinkan sistem untuk merespons serangan berdasarkan karakteristik trafik aktual, sehingga lebih adaptif terhadap variasi serangan modern.

Konsep utama dari metode ini adalah membandingkan parameter trafik—seperti packet rate, koneksi *TCP half-open,* atau frekuensi permintaan protokol tertentu—dengan *threshold* yang telah ditentukan. Jika nilai trafik melebihi *threshold* tersebut, sistem secara otomatis mengklasifikasikan kondisi tersebut sebagai anomali dan memicu mekanisme mitigasi. Penelitian [5] menunjukkan bahwa pendekatan anomaly-based lebih efektif dalam menghadapi serangan *DDoS* dengan pola tidak teratur dan serangan low-rate yang sering lolos dari sistem *signature-based*. Hal ini juga didukung oleh penelitian [8], yang menjelaskan bahwa pendekatan berbasis perilaku (*behavioral thresholding*) dapat beradaptasi dengan dinamika trafik jaringan secara *real-time*.

Kelebihan utama *anomaly-based mitigation* adalah kemampuannya dalam mendeteksi pola serangan baru (zero-day attacks), di mana *signature* belum tersedia. Metode ini tidak memerlukan basis data tanda serangan, tetapi bergantung pada pengamatan pola trafik jangka pendek dan jangka panjang. Dengan demikian, sistem dapat mengenali serangan lebih cepat meskipun bentuk atau teknik serangan berubah-ubah.

Namun, metode ini juga memiliki tantangan, terutama pada penentuan *threshold*. *Threshold* yang terlalu rendah berpotensi menghasilkan *false positive*, yaitu trafik sah dianggap sebagai serangan. Sebaliknya, *threshold* yang terlalu tinggi dapat menyebabkan serangan lolos (*false negative*). Oleh karena itu, banyak penelitian merekomendasikan penggunaan *threshold* dinamis yang menyesuaikan pola trafik jaringan [5] [8].

Dalam penelitian ini, *anomaly-based mitigation* digunakan sebagai mekanisme pemicu untuk mengaktifkan *blackhole routing*. Wazuh, sebagai platform deteksi yang digunakan, menganalisis log trafik pada gateway dan menghasilkan alert ketika pola anomali terdeteksi berdasarkan *threshold* yang ditentukan. Alert tersebut kemudian memicu *Active Response* yang menjalankan perintah otomatis untuk mengalihkan trafik berbahaya ke *null route.*

Dengan demikian, *anomaly-based mitigation* berfungsi sebagai lapisan deteksi sekaligus pemicu tindakan mitigasi untuk memastikan trafik sah tetap dapat diteruskan ke server target.

2.3 *Anomaly Detection*

Pada sub bab ini akan dibahas mengenai *anomaly detection* sebagai salah satu pendekatan penting dalam mendeteksi pola serangan *Distributed Denial of Service (DDoS). Anomaly detection* dipilih karena metode ini mampu mengenali trafik yang menyimpang dari kondisi normal, sehingga dapat mendeteksi serangan yang tidak tercatat dalam aturan statis. Pembahasan pada sub bab ini mencakup defenisi *anomaly detection*, proses kerja dalam sistem jaringan, dan berbagai pendekatan yang dapat digunakan baik tradisional berbasis *threshold* maupun metode modern berbasis *machine learning*. Dengan memahami konsep dasar dan mekanisme *anomaly detection*, penelitian ini mendapatkan landasan teori yang kuat untuk mengembangkan sistem deteksi dini terhadap serangan *DDoS* yang efisien.

2.3.1 Defenisi *Anomaly Detection*

Anomaly detection merupakan proses identifikasi pola perilaku yang menyimpang dari kondisi normal dalam suatu sistem jaringan. Dalam konteks keamanan jaringan, *anomaly detection* digunakan untuk mengenali lalu lintas tidak biasa yang berpotensi berbahaya, termasuk serangan *Distributed Denial of Service (DDoS). Anomaly detection* berfungsi membedakan trafik normal dari trafik abnormal berdasarkan deviasi perilaku jaringan, di mana penyimpangan pola sering kali menandakan adanya indikasi intrusi atau aktivitas serangan.

Pendekatan ini menjadi penting karena metode tradisional berbasis *signature*, seperti pada *Intrusion Detection System (IDS),* hanya mampu mendeteksi serangan yang sudah terdokumentasi sebelumnya. *Signature-based IDS* tidak efektif menghadapi serangan baru (zero-day attack) atau variasi serangan DDoS yang memodifikasi pola trafiknya. Penelitian [1] menegaskan bahwa *anomaly detection* mampu mendeteksi pola trafik abnormal, seperti lonjakan paket *UDP*, peningkatan koneksi *SYN*, atau intensitas *ICMP* yang tidak wajar, tanpa bergantung pada database *signature*.

Kelebihan utama *anomaly detection* adalah kemampuannya mengenali serangan baru yang belum memiliki pola yang jelas. Studi [3] menunjukkan bahwa sistem deteksi berbasis model cerdas, seperti *digital twin* dan pembelajaran adaptif, dapat meningkatkan efektivitas

deteksi terhadap serangan DDoS yang memiliki pola dinamis. Namun demikian, pendekatan *anomaly detection* juga memiliki kelemahan, terutama potensi terjadinya false positive ketika trafik sah secara tidak sengaja diklasifikasikan sebagai serangan, sebagaimana dijelaskan dalam penelitian [22].

Dengan demikian, *anomaly detection* dapat didefinisikan sebagai pendekatan yang mendeteksi aktivitas jaringan abnormal melalui perbandingan antara pola trafik aktual dengan baseline perilaku normal yang telah dipelajari. Dalam penelitian ini, *anomaly detection* menjadi fondasi utama dalam sistem deteksi dini untuk mengidentifikasi indikasi serangan *DDoS* secara adaptif sebelum mekanisme mitigasi otomatis melalui *blackhole routing* dijalankan.

2.3.2 Proses *Anomaly Detection*

Anomaly detection merupakan proses penting dalam mitigasi serangan DDoS karena berfungsi mengidentifikasi aktivitas jaringan yang menyimpang dari pola normal. Proses ini dimulai dengan memantau trafik jaringan secara *real-time* dan membandingkannya dengan baseline perilaku normal yang telah dibentuk sebelumnya. Jika terjadi penyimpangan signifikan—misalnya lonjakan *packet per second* (*PPS*), peningkatan koneksi *TCP half-open*, atau intensitas permintaan *ICMP* yang melampaui nilai wajar—sistem akan mengklasifikasikan trafik tersebut sebagai anomali. Menurut [11], anomali dapat dikenali melalui penyimpangan statistik yang muncul secara tiba-tiba pada parameter trafik.

Tahap pertama dalam *anomaly detection* adalah pengumpulan data trafik mencakup *PPS*, jenis protokol, ukuran paket, dan alamat IP sumber. Data ini kemudian digunakan untuk membangun baseline trafik normal, yang dihitung menggunakan parameter statistik seperti mean, standar deviasi, atau model time-series sederhana. Penelitian [13] menegaskan bahwa baseline yang dibentuk dengan akurat akan meningkatkan kemampuan sistem dalam membedakan antara aktivitas sah dan pola serangan.

Berikutnya, sistem melakukan perbandingan trafik aktual dengan *threshold* yang telah ditetapkan. Dalam penelitian ini digunakan pendekatan *threshold* dinamis berbasis acuan manual yang diperkuat oleh perhitungan statistik. Pendekatan ini sejalan dengan metode yang diperkenalkan oleh [4], yang memanfaatkan mekanisme adaptive *thresholding* untuk menyesuaikan batas deteksi berdasarkan perubahan pola trafik. Penelitian [8] juga

menjelaskan bahwa pendekatan *threshold* adaptif lebih efektif dalam mendeteksi anomali pada jaringan yang memiliki variasi trafik tinggi.

Jika trafik masih berada di bawah *threshold*, maka dikategorikan sebagai trafik normal. Namun apabila nilai parameter trafik melampaui *threshold*, maka sistem mengidentifikasikan kondisi tersebut sebagai anomali. Proses ini sangat penting terutama untuk mendeteksi serangan low-rate maupun *burst traffic* yang sulit dikenali oleh sistem berbasis *signature*. Penelitian [5] menunjukkan bahwa deteksi *real-time* terhadap pola anomali merupakan elemen kunci dalam sistem pertahanan modern.

Setelah anomali terdeteksi, hasil klasifikasi ini dijadikan trigger untuk memulai mekanisme mitigasi. Dalam konteks penelitian ini, hasil deteksi akan memicu *Active Response* pada Wazuh, yang kemudian menginstruksikan *Linux Gateway* untuk menerapkan *blackhole routing*. Dengan demikian, trafik berbahaya akan dialihkan ke *null route* sehingga tidak mencapai server target. Integrasi proses deteksi dan respons otomatis ini membuat sistem lebih adaptif dan efektif dalam menghentikan serangan sebelum mengganggu layanan.

2.3.3 Pendekatan *Anomaly Detection*

Anomaly detection dalam keamanan jaringan memiliki berbagai pendekatan yang digunakan untuk membedakan trafik normal dari trafik mencurigakan. Pemilihan pendekatan bergantung pada kebutuhan sistem, kapasitas komputasi, karakteristik jaringan, serta tingkat akurasi yang diharapkan. Pendekatan utama yang sering digunakan dalam penelitian mencakup metode berbasis statistik, *thresholding*, *machine learning*, dan *deep learning*.

1. **Pendekatan Statistical-Based**

   Pendekatan ini menganalisis parameter statistik dari trafik jaringan seperti mean, standar deviasi, variansi jumlah koneksi, entropi IP sumber, dan distribusi ukuran paket. Trafik dianggap anomali apabila nilai parameter menyimpang jauh dari pola historis (baseline). Metode ini relatif ringan secara komputasi, sehingga cocok untuk sistem sederhana, namun kurang efektif dalam mendeteksi serangan baru yang tidak memiliki pola jelas atau serangan low-rate dengan perubahan perlahan. Penelitian [11] menunjukkan bahwa pendekatan statistik mampu mendeteksi serangan low-rate melalui analisis distribusi trafik, tetapi tetap sensitif terhadap fluktuasi normal jaringan.

2. **Pendekatan *Threshold-Based***

   Threshold-based detection merupakan pendekatan paling umum untuk mendeteksi serangan DDoS dalam jaringan berskala kecil. Prinsipnya adalah membandingkan nilai trafik (misalnya packet per second, koneksi *TCP half-open*, atau volume byte per detik) dengan ambang batas tertentu. Jika nilai trafik melampaui *threshold*, sistem mengidentifikasi aktivitas tersebut sebagai anomali.

   Kelebihan metode ini adalah implementasinya sederhana dan membutuhkan komputasi yang rendah, sehingga sangat cocok untuk lingkungan seperti small-scale network berbasis *Virtual Machine*. Namun *threshold* statis memiliki kelemahan signifikan, seperti tidak adaptif terhadap trafik sah pada jam sibuk, sehingga berpotensi menghasilkan *false positive* maupun *false negative.*

   Penelitian mutakhir menunjukkan pergeseran dari *threshold* statis menuju *threshold* dinamis yang menyesuaikan kondisi trafik secara *real-time*. Penelitian [4] memperkenalkan *Threshold Moving Autoencoder*, yaitu mekanisme adaptif yang mengubah nilai *threshold* berdasarkan karakteristik trafik saat itu. Penelitian [8] juga mengembangkan *variable-trust threshold*, di mana ambang deteksi dihitung berdasarkan tingkat kepercayaan (trust value) setiap pengguna, dan terbukti meningkatkan akurasi deteksi hingga 98,83%.

Seluruh pendekatan threshold adaptif ini dapat diimplementasikan melalui mekanisme pemantauan bawaan Wazuh. Wazuh Manager melakukan pengumpulan, normalisasi, da analisis trafik berdasarkan log yang dikirimkan oleh Wazuh Agent, kemudian menerapkan rule-based detection sesuai nilai ambang batas yang ditetapkan. Ketika suatu pola trafik melampaui threshold yang ditentukan, misalnya lonjakan SYN, peningkatan paket UDP atau intensitas ICMP yang tidak wajar, Wazuh Indexer menyimpan alert tersebut dan Wazuh Manager dapat memicu Active Response untuk menjalankan tindakan mitigasi, yaitu penerapan blackhole routing pada Linux Gateway.

3. **Pendekatan *Machine Learning-Based***

   Pendekatan *machine learning* menggunakan algoritma supervised maupun unsupervised, seperti Support Vector Machine (SVM), Naive Bayes, atau K-Means Clustering, untuk mengklasifikasikan trafik normal dan anomali. Dengan mempelajari data historis, sistem dapat mengenali pola trafik yang kompleks.

Penelitian [11] menunjukkan bahwa Naive Bayes efektif mendeteksi serangan low-rate dengan tingkat akurasi yang baik.

Kelebihan ML-based detection:

- mampu mengenali pola non-linier,
- lebih adaptif daripada *threshold* statis,
- akurat pada dataset besar.

Kelemahan:

- memerlukan dataset yang representatif,
- perlu training time,
- tidak cocok untuk lingkungan dengan sumber daya terbatas.

4. Pendekatan *Deep Learning-Based*

Pendekatan ini merupakan pengembangan *machine learning* menggunakan model neural network yang lebih kompleks, seperti *Deep Neural Networks (DNN), Recurrent Neural Networks (RNN),* dan *LSTM Autoencoder*. Model ini mampu mengenali pola trafik yang sangat kompleks dan non-linear.

Penelitian [5] membuktikan bahwa *DNN* dapat meningkatkan performa deteksi intrusi *real-time* secara signifikan. Penelitian [13] menunjukkan bahwa *LSTM Autoencoder* efektif dalam mempelajari perilaku multivariat trafik jaringan dan mendeteksi anomali dengan akurasi tinggi.

Kelemahan *deep learning*:

- membutuhkan komputasi besar (*CPU/GPU*),
- membutuhkan dataset dalam jumlah besar,
- kurang cocok untuk implementasi small-scale network seperti penelitian ini.

2.4 *Black Hole Routing*

Pada sub bab ini akan dijelaskan mengenai *black hole routing* sebagai salah satu strategi penanganan lalu lintas berbahaya dalam konteks serangan *Distributed Denial of Service (DDoS)*. Pembahasan mencakup definisi, prinsip kerja, serta kelebihan dan kelemahan dari pendekatan ini. *Black hole routing* dipilih karena merupakan mekanisme yang sederhana, relatif mudah diimplementasikan, dan banyak digunakan baik pada lingkungan berskala kecil maupun besar. Dengan memahami konsep dasar dan keterbatasannya, dapat diperoleh

gambaran yang jelas mengenai peran black hole routing dalam sistem mitigasi *DDoS* serta pentingnya integrasi dengan mekanisme deteksi anomali agar lebih efektif.

2.4.1 Defenisi Black Hole Routing

Dalam konteks mitigasi serangan *Distributed Denial of Service (DDoS)*, salah satu mekanisme yang paling sederhana dan banyak digunakan adalah black hole routing atau *route nullification*. Prinsip dasarnya adalah mengalihkan trafik berbahaya menuju *null route*, yaitu rute tanpa *next-hop* sehingga seluruh paket yang melewati jalur tersebut akan dibuang secara otomatis tanpa diproses oleh server. Dengan cara ini, trafik serangan dapat diisolasi sebelum membebani infrastruktur utama, sehingga server tetap dapat mempertahankan ketersediaan layanan meskipun terjadi banjir trafik [9].

Implementasi black hole routing umumnya dilakukan pada dua pendekatan:

1. **Skala Kecil (Local Blackhole)**

   Pada lingkungan kecil seperti server Linux atau gateway lokal, black hole routing dapat diimplementasikan menggunakan tabel routing atau *firewall* seperti *iptables*. Metode ini efektif dalam lingkungan kecil karena dapat diaktifkan secara cepat dan tidak memerlukan perangkat tambahan.

   Contoh implementasi:

   ```
   ip route add blackhole <IP_penyerang>
   ```

2. **Skala Besar (RTBH - Remotely Triggered Black Hole)**

   Pada infrastruktur besar atau ISP, implementasi dilakukan menggunakan RTBH, yaitu mekanisme pemicu remote yang mengalihkan trafik berbahaya ke *null route* pada router upstream. Studi kinerja RTBH oleh [9] menunjukkan bahwa metode ini sangat efektif untuk menangani serangan volumetrik karena paket dihentikan di lapisan provider, bukan di jaringan internal.

Penelitian terkait perilaku black-hole routing di jaringan modern (misalnya segment routing dan SDN) juga menunjukkan bahwa teknik ini dapat mengurangi dampak serangan, namun membutuhkan akurasi deteksi agar trafik sah tidak ikut terbuang [7].

Kelebihan Black Hole Routing

- Implementasi cepat dan sederhana.
- Tidak memerlukan perangkat tambahan.

- Sangat efektif untuk menyerang volumetrik besar yang bertujuan menghabiskan *bandwidth*.

Kelemahan Black Hole Routing

- Bersifat *non-selective*: semua trafik menuju IP yang diblackhole akan hilang, termasuk trafik sah.
- Bergantung pada akurasi sistem deteksi serangan (rawan false positive).
- Tidak menganalisis pola trafik, hanya membuang berdasarkan IP atau prefix [7].

Karena itu, black hole routing harus digabungkan dengan mekanisme *anomaly detection* agar pemicu blackhole benar-benar berasal dari trafik berbahaya.

Dalam penelitian ini, *Linux Gateway* akan menjalankan black hole routing berdasarkan hasil deteksi Wazuh *Active Response*, sehingga pengalihan trafik dilakukan secara otomatis setelah anomali teridentifikasi.

2.4.2 Strategi Penanganan Black Hole Routing

Untuk mengatasi kelemahan *black hole routing* yang bersifat tidak selektif, beberapa strategi umum diterapkan dalam penelitian dan praktik modern:

1. **Integrasi *Black Hole Routing* dengan *Anomaly Detection***

   Pendekatan ini menjadi standar modern mitigasi DDoS. *Anomaly detection* menganalisis pola trafik, membandingkan dengan baseline, dan memutuskan apakah terjadi lonjakan abnormal seperti:

   - peningkatan *PPS (packet per second),*
   - koneksi *SYN* berulang,
   - *ICMP* request masif,
   - pola akses mencurigakan.

   Jika sistem mendeteksi anomali, barulah gateway memicu black hole routing. Integrasi ini menekan risiko salah buang trafik sah dan membuat mitigasi jauh lebih selektif [4].

2. **Menggunakan RTBH (Remotely Triggered Black Hole)**

   RTBH menawarkan fleksibilitas dan skalabilitas yang lebih baik dibandingkan black hole lokal. Dengan RTBH:

   - pemblokiran terjadi di router upstream (ISP),

- mitigasi berlangsung lebih cepat,
- beban tidak mencapai jaringan internal.

Penelitian [9] menegaskan bahwa RTBH secara signifikan mengurangi dampak volumetric attack karena paket dihentikan sebelum masuk ke jaringan target.

3. **Mengkombinasikan dengan rate limiting dan filtering**

   Beberapa penelitian menyarankan agar *black hole routing* tidak berdiri sendiri, tetapi dikombinasikan dengan teknik lain seperti:

   - Rate limiting untuk membatasi jumlah request tertentu,
   - Filtering (misalnya berdasarkan protokol atau port),
   - Classification untuk mengidentifikasi tipe trafik.

   Kombinasi ini memastikan *black hole routing* hanya dipicu ketika serangan benar-benar terverifikasi, sehingga false positive dapat diminimalkan [8].

2.4.3 Tantangan Implementasi Black Hole Routing

Meskipun *black hole routing* merupakan mekanisme mitigasi DDoS yang sederhana dan efisien, implementasinya tidak terlepas dari sejumlah tantangan teknis yang perlu diperhatikan agar tidak berdampak negatif pada layanan jaringan. Tantangan utama terletak pada sifatnya yang *non-selective*; ketika sebuah IP atau prefix diarahkan ke *null route*, seluruh trafik ke arah tersebut baik sah maupun berbahaya akan dibuang tanpa pengecualian. Risiko ini menjadi semakin besar jika mekanisme pemicu (trigger) tidak akurat atau jika *threshold* deteksi anomali kurang presisi.

Menurut penelitian [9], salah satu masalah penting dalam penerapan *black hole routing* adalah risiko *overblocking*, yaitu kondisi di mana trafik sah ikut terbuang akibat salah identifikasi atau konfigurasi yang kurang tepat. Pada lingkungan jaringan besar, seperti ISP atau backbone, kesalahan konfigurasi RTBH dapat menyebabkan gangguan layanan lintas wilayah. Penelitian lain menunjukkan bahwa *black hole routing* juga rentan terhadap serangan IP spoofing, di mana penyerang memalsukan alamat IP sah sehingga sistem secara keliru memasukkan IP tersebut ke *null route*, yang pada akhirnya merugikan pengguna legitimate.

Selain itu, penelitian [7] mengungkapkan bahwa pada jaringan modern berbasis segment routing atau arsitektur terdistribusi, keberadaan black hole yang tidak terkelola dapat menimbulkan ketidakseimbangan jalur (*path divergence*) atau menurunkan efisiensi routing global. Tantangan ini semakin kompleks dalam jaringan berskala besar dengan banyak router, di mana sinkronisasi state routing harus dilakukan secara presisi.

Tantangan lainnya mencakup:

- Kesalahan *threshold anomaly detection*, yang dapat memicu black hole secara prematur atau terlambat.
- Tidak adanya inspeksi konten, sehingga black hole hanya berbasis IP tanpa mempertimbangkan jenis trafik.
- Keterbatasan visibilitas, terutama ketika serangan berasal dari IP spoofed atau botnet dengan sumber yang tersebar luas.

Dengan memahami tantangan-tantangan ini, diperlukan integrasi antara *black hole routing* dengan sistem deteksi anomali yang andal agar mitigasi lebih selektif dan mengurangi risiko terbuangnya trafik sah.

2.4.4 Integrasi *Black Hole Routing* dengan Wazuh *Active Response*

Untuk mengatasi keterbatasan *black hole routing* yang bersifat statis dan tidak selektif, penelitian ini mengimplementasikan integrasi antara *black hole routing* dengan mekanisme otomatisasi melalui Wazuh *Active Response*. Integrasi ini memungkinkan sistem untuk melakukan mitigasi *DDoS* secara otomatis berdasarkan deteksi anomali *real-time*, sehingga *black hole routing* hanya diaktifkan ketika serangan benar-benar teridentifikasi.

Wazuh merupakan platform keamanan berbasis log yang mampu melakukan pengumpulan data, analisis perilaku, deteksi anomali, serta eksekusi tindakan otomatis (*active response*). Menurut penelitian opsional [12], Wazuh dapat menjalankan skrip mitigasi ketika mendeteksi pola trafik tidak wajar yang melampaui ambang batas tertentu. Mekanisme inilah yang dijadikan dasar integrasi sistem pada penelitian ini.

Proses integrasi bekerja dalam beberapa langkah:

1. *Linux Gateway* mencatat metrik trafik seperti jumlah paket per detik (*PPS*), protokol yang digunakan (*SYN*, *UDP*, *ICMP*), dan pola koneksi.
2. Wazuh Agent pada gateway mengirimkan log tersebut ke Wazuh Manager untuk dianalisis.

3. Wazuh Manager memproses log menggunakan aturan *anomaly detection*, kemudian menghasilkan alert ketika pola trafik melebihi *threshold* dinamis.
4. Ketika alert terpicu, Wazuh *Active Response* menjalankan skrip otomatis, misalnya:
5. ip route add blackhole `<IP_attacker>`
   sehingga IP penyerang segera diarahkan ke *null route*.
6. Setelah trafik kembali normal, administrator dapat menghapus aturan tersebut atau sistem dapat menjalankan skrip pemulihan otomatis.

Keunggulan integrasi Wazuh ini adalah:

- Mitigasi lebih selektif, karena black hole hanya berjalan berdasarkan deteksi anomali yang valid.
- Respon otomatis, mengurangi waktu jeda antara deteksi dan tindakan mitigasi.
- Monitoring terpusat, memungkinkan analisis pola serangan secara historis.
- Kompatibel dengan *small-scale network*, sesuai kebutuhan penelitian.

Dengan demikian, integrasi *black hole routing* dengan Wazuh *Active Response* menjadikan sistem mitigasi jauh lebih responsif, akurat, dan efisien dibandingkan penggunaan *black hole routing* secara manual atau statis.

2.5 Penggunaan *IPTables Rules* untuk Mitigasi DDoS

IPTables merupakan utilitas *firewall* bawaan Linux yang bekerja pada *kernel space* melalui modul Netfilter, sehingga mampu melakukan *packet filtering*, *Network Address Translation* (NAT), dan *packet mangling*. Dengan fleksibilitas aturan (rules) yang dapat dikonfigurasi, *IPTables* memungkinkan administrator jaringan mengatur lalu lintas masuk (*incoming*), keluar (*outgoing*), dan diteruskan (*forwarding*), menjadikannya salah satu mekanisme pertahanan dasar pada sistem berbasis Linux.

Dalam konteks mitigasi *Distributed Denial of Service (DDoS)*, *IPTables* banyak digunakan sebagai teknik penyaringan awal untuk membuang paket berbahaya. Mekanisme filtering ini sangat efisien karena diproses langsung pada level kernel. Penelitian [12] menunjukkan bahwa *IPTables* mampu memblokir serangan berbahaya pada jaringan IoT secara cepat berkat proses filtering yang terjadi sebelum paket mencapai aplikasi. Teknik tersebut efektif menangani serangan volumetrik seperti *UDP Flood* maupun eksploitasi protokol seperti *SYN Flood* dengan menetapkan batas jumlah koneksi terhadap port tertentu atau memblokir paket dengan pola mencurigakan.

Selain filtering, *IPTables* juga dapat digunakan sebagai mekanisme *black hole routing* skala kecil, tanpa memerlukan perangkat router khusus. Administrator dapat menambahkan aturan yang mengarahkan IP penyerang ke *null route*, sehingga paket langsung dibuang sebelum mencapai server target. Pendekatan ini mendukung mitigasi cepat pada lingkungan jaringan berskala kecil atau menengah.

Penelitian modern memperluas peran *IPTables* melalui integrasi analisis berbasis data. Studi dalam [11] menunjukkan bahwa penggabungan teknik *Hierarchical Clustering* dengan *firewall IPTables* dan *IPSet* berhasil mengurangi beban *CPU* hingga 14% dan penggunaan memori hingga 11% pada router ketika memproses log serangan. Hal ini membuktikan bahwa *IPTables* mampu menjadi bagian dari metode mitigasi cerdas berbasis data.

IPTables juga mendukung otomatisasi respons melalui skrip shell yang memicu aturan dinamis berdasarkan deteksi anomali. Pendekatan ini selaras dengan tren mitigasi modern yang menekankan integrasi antara *anomaly detection* dan tindakan otomatis, sebagaimana dijelaskan dalam penelitian [8] dan [4], di mana paket berbahaya dapat diblokir segera setelah deteksi anomali terpicu.

Meskipun *IPTables* memiliki keunggulan pada efisiensi dan fleksibilitas, dalam penelitian ini *IPTables* tidak digunakan sebagai mekanisme mitigasi utama. Pembahasan *IPTables* dimasukkan sebagai tinjauan pustaka untuk menunjukkan bahwa beberapa penelitian sebelumnya menggunakan *firewall* berbasis aturan statis atau semi-otomatis untuk merespons serangan *DDoS*. Namun, fokus penelitian ini adalah pada penerapan *black hole routing* berbasis *Linux Gateway* yang terintegrasi dengan *anomaly detection* dan otomatisasi Wazuh *Active Response*, sehingga lebih sesuai untuk simulasi pada *small-scale network*.

2.6 Otomatisasi Serangan DDoS

Dalam menghadapi serangan *Distributed Denial of Service (DDoS)*, kecepatan respons merupakan faktor yang sangat penting untuk meminimalisasi dampak serangan terhadap ketersediaan layanan. Proses mitigasi manual sering kali tidak efektif karena membutuhkan waktu analisis dan konfigurasi yang cukup lama, sementara serangan dapat berlangsung hanya dalam hitungan detik. Oleh karena itu, dibutuhkan mekanisme otomatisasi respons

DDoS yang mampu mendeteksi pola lalu lintas anomali secara *real-time* dan segera menerapkan tindakan pencegahan, seperti *filtering*, *rate limiting*, atau *rerouting* trafik berbahaya. Dengan adanya otomatisasi, sistem keamanan dapat merespons secara lebih cepat, konsisten, dan adaptif, sehingga layanan tetap stabil meskipun sedang mengalami serangan besar.

2.6.1 Komponen Utama Otomatisasi Respons DDoS

Otomatisasi respons *DDoS* merupakan pendekatan yang memanfaatkan sistem terintegrasi untuk mendeteksi, menganalisis, dan merespons serangan secara cepat tanpa banyak intervensi manual. Dengan adanya otomatisasi, waktu tanggap terhadap serangan dapat diminimalkan, sehingga layanan tetap tersedia dan dampak serangan dapat diatasi. Pendekatan ini menjadi penting terutama pada lingkungan dengan sumber daya terbatas, karena mampu meningkatkan efisiensi sekaligus menjaga kestabilan jaringan.

1. ***Detection* (Deteksi)**

   Tahap awal berupa identifikasi pola lalu lintas jaringan untuk mendeteksi anomali yang mengindikasi serangan *DDoS*, misalnya lonjakan koneksi *SYN*, request *UDP* yang berlebihan, atau trafik *ICMP* yang tidak wajar. Deteksi dalam penelitian ini dilakukan dengan *threshold* dinamis berbasis acuan manual, sehingga penelitian ini dilakukan dengan kondisi trafik *real-time*.

2. ***Decision* (Pengambilan Keputusan)**

   Setelah anomali terdeteksi, sistem akan menganalisis data dan menentukan jenis respons yang paling tepat. Pada penelitian ini, keputusan difokuskan pada pengalohan trafik berbahaya ke *null route*. Dengan integrasi *anomaly detection*, sistem hanya menandai IP yang melampaui *threshold* untuk di alihkan, sheingga trafik sah tetap dapat dilayani.

3. ***Action* (Tindakan)**

   Merupakan implementasi keputusan yang sudah ditetapkan. Dalam penelitian ini, Tindakan berupa eksekusi *black hole routing* pada *Linux Gateway* secara otomatis melalui skrip sedrhana. Dengan cara ini, trafik berbahaya langsung dibuang di kernel tanpa menunggu proses manual. Pada konteks lain, Tindakan otomatis juga dapat berupa aktivasi rate limiting, Ip filtering, atau load balancing, namun focus penelitian ini adalah *black hole routing* yang terintegrasi dengan *anomaly detection*.

2.6.2 Konsep Otomatisasi dalam Mitigasi Serangan

Konsep otomatisasi dalam mitigasi serangan berfokus pada upaya mendeteksi dan merespons ancaman tanpa bergantung sepenuhnya pada intervensi manual. Beberapa penelitian menunjukkan bahwa penerapan otomatisasi, seperti *Intrusion Detection System* dengan mekanisme *black hole routing*, mampu mempercepat proses pemblokiran trafik berbahaya dibandingkan dengan metode manual [23]. Pendekatan berbasisi *anomaly detection* juga banyak digunakan untuk mengidentifikasi pola trafik yang menyimpang, bahkan terhadap serangan baru yang belum memiliki *signature* [1]. Selanjutnya, teknik mitigasi *real-time* dengan dukungan algoritma *machine learning* terbukti meningkatkan akurasi deteksi hingga lebih 95% sekaligus memungkinkan mitigasi adaptif [24]. Di sisi lain, penggunaan RTBH *(remotely Triggered Black Hole)* memungkinkan pengalihan trafik berbahaya langsung pada level router, meskipun tetap perlu seleksi agar tidak membuang trafik sah [15]. Bahkan pada arsitektur jairngan modern seperti *Software Defined Networking* (*SDN*), otomatisasi berbasis *flow entropy* dan *packet rate* telah terbukti mampu mendeteksi anomali dengan cepat dan melakukan mitigasi melalui selective blackholing, sehingga kinerja jaringan dapat dipulihkan hampir mendekati kondisi normal [25]. Dengan demikian, otomatisasi tidak hanya meningkatkan kecepatan respons, tetapi juga memberikan efektivitas dan efisiensi yang lebih tinggi dalam menjaga ketersediaan layanan pada saat terjadi serangan *DDoS.*

2.7 Integrasi *Anomaly Detection* dan Black Hole Routing

Integrasi *anomaly detection* dengan *black hole routing* merupakan strategi mitigasi serangan DDoS yang dirancang untuk mengatasi keterbatasan metode *black hole routing* konvensional. Secara tradisional, *black hole routing* bekerja dengan mengalihkan seluruh trafik yang menuju alamat target ke *null route* sehingga trafik tersebut terbuang. Meskipun efektif menghentikan serangan dalam kapasitas besar, kelemahan utamanya adalah sifatnya yang tidak selektif, sehingga trafik sah yang menuju target juga ikut terbuang. Oleh karena itu, dibutuhkan integrasi dengan *anomaly detection* agar proses pemlokiran lebih selektif.

Anomaly detection berperan dalam memantau lalu lintas jaringan secara *real-time* dan mengidentifikasi pola trafik yang menyimpang dari baseline normal, misalnya berdasarkan *packet rate*, frekuensi koneksi, maupun entropi aliran trafik. Penelitian oleh [25] menujukkan bahwa penggunaan algoritma *anomaly detection* berbasis *packet rate* dan *flow entropy* pada arsitektur *SDN* mampu mendeteksi serangan dengan akurasi di atas 95% dan

false positive rate rendah, dengan waktu deteksi dibawah 250 ms [25]. Setelah anomali terdeteksi, sistem kemudian memicu *black hole routing* secara otomatis namun hanya terhadap sumber trafik berbahaya yang terindikasi, sehingga trafik sah tetap dapat dilayani. Integrasi ini sejalan dengan penelitian sebelumnya yang menekankan pentingnya otomatisasi respons *DDoS*. Menurut penelitian [23], otomatisasi *black hole routing* yang dikombinasikan dengan sistem deteksi (misalnya *Snort IDS*) mempercepat proses mitigasi disbanding proses manual, sehingga waktu terhadap serangan dapat ditekan secara signifikan. Sementara itu, penelitian lain, menegaskan bahwa *selective black hole routing* lebih efektif dibandingkan penerapan black hole secara penuh karena mampu mempertahankan throughput hingga 90% dari kondisi normal meskipun serangan sedang berlangsung.

Dengan demikian, integrasi *anomaly detection* dan *black hole routing* dapat dianggap sebagai solusi kompromi yang menggabungkan kecepatan pemblokiran *black hole routing* dengan kecerdasan adaptif *anomaly detection*. Pendekatan ini membuat mitigasi lebih selektif, khususnya pada jaringan kecil yang memiliki keterbatasan infrastruktur.

2.8 Tools (Perangkat Lunak)

Dalam penelitian ini, digunakan beberapa *tools* untuk mendukung monitoring, deteksi, dan mitigasi serangan *DDoS*. Kombinasi *tools* ini memungkinkan sistem bekerja secara efisien pada lingkungan *small scale network.*

2.8.1 Wazuh Platform

Wazuh merupakan platform keamanan open-source yang berfungsi sebagai sistem monitoring, analisis log, dan deteksi ancaman pada infrastruktur jaringan. Wazuh mengumpulkan berbagai jenis log dari host yang diawasi, menganalisisnya menggunakan rule keamanan, dan menghasilkan peringatan ketika terdeteksi aktivitas dan menyimpang dari pola normal. Dalam penelitian ini, Wazuh dimanfaatkan sebagai pusat deteksi dan otomasi mitigasi serangan DDoS berbasis trafik. Sistem tidak hanya berperan sebagai *IDS* berbasis log *(log-based intrusion detection),* tetapi juga sebagai pengendali respons otomatis melalui fitur *Active Response* yang mampu menjalankan skrip mitigasi secara langsung di host yang dipantau. Arsitektur Wazuh terdiri dari tiga komponen utama, yaitu Wazuh Manager, Wazuh Agent, serta Wazuh Dashboard (yang terhubung dengan Wazuh

Indexer), yang saling bekerja sama untuk membentuk satu rangkaian sistem deteksi dan mitigasi terpadu.

2.8.1.1 Wazuh Manager

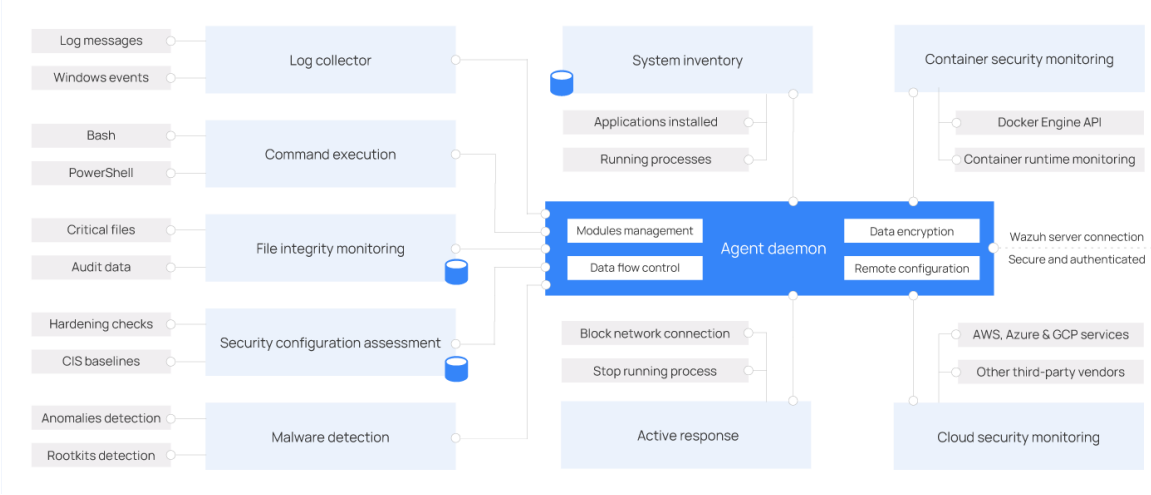
Wazuh Manager merupakan inti dari arsitektur Wazuh dan berperan sebagai pusat pemrosesan seluruh data keamanan yang dikirimkan oleh agent. Komponen ini menerima log dari berbagai host, kemudian memprosesnya melalui *rule engine* untuk menentukan apakah suatu aktivitas termasuk normal, mencurigakan, atau berbahaya. Proses analisis ini mencakup parsing log, pencocokan dengan rule bawaan maupun rule kustom, korelasi event, serta penentuan tingkat keparahan (severity) dari setiap alert yang dihasilkan. Dalam penelitian ini, Wazuh Manager digunakan untuk menganalisis log yang dikirimkan dari Linux Gateway, yang berisi informasi mengenai pola trafik seperti jumlah paket per detik, koneksi TCP, serta aktivitas protokol UDP dan *ICMP*. Berdasarkan rule deteksi yang telah disusun, Manager akan mengidentifikasi anomali yang mengindikasikan adanya serangan DDoS, misalnya lonjakan mendadak trafik *SYN*, peningkatan signifikan trafik *UDP*, atau *ICMP flooding*.

Selain melakukan fungsi analisis dan deteksi, Wazuh Manager juga memiliki modul *Active Response* yang memungkinkan sistem untuk merespons ancaman secara otomatis tanpa intervensi manual. Ketika sebuah alert mencapai tingkat keparahan tertentu sesuai dengan kriteria yang telah ditentukan, Manager akan memicu aksi *Active Response* dengan mengirimkan perintah ke Wazuh Agent pada host yang relevan, dalam hal ini Linux Gateway. Perintah tersebut dapat berupa eksekusi skrip shell yang menambahkan rute black hole ke IP penyerang, sehingga seluruh trafik dari sumber tersebut langsung dibuang di gateway. Dengan demikian, Wazuh Manager tidak hanya bertindak sebagai “otak” dalam proses deteksi, tetapi juga sebagai pengendali utama dalam rantai proses mitigasi serangan yang terintegrasi.

2.8.1.2 Wazuh Agent

Wazuh Agent adalah komponen yang dipasang pada host yang ingin dimonitor, dan berfungsi sebagai pengumpul data (data collector) serta eksekutor tindakan yang dikirimkan oleh Manager. Agent berjalan sebagai layanan di sistem operasi dan bertugas mengumpulkan berbagai jenis log, seperti log sistem, log jaringan, log aplikasi, maupun output dari skrip kustom yang dijalankan pada host. Dalam penelitian ini, Wazuh Agent dipasang pada *Linux Gateway* sebagai titik utama pengamatan, karena seluruh trafik baik

dari *client* maupun *attacker* akan melewati gateway sebelum mencapai server target. Selain itu, Agent juga dapat dipasang pada server target jika diperlukan pemantauan tambahan terhadap kondisi layanan.



Gambar 2. 2 Wazuh Agent

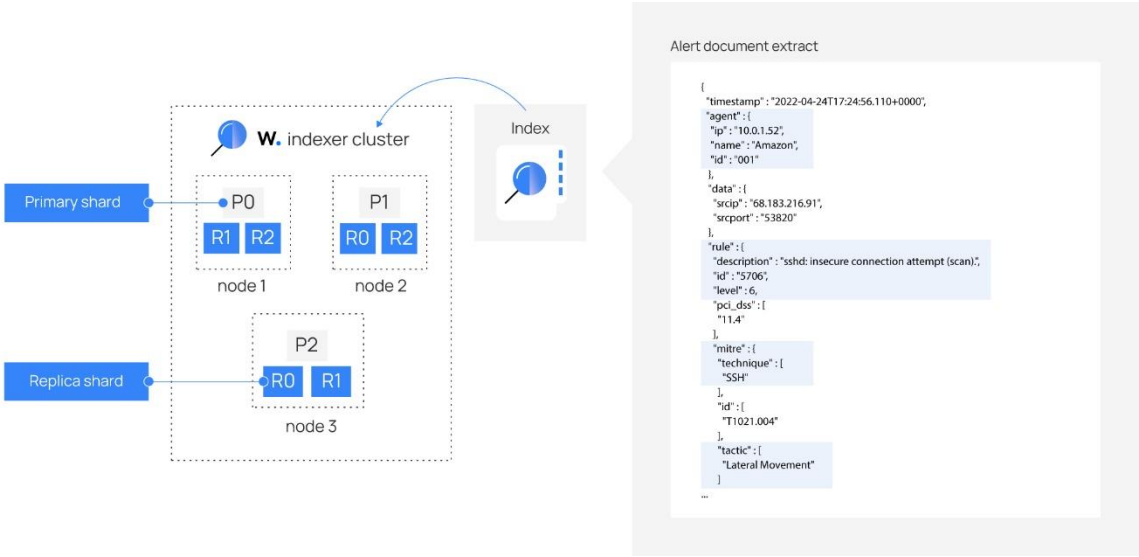
(sumber:[26])

Pada Linux Gateway, Wazuh Agent mengumpulkan log terkait aktivitas trafik hasil pemrosesan skrip *anomaly detection*, termasuk informasi mengenai *packet per second* (*PPS*), jumlah permintaan *SYN*, trafik *UDP*, dan *ICMP*. Log-log ini kemudian dikirimkan secara berkala maupun *real-time* ke Wazuh Manager untuk dianalisis. Di sisi lain, ketika Wazuh Manager memicu mekanisme *Active Response*, perintah yang dihasilkan akan dikirim kembali ke Agent untuk dieksekusi pada host yang bersangkutan. Misalnya, ketika sebuah IP sumber terdeteksi melampaui *threshold* yang ditentukan, Agent akan menjalankan skrip yang menambahkan rute null (black hole route) untuk IP tersebut. Dengan cara ini, Agent berfungsi sebagai perpanjangan tangan Manager di sisi host: mengumpulkan data, menerima instruksi, dan mengeksekusi tindakan mitigasi secara langsung di level sistem operasi.

2.8.1.3 Wazuh Indexer

Wazuh Indexer merupakan komponen inti dalam arsitektur Wazuh yang berfungsi sebagai sistem penyimpanan dan pencarian log berbasis teknologi OpenSearch. Indexer bertanggung jawab untuk menampung seluruh data keamanan yang dikirimkan oleh Wazuh Manager, termasuk event log, alert hasil deteksi, metadata agent, serta informasi aktivitas jaringan. Dengan mekanisme indexing yang terstruktur, Wazuh Indexer mampu melakukan pencarian cepat dan mengembalikan data dalam hitungan milidetik, sehingga mendukung proses analisis dan visualisasi secara *real-time* melalui Wazuh Dashboard. Sistem ini

bekerja sebagai repositori utama yang menyimpan semua data historis yang diperlukan untuk audit, korelasi event, investigasi forensik, maupun validasi hasil mitigasi.



Gambar 2. 3 Wazuh Indexer

(sumber:[26])

Dalam konteks penelitian ini, Wazuh Indexer tidak digunakan sebagai komponen utama untuk proses mitigasi, namun tetap berperan penting dalam menyimpan dan menyajikan informasi yang muncul selama pengujian serangan DDoS. Semua alert yang dihasilkan oleh rule-based *anomaly detection* termasuk indikasi trafik abnormal seperti lonjakan *SYN*, trafik UDP berlebihan, atau pola *ICMP flood* akan tersimpan dalam Indexer. Dengan demikian, peneliti dapat meninjau kembali alur serangan dan proses mitigasi melalui Dashboard yang mengambil data langsung dari Indexer. Keberadaan Indexer juga memastikan bahwa seluruh aktivitas selama eksperimen terdokumentasi dengan baik, termasuk eksekusi *Active Response* yang memicu mekanisme *black hole routing* di *Linux Gateway*.

Struktur indeks utama dalam wazuh indexer:

1. **Wazuh-alerts**

Index wazuh-alerts merupakan ruang penyimpanan utama untuk semua alert yang dihasilkan oleh wazuh manager sebagai bentuk respons terhadap aktivitas yang dianggap mencurigakan atau berpotensi membahayakan sistem. Setiap alert tersimpan di dalam indeks ini berasal dari proses rule-based detection, mulai dari deteksi brute force, pemindaian port, anomali trafik jaringan, hingga ekspoitasi dalam jaringan tertentu. Informasi yang tersimpan mencakup timestamp kejadian, tingkat keparahan alert, identitas agent, alamat IP sumber dan tujuan, nama rule yang aktif, serta ringkasan log yang memicu alert. Keberadaan indeks ini sangat penting karena menjadi dasar visualisasi kemanan di

Wazuh Dashboard sekaligus sumber utama yang digunakan peneliti dalam mengevaluasi pola serangan DDoS dan efektivitas mitigasi yang diterapkan.

2. **Wazuh-archieves**

Indeks Wazuh-archieves berfungsi untuk menyimpan seluruh log mentah yang dikirimkan oleh Wazuh Agent dari berbagai perangkat yang dimonitor. Tidak seperti Wazuh-alert yang hanya menampung event berbahaya, indeks ini menyimpan semua aktivitas dalam bentuk asli tanpa modifikasi, termasuk log sistem, aplikasi, autentikasi, firewall, webserver, serta trafik jaringan yang masuk ke server. Data mentah ini sangat penting untuk rekontruksi kejadian secara detail, termasuk dalam melakukan analisis forensik dan penelusuran kronologi serangan. Dalam konteks pengujian DDoS, indeks ini memungkinkan peneliti melihat bukti autentik seperti frekuensi request, pola flooding, serta log layanan yang mengalami tekanan dari trafik berbahaya.

3. **Wazuh-monitoring**

Indeks Wazuh-monitoring berfungsi untuk mencatat kondisi dan status operasional setiap agent yang terhubung ke Wazuh Manager. Informasi yang disimpan meliputi status koneksi agent (aktif atau offline), versi agent, jenis sistem operasi, penggunaan sumber daya, modul yang sedang berjalan, serta waktu terakhir agent mengirimkan data. Indeks ini memastikan bahwa administrator dapat memantau keamanan seluruh perangkat dalam jaringan secara real-time, termasuk mendeteksi agent yang berhenti merespons akibat beban serangan atau gangguan sistem. Dalam penelitian ini, indeks wazuh-monitoring membantu mengamati kestabilan agent selama eksperimen DDoS, terutama ketika sistem mengalami lonjakan trafik dan potensi overload.

4. **Wazuh-statistics**

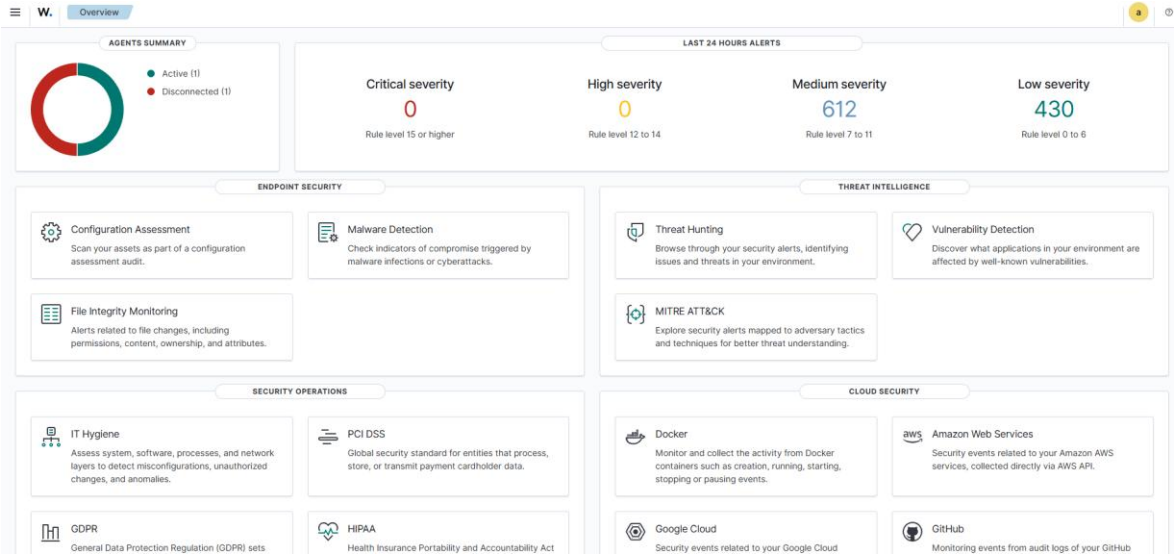
Indeks *wazuh-statistics* menyimpan berbagai data statistik mengenai performa internal Wazuh Manager dan Wazuh Indexer dalam memproses log keamanan. Data yang direkam mencakup jumlah alert per waktu tertentu, volume log yang diterima, kinerja rule engine, pemakaian CPU dan memori server Wazuh, serta intensitas aktivitas setiap modul pemantauan. Informasi ini berfungsi untuk menilai apakah Wazuh mampu menangani beban log yang tinggi, terutama saat terjadi serangan DDoS yang menghasilkan banyak event secara simultan. Melalui indeks ini, peneliti dapat mengevaluasi efisiensi proses

analitik Wazuh dan memastikan bahwa sistem tetap responsif meskipun berada dalam kondisi tekanan jaringan yang ekstrem.

Secara keseluruhan, Wazuh Indexer berfungsi sebagai lapisan penyimpanan terstruktur yang memungkinkan sistem Wazuh bekerja secara efisien, cepat, dan konsisten. Meskipun penelitian ini tidak menitikberatkan pada fungsi analitik lanjutan dari Indexer, komponen ini tetap penting untuk menyediakan rekam jejak aktivitas keamanan dan memperkuat pemantauan selama simulasi *DDoS* pada *small-scale network*.

2.8.1.4 Wazuh Dasboard

Wazuh Dashboard merupakan antarmuka grafis berbasis web yang memungkinkan administrator memantau kondisi keamanan sistem secara visual dan *real-time*. Dashboard terhubung dengan Wazuh Manager dan Wazuh Indexer untuk menampilkan informasi seperti daftar agent yang aktif, event log, alert yang dihasilkan, statistik serangan, serta detail tiap event yang dianggap berbahaya. Dalam konteks penelitian ini, Wazuh Dashboard digunakan sebagai media pemantauan selama proses pengujian serangan DDoS dan mitigasinya, sehingga peneliti dapat melihat secara langsung bagaimana sistem mendeteksi anomali, menghasilkan alert, dan mengeksekusi *Active Response*.



Gambar 2. 4 Wazuh Dashboard

(sumber:[26])

Melalui Dashboard, administrator dapat mengamati status Linux Gateway, *client*, dan server target, termasuk apakah agent dalam kondisi aktif, kapan terakhir mengirimkan data, serta jenis event yang tercatat. Setiap kali terjadi serangan DDoS simulasi, Wazuh Dashboard akan menampilkan alert dengan tingkat keparahan tertentu, lengkap dengan

informasi IP sumber, jenis rule yang terpicu, serta waktu terjadinya serangan. Selain itu, Dashboard juga dapat digunakan untuk memverifikasi keberhasilan mekanisme mitigasi, misalnya dengan melihat adanya *Active Response* yang dijalankan terhadap IP penyerang. Dengan demikian, meskipun penelitian ini tidak berfokus pada visualisasi data secara kompleks seperti pada Grafana, Wazuh Dashboard tetap memegang peran penting sebagai sarana monitoring, validasi, dan dokumentasi perilaku sistem selama pengujian.

2.8.2 Ubuntu Server 22.04

Ubuntu Server 22.04 LTS digunakan sebagai sistem operasi utama pada seluruh node penelitian, termasuk Wazuh server, Linux Gateway, Webserver. Pemilihan Ubuntu Server 22.04 didasarkan pada kestabilan, dukungan jangka panjang (LTS), serta kompatibilitas yang sangat baik terhadap berbagai tools keamanan dan aplikasi server yang digunakan dalam penelitian ini. Sistem operasi ini menyediakan lingkungan yang ringan, stabil, bebas antarmuka grafis sehingga seluruh sumber daya dapat difokuskan untuk menjalankan proses deteksi dan mitigasi DDoS. Pada Wazuh Server, ubuntu berperan sebagai fondasi untuk menjalankan komponen Wazuh Manager, Wazh Indexer, dan Wazuh Dashboard secara optimal. Sementara Linux Gateway, sistem operasi ini mendukung berbagai modul seperti iptables, routing tables, dan konfigurasi network interface yang diperlukan dalam skenario blackhole routing sebagai mekanisme mitigasi. Pada Webserver, Ubuntu Server menyediakan dukungan untuk layanan HTTPS serta kemudahan integrasi dengan Wazuh Agent, sehingga seluruh aktivitas sistem dapat dikumpulkan dan dianalisis secara terpusat.

2.8.3 Kali Linux sebagai Mesin Penyerang

Kali Linux digunakan sebagai mesin penyerang dalam penelitian ini, khususnya untuk menstimulasikan berbagai jenis serangan DDoS dan aktivitas berbahaya lainnya. Distribusi ini dipilih karena merupakan sistem operasi berbasis Debian yang secara khusus dibuat untuk keperluan penetration testing dan dilengkapi dengan ratusan tools keamana seperti hping3, Nmap, Hydra, Netcat, dan berbagai utilities lain yang sangat relevan untuk simulasi serangan jaringan. Dengan menggunakan Kali Linux, peneliti dapat menghasilkan trafik berlebih untuk mengujia kemmapuan deteksi Wazuh dan respons mitigasi pada Linux Gateway dalam kondisi yang mendekati serangan nyata. Selain itu, penggunaan Kali Linux memungkinkan peneliti melakukan variasi serangan, mulai dari SYN Flood, UDP Flood, ICMP Flood, brute force login, hingga scanning intensif yang semuanya dapat memicu alert

pada Wazuh Indexer. Dengan demikian, Kali Linux berperan penting sebagai generator serangan yang menguji efektivitas keseluruhan sistem monitoring dan mitigasi.

2.8.4 Hping3

Hping3 adalah sebuah *packet crafting tool* berbasis command line yang digunakan untuk membuat dan mengirim paket-paket jaringan secara manual sesuai dengan kebutuhan pengguna. Tool ini mendukung berbagai protokol seperti *TCP*, *UDP*, dan *ICMP*, serta menyediakan kemampuan untuk memodifikasi hampir seluruh field dalam header paket. Karena fleksibilitas dan kemampuannya menghasilkan trafik dalam volume besar, *hping3* banyak digunakan dalam pengujian keamanan jaringan, terutama untuk mensimulasikan berbagai jenis serangan *Denial of Service* (*DoS*) maupun *Distributed Denial of Service* (*DDoS*).

Dalam penelitian ini, *hping3* digunakan sebagai generator trafik berbahaya *(attack traffic)* pada VM Kali Linux. Tool ini memungkinkan peneliti untuk melakukan serangan *SYN Flood*, *UDP Flood*, dan *ICMP Flood* dengan menentukan parameter seperti jumlah paket per detik, ukuran paket, *flag TCP*, serta target IP. Dengan perintah sederhana seperti:

```
hping3 -S --flood -p 80 <IP-target>
```

peneliti dapat menghasilkan serangan *SYN Flood* dengan intensitas tinggi secara *real-time*. Penggunaan *hping3* dalam lingkungan simulasi memberikan fleksibilitas untuk mengatur skala dan variasi serangan, sehingga sistem deteksi anomali dapat diuji dengan berbagai skenario trafik.

Selain itu, *hping3* juga mendukung fitur *traceroute*, *firewall testing*, dan analisis paket, sehingga dapat digunakan untuk memvalidasi apakah aturan mitigasi seperti *black hole routing* berhasil menghentikan aliran paket penyerang. Dalam konteks penelitian ini, *hping3* tidak hanya berfungsi sebagai alat serangan, tetapi juga sebagai sarana untuk mengukur efektivitas sistem mitigasi yang diimplementasikan pada Linux Gateway.

Sebagai tool yang sudah lama menjadi standar dalam dunia *penetration testing*, *hping3* banyak digunakan dalam penelitian akademik terkait serangan berbasis trafik. Penelitian mengenai deteksi *DDoS*, performa jaringan, dan pengujian *firewall* sering menggunakan *hping3* karena kemampuannya menghasilkan pola trafik yang konsisten dan terukur. Hal ini membuat *hping3* menjadi pilihan yang tepat dalam simulasi *DDoS* berskala kecil hingga menengah seperti yang dilakukan dalam proyek tugas akhir ini.

Dengan demikian, *hping3* menjadi komponen penting dalam rangkaian pengujian sistem mitigasi *DDoS* karena menyediakan data trafik nyata yang diperlukan untuk menguji akurasi *anomaly detection* serta respons *black hole routing* pada *Linux Gateway*.

2.9 Infrastruktur (Perangkat Keras dan Jaringan)

Dalam penelitian ini, penggunaan infrastruktur yang tepat memegang peranan penting untuk memastikan seluruh komponen sistem dapat berjalan secara optimal. Infrastruktur yang digunakan mencakup perangkat keras utama, topologi jaringan, serta perangkat pendukung yang berfungsi sebagai media komunikasi antar node dalam lingkungan pengujian. Setiap elemen infrastruktur dirancang sedemikian rupa untuk mensimulasikan kondisi jaringan nyata dalam skala kecil, sehingga proses monitoring, deteksi, dan mitigasi serangan DDoS dapat dilakukan secara efektif dan terukur. Penjelasan mengenai perangkat keras dan jaringan yang digunakan dalam penelitian ini disajikan pada subbab berikut.

2.9.1 VMWare Workstation (Lingkungan Instalasi)

VMware Workstation digunakan sebagai platform visualisasi yang menyediakan lingkungan terisolasi untuk seluruh node yang terlibat dalam penelitian. Platform ini memungkinkan peneliti membuat banyak Virtual Machine dalam satu host fisik, sehingga topologi jaringan yang kompleks dapat dibangun tanpa emerlukan perangkat keras tambahan. VMware Workstation mendukung konfigurasi jaringan yang fleksibel melalui berbagai mode seperti NAT, Bridge, Host-Only, sehingga penelrii dapat menentukan alur trafik, membuat segmentasi jaringan, dan menghubungkan VM dengan perangkat keras fisik seusai kebutuhan skenario. Selain itu, WMware menyediakan keandalan tinggi dalam manajemen snapshot, sehingga peneliti dapat mengembalikan kondisi sistem ke titik tertentu dengan mudah jika dibutuhkan untuk pengujian ulang. Dengan kemampuan tersebut, VMware Workstation menjadi media yang sangat efektif untuk menguji serangan dan mitigasi dalam small-scale network tanpa risiko mengganggu jaringan produksi

Gambar 2. 5 VMWare Workstation

(sumber:[27]**)**

2.9.2 Oracle VirtualBox

Selain VMware Workstation, penelitian ini juga memanfaatkan Oracle VirtualBox sebagai alternatif platform virtualisasi dalam beberapa tahap pengujian. VirtualBox digunakan terutama pada proses validasi konfigurasi jaringan dan pengujian kompabilitas sistem, mengingat fleksibilitasnya dalma mengelola berbagai jenis mesin virtual dengan kebutuhan sumber daya ringan.

virtualBox menyediakan dukungan jaringan yang luas seperti NAT, Bridge Adapter, Host-Only Adapter, dan Internal Network, sehingga dapat digunakan untuk mensimulasikan skenario routing, segmentasi jaringan, serta komunikasi antar VM dalam kondisi yang menyerupai lingkungan nyata.

Kemampuan VirtualBox dalam menjalankan berbagai sistem operasi dengan overhead rendah menjadikannya pendukung yang efektif selama penelitian, terutama ketika diperlukan pemisahan lingkungan kerja antara node penyerang dan node server. Dengan demikian, penggunaan Oracle VirtualBox melengkapi VMware Workstation sebagai bagian dari infrastruktur virtualisasi yang fleksibel dan serbaguna.

Gambar 2. 6 Oracle VirtualBox

(**sumber:** [28])

2.9.3 Modem Telkomsel Orbit

Modem Telkomsel Orbit digunakan sebagai sumber koneksi internet utama untuk host fisik yang menjalankan VMware Workstation. Perangkat ini memungkinkan seluruh Vritual Machine dalam lingkungan virtual menerima akses internet melalui host, baik untuk pembaruan sistem, instalasi paket, maupun simulasi trafik eksternal yang relevan dengan penelitian. Orbit dipilih karena kestabilan sinyal dan kemampuannya menyediakan jaringan yang cukup untuk kebutuhan pengujian berbasis trafik. Selain itu, keberadaan modem ini memberikan konteks realistis terhadap skenario serangan DDoS, di mana lalu lintas jaringan diproses melalui jaringan publik yang sesungguhnya sebelum masuk ke host virtual. Dengan demikian, Telkomsel Orbit tidak hanya menjadi sumber konektivitas tetapi juga komponen penting untuk meniru kondisi jaringan yang sering dijumpai pada lingkungan operasional skala kecil.

2.10 Related Works

Sejumlah penelitian telah dilakukan untuk mengembangkan metode deteksi dan mitigasi serangan *Distributed Denial of Service (DDoS).* Penelitian-penelitian tersebut memberikan gambaran mengenai efektivitas teknik *anomaly detection*, *adaptive thresholding*, serta mekanisme *blackhole routing*, meskipun masing-masing masih memiliki keterbatasan.

Penelitian oleh [1] mengusulkan *Lightweight Anomaly Detection System* yang mampu mendeteksi pola trafik abnormal pada jaringan dengan sumber daya terbatas. Pendekatan ini menegaskan bahwa deteksi berbasis anomali dapat dijalankan dengan efisien, namun penelitian tersebut berfokus pada serangan *blackhole* di MANET, bukan pada serangan *DDoS* berbasis trafik berskala besar seperti *SYN*, *UDP*, atau *ICMP Flood*.

Penelitian [3] memanfaatkan teknologi *Digital Twin* untuk mendeteksi serangan *DDoS* secara *real-time* melalui pemodelan perilaku jaringan. Pendekatan ini terbukti efektif dalam meningkatkan sensitivitas deteksi, tetapi memerlukan infrastruktur yang kompleks sehingga kurang sesuai diterapkan pada jaringan kecil berbasis VM seperti dalam penelitian ini.

Sementara itu, penelitian [5] mengkaji pemanfaatan *Deep Neural Network (DNN)* untuk deteksi intrusi *real-time*. Hasilnya menunjukkan akurasi tinggi dalam mengidentifikasi pola serangan jaringan. Namun, metode ini memerlukan komputasi besar sehingga kurang efisien jika diterapkan pada *small scale network* yang menjadi fokus penelitian ini.

Penelitian [8] mengembangkan *variable-trust threshold-based mitigation* pada arsitektur *Software-Defined Networking (SDN).* Sistem ini mampu menyesuaikan ambang batas deteksi berdasarkan karakteristik trafik setiap pengguna, dan mencapai akurasi hingga 98%. Walaupun efektif, arsitektur SDN tidak digunakan dalam penelitian ini yang berfokus pada routing berbasis Linux Gateway.

Di sisi mitigasi, penelitian [9] melakukan studi pengukuran kinerja *BGP black hole routing* untuk mengalihkan trafik berbahaya di sisi ISP. Pendekatan ini sangat efektif secara skala besar, tetapi tidak dapat diterapkan langsung pada lingkungan VM berskala kecil.

Pendekatan deteksi berbasis *adaptive thresholding* pada penelitian [10] terbukti mampu mendeteksi *Distributed Reflection DDoS (DRDoS)* secara lebih akurat dibanding *threshold* statis. Hal ini memperkuat urgensi penggunaan *threshold* dinamis seperti yang diterapkan dalam penelitian ini. Selain itu, penelitian [11] dan [13] memperkenalkan metode *anomaly*

detection berbasis Naive Bayes dan LSTM Autoencoder untuk mendeteksi trafik *DDoS*, memberikan bukti bahwa pendekatan berbasis data sangat efektif dalam mengidentifikasi pola serangan.

Sejumlah penelitian pendukung juga relevan sebagai referensi tambahan. Misalnya, [12] menunjukkan bahwa Wazuh efektif dalam monitoring log dan deteksi DoS berbasis log analysis, sehingga mendukung pemilihan Wazuh sebagai komponen inti dalam penelitian ini. Penelitian [14] mengusulkan deteksi *DDoS* dengan federated GAN berbasis flow, namun metode ini membutuhkan dataset besar dan infrastruktur kompleks sehingga tidak sesuai untuk eksperimen *small scale network*.

Berdasarkan kajian terhadap penelitian-penelitian tersebut, terdapat beberapa gap yang menjadi dasar penelitian ini:

1. ***Anomaly Detection* umumnya hanya menghasilkan alert tanpa tindakan mitigasi otomatis**

   Sebagian besar penelitian seperti [1] [5] [11] dan [13] berfokus pada peningkatan akurasi deteksi, namun tidak menyediakan integrasi langsung dengan mekanisme mitigasi seperti black hole routing.
2. **Black Hole Routing efektif tetapi tidak selektif**

   Penelitian [9] menunjukkan bahwa *black hole routing* bekerja cepat namun memiliki risiko tinggi membuang trafik sah jika tidak dikombinasikan dengan deteksi yang akurat.
3. **Minimnya penelitian yang mengintegrasikan *anomaly detection* dan *black hole routing* dalam sistem otomatis**

   Sebagian besar penelitian memisahkan kedua pendekatan tersebut. Penelitian yang menggabungkannya dalam satu sistem komprehensif mulai dari deteksi, pengambilan keputusan, hingga eksekusi *black hole routing* masih jarang dibahas, terutama pada lingkungan *small scale network.*

4. **Belum adanya studi yang memanfaatkan Wazuh sebagai deteksi berbasis log untuk memicu mitigasi otomatis**

   Meskipun penelitian [12] menunjukkan kemampuan Wazuh untuk monitoring dan alerting, belum banyak penelitian yang mengintegrasikan fitur *Active Response* untuk melakukan mitigasi *DDoS* secara otomatis.

Dengan mempertimbangkan gap tersebut, penelitian ini berkontribusi dengan mengintegrasikan:

- *Anomaly Detection* berbasis *threshold* dinamis,
- Wazuh sebagai engine deteksi dan pemicu mitigasi,
- *Black hole routing* melalui *Linux Gateway*,
- Otomatisasi penuh dari deteksi, keputusan hingga mitigasi,

Dalam sebuah sistem yang dirancang khusus untuk lingkungan *small scale network* berbasis VM, sehingga mengisi kekosongan penelitian yang belum banyak dibahas sebelumnya.

2.11 Jadwal Penelitian

KEGIATAN	JADWAL																
	Agustus				September				Oktober				November			Desember	
	Minggu Ke -																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Pembagian Kelompok dan Dos																	
Penentuan Topik																	
Survei dan Review Jurnal																	
Survei dan Review Jurnal																	
Menulis / Menyusun Bab 2																	
Menulis / Menyusun Bab 3																	
Seminar Proposal																	
Revisi Proposal																	
Menulis / Menyusun Bab 4																	
Seminar TA 1																	
Revisi Seminar TA 1																	
Menulis / Menyusun Bab 5																	
Pra Sidang																	
Revisi Pra Sidang																	
Menulis / Menyusun Bab 6																	
Sidang Akhir																	

Gambar 2. 7 Jadwal Penelitian

BAB III
ANALISIS DAN DESAIN

3.1 Analisis

Bagian ini membahas proses analisis yang dilakukan sebelum perancangan dan implementasi sistem. Analisis bertujuan untuk memahami akar permasalahan, kebutuhan sistem, serta batasan lingkungan pengujian sehingga solusi yang dibangun benar-benar menjawab permasalahan serangan *Distirbuted Denial of Service* (*DDoS*) pada jaringan berskala kecil.

3.1.1 Analisis Masalah

Serangan *Distributed Denial of Service (DDoS)* merupakan ancaman yang secara langsung mengganggu ketersediaan layanan (availability) dengan membanjiri server atau jaringan menggunakan lalu lintas berbahaya dalam jumlah besar. Ketika volume trafik serangan melebihi kapasitas server atau *bandwidth*, layanan menjadi lambat, tidak responsif, bahkan tidak dapat diakses oleh pengguna sah. Permasalahan ini menjadi semakin kompleks karena pola serangan *DDoS* bersifat dinamis dan sering kali menyerupai trafik normal, sehingga sulit dideteksi secara cepat.

Dalam banyak implementasi, proses deteksi dan mitigasi serangan masih dilakukan secara manual, seperti analisis log, pemblokiran IP, atau perubahan aturan *firewall* saat serangan berlangsung. Metode manual ini membutuhkan waktu respons yang relatif lama, sehingga memperpanjang downtime layanan. Sementara itu, solusi tradisional seperti *firewall* atau *Intrusion Detection System (IDS)* berbasis *signature* memiliki keterbatasan karena hanya mengenali pola serangan yang sudah terdokumentasi, sehingga tidak efektif terhadap serangan baru (zero-day) atau variasi trafik berbahaya yang tidak memiliki *signature* jelas.

Di sisi lain, mekanisme mitigasi seperti *black hole routing* mampu membuang trafik berbahaya dengan cepat melalui null-route, namun teknik ini memiliki kelemahan mendasar: sifatnya tidak selektif. Tanpa adanya sistem pendeteksi anomali yang akurat, *black hole routing* berpotensi membuang trafik sah bersama trafik berbahaya, terutama jika pengalihan dilakukan pada alamat tujuan yang luas.

Dalam konteks penelitian akademik menggunakan *small scale network* berbasis *Virtual Machine (VM)*, tantangan utamanya adalah bagaimana merancang sistem yang:

1. Mendeteksi anomali trafik secara adaptif dan *real-time*, meskipun infrastruktur terbatas.
2. Menjalankan mitigasi otomatis melalui *black hole routing* hanya terhadap sumber trafik berbahaya.
3. Mengurangi kelemahan metode manual dan statis, sehingga ketersediaan layanan tetap terjaga.
4. Mengintegrasikan Wazuh sebagai engine deteksi dan pemicu *Active Response*, tanpa memerlukan perangkat jaringan tingkat ISP.

Dengan demikian, solusi yang dibutuhkan adalah integrasi *anomaly detection* berbasis *threshold* dinamis dengan mekanisme *black hole routing* otomatis, sehingga proses deteksi hingga mitigasi dapat berjalan cepat, selektif, dan efisien pada lingkungan jaringan kecil.

3.1.2 Analisis Pemecahan Masalah

Pemecahan masalah dalam penelitian ini difokuskan pada pengembangan sistem deteksi dan mitigasi serangan *DDoS* yang mampu bekerja secara otomatis pada lingkungan *small scale network*. Berdasarkan analisis sebelumnya, terdapat dua isu utama yang perlu ditangani, yaitu keterlambatan proses deteksi akibat metode manual dan kelemahan *black hole routing* yang bersifat tidak selektif. Oleh karena itu, penelitian ini mengusulkan solusi berupa integrasi *anomaly detection* berbasis *threshold* dinamis dengan mekanisme *black hole routing* otomatis pada Linux Gateway.

Anomaly detection digunakan untuk mendeteksi pola trafik mencurigakan secara *real-time* berdasarkan parameter dasar jaringan seperti *packet per second (PPS)*, jumlah koneksi *SYN*, dan intensitas trafik *UDP* maupun *ICMP*. Pendekatan *threshold* dinamis dipilih karena lebih adaptif terhadap variasi trafik dibandingkan *threshold* statis, sehingga dapat membedakan lonjakan trafik normal dan aktivitas serangan dengan lebih akurat dalam lingkungan berskala kecil.

Selanjutnya, hasil deteksi anomali digunakan sebagai pemicu aktif untuk melakukan mitigasi melalui black hole routing. Ketika trafik dari sebuah sumber melebihi *threshold*

yang ditentukan, sistem secara otomatis menambahkan rute null (*null route*) untuk alamat IP tersebut pada Linux Gateway. Dengan demikian, seluruh trafik berbahaya akan dibuang langsung pada gateway sebelum mencapai server target. Integrasi ini bertujuan untuk mengatasi kelemahan *black hole routing* yang umumnya membuang seluruh trafik ke suatu alamat tanpa seleksi, sehingga hanya sumber yang teridentifikasi sebagai penyerang yang dialihkan ke rute null.

Implementasi dilakukan pada *Linux Gateway* berbasis Ubuntu yang bertindak sebagai titik sentral seluruh trafik jaringan. Gateway ini menjadi lokasi eksekusi *anomaly detection*, pemrosesan log, dan penerapan aturan routing. Untuk menghubungkan proses deteksi dengan tindakan mitigasi otomatis, penelitian ini memanfaatkan platform keamanan Wazuh, yang menyediakan modul *Active Response* untuk menjalankan skrip mitigasi secara langsung ketika terjadi pelanggaran *threshold*.

Dengan pendekatan ini, sistem diharapkan mampu:

1. Mendeteksi serangan *DDoS* secara cepat dan adaptif, meskipun tanpa infrastruktur kompleks.
2. Melakukan mitigasi otomatis tanpa intervensi manual, sehingga mempercepat waktu respons.
3. Mengurangi risiko false blocking, karena pemblokiran dilakukan secara selektif berdasarkan hasil *anomaly detection*.
4. Cocok diterapkan pada lingkungan penelitian dengan sumber daya terbatas, seperti jaringan berbasis VM.

Pemecahan masalah ini menjadi dasar dari rancangan sistem yang diimplementasikan pada tahap berikutnya dan memastikan bahwa solusi yang diusulkan dapat bekerja efektif dalam konteks *small scale network*.

3.1.3 Batasan Lingkungan *Small-Scale Network*

Penelitian ini dilakukan dalam lingkungan *small-scale network* berbasis virtualisasi, sehingga seluruh proses deteksi dan mitigasi serangan *DDoS* berjalan di dalam *Virtual Machine* (VM). Lingkungan ini memiliki keterbatasan kapasitas *CPU*, memori, dan *bandwidth* dibandingkan jaringan nyata berskala besar. Oleh karena itu, serangan yang dihasilkan hanya berupa skenario *DDoS* sederhana berbasis trafik (*SYN Flood*, *UDP Flood*, *ICMP Flood*), tanpa melibatkan serangan volumetrik masif seperti Tbps-scale attacks.

Selain itu, routing yang digunakan bersifat lokal melalui *Linux Gateway* tanpa melibatkan router fisik atau mekanisme RTBH tingkat ISP. *Threshold anomaly detection* juga ditentukan secara manual berdasarkan pengamatan langsung terhadap pola trafik, bukan melalui analisis dataset besar atau metode pembelajaran otomatis. Dengan batasan ini, penelitian lebih difokuskan pada validasi konsep otomatisasi mitigasi, bukan performa sistem dalam skala produksi.

3.2 Analisis Kebutuhan Sistem

Dalam sub bab ini dijelaskan mengenai kebutuhan untuk mendukung implementasi otomatisasi respons *DDoS* berbasis *anomaly detection* dan *black hole routing*. Sistem ini memanfaatkan *tools* monitoring *open-source* untuk mengamati trafik jaringan secara *real-time*. Data yang terkumpul kemudian dianalisis menggunakan pendekatan *anomaly detection* sederhana berbasis *threshold* untuk mengenali adanya anomali trafik. Jika anomali terdeteksi, maka proses mitigasi dilakukan dengan cara mengeksekusi aturan *black hole routing* sehingga trafik berbahaya langsung dibuang sebelum mencapai server utama. Kebutuhan tersebut mencakup kebutuhan perangkat keras dan perangkat lunak.

3.2.1 Analisis Kebutuhan Perangkat Keras (Hardware)

Perangkat keras pada penelitian ini digunakan untuk mendukung proses pemantauan dan mitigasi serangan *DDoS* pada jaringan skala kecil (*small scale network*) yang disimulasikan menggunakan beberapa *Virtual Machine* (VM). Setiap VM memiliki fungsi spesifik, seperti server target, gateway mitigasi, mesin serangan, serta Wazuh Server dan Wazuh Agent sebagai komponen utama sistem deteksi. Trafik jaringan yang masuk dan keluar dari gateway akan dipantau secara *real-time* oleh Wazuh Agent, kemudian dianalisis oleh Wazuh Manager melalui rule engine untuk menentukan apakah trafik tersebut normal atau mengindikasikan anomali. Jika terdeteksi aktivitas mencurigakan seperti lonjakan *SYN*, *UDP*, atau *ICMP* yang melebihi *threshold* dinamis, Wazuh akan memicu mekanisme *Active Response* pada gateway untuk menjalankan *black hole routing* sehingga paket berbahaya dialihkan ke *null route* dan tidak mencapai server target. Dengan dukungan perangkat keras yang memadai, seluruh proses monitoring, deteksi, dan mitigasi dapat berlangsung stabil, responsif, dan tidak mengganggu akses pengguna sah terhadap layanan yang dilindungi.

Tabel 3. 1 Analisis Kebutuhan Perangkat Keras (Hardware)

Perangkat	Spesifikasi	Keterangan
Laptop 1 - ASUS TUF GAMING F15	- OS: Windows 11 64-bit - Processor: Intel Core i7 / i5 Generasi terbaru - RAM: 16 GB DDR4/DDR5 - Storage: 512 GB SSD NVMe	Laptop pertama digunakan sebagai host utama untuk menjalankan beberapa *Virtual Machine* (VM), seperti gateway mitigasi, Wazuh Manager, dan salah satu mesin serangan. Laptop ini memiliki spesifikasi yang cukup tinggi untuk menjalankan proses monitoring *real-time* dan analisis rule Wazuh secara stabil.
Laptop 2 - ASUS TUF GAMING F15	- OS: Windows 11 64-bit - Processor: Intel Core i7 / i5 - RAM: 16 GB DDR4/DDR5 - Storage: 512 GB SSD NVMe	Laptop kedua berfungsi menjalankan VM tambahan seperti server target, *client* sah, dan mesin serangan lain (*UDP*/*ICMP*). Perangkat ini membantu membagi beban virtualisasi agar simulasi jaringan berjalan lancar dan tidak membebani satu perangkat saja.
Laptop 3 - ASUS TUF GAMING A15	- OS: Windows 11 Home Single Language 64-bit - Processor: AMD Ryzen 7 6800H - RAM: 16 GB DDR5 - Storage: 512 GB SSD NVMe	Laptop ketiga digunakan sebagai host pendukung untuk menjalankan VM serangan tambahan, pengujian trafik, atau observasi sistem. Perangkat ini juga digunakan untuk membantu eksekusi script, monitoring, serta verifikasi hasil mitigasi secara paralel dengan dua laptop lain.
Telkomsel Orbit (Modem Router)	- Jenis: Modem Router Orbit (Adaptor-powered) - Koneksi: 4G/LTE Broadband	Digunakan untuk menyediakan koneksi internet stabil ke seluruh laptop selama proses instalasi dan konfigurasi

Perangkat	Spesifikasi	Keterangan
	- Fungsi: Router dan sumber konektivitas	Wazuh, pembaruan sistem, dan kebutuhan jaringan lainnya. Orbit berfungsi sebagai router fisik (bukan USB modem), memastikan ketiga laptop tetap terhubung ke jaringan lokal (LAN/WLAN) selama menjalankan simulasi *DDoS* dan proses monitoring.

3.2.2 Analisis Kebutuhan Perangkat Lunak (Software)

Perangkat lunak diperlukan untuk mendukung implementasi sistem otomatisasi respons *DDoS* berbasis *anomaly detection* dan black hole routing. Seluruh perangkat lunak yang digunakan bersifat open-source sehingga dapat diimplementasikan tanpa biaya lisensi. Kebutuhan perangkat lunak tersebut meliputi:

Tabel 3. 2 Analisis Kebutuhan Perangkat Lunak (Software)

Perangkat Lunak	Fungsi	Keterangan
VirtualBox	Platform virtualisasi	Digunakan untuk membuat dan menjalankan VM berbasis Ubuntu Server.
Ubuntu Server (VM Guest OS)	Sistem operasi pada VM	Dipasang pada 4 VM dengan peran berbeda: transmitter (penyerang), *Client* (pengguna sah), Server Target, dan *Linux Gateway* (*anomaly detection* + black hole routing).
Wazuh Agent	Monitoring dan Pengumpulan log	Dipasang pada Linux Gateway. Mengirimkan log, data trafik, dan event jaringan ke Wazuh Manager untuk dianalisis. Agent juga menjalankan skrip mitigasi otomatis saat

Perangkat Lunak	Fungsi	Keterangan
		diperintahkan melalui *Active Response*.
Wazuh Manager	Sistem deteksi dan korelasi	Berfungsi sebagai pusat analisis keamanan, memproses log yang dikirim dari agent, menerapkan rule untuk mendeteksi anomali seperti lonjakan trafik, serta mentrigger *Active Response* untuk mitigasi otomatis.
Wazuh Dashboard	Visualisasi alert	Digunakan untuk melihat event, alert, dan aktivitas deteksi. Tidak menjadi fokus utama dalam penelitian, tetapi membantu observasi hasil deteksi.
Bash Script	*Anomaly detection*	Mengimplementasikan *threshold-based detection* untuk mendeteksi lonjakan trafik dan memicu *black hole routing* secara otomatis
Web Server (Nginx)	Target layanan	Dipasang pada VM target sebagai layanan yang disimulasikan untuk menerima permintaan *client* sah dan menjadi target serangan *DDoS*

3.2.3 Analisis Kebutuhan Keamanan Sistem

Agar sistem mitigasi otomatis dapat berjalan dengan stabil, terdapat kebuthan keamanan yang harus dipenuhi, antara lain:

Tabel 3. 3 Analisis Kebutuhan Keamanan Sistem

Kebuhan Keamanan	Deskripsi
Monitoring Log *Real-Time*	Sistem harus mampu mengumpulkan log dari *Linux Gateway* secara terus-menerus agar anomali dapat terdeteksi dalam

Kebuhan Keamanan	Deskripsi
	hitungan detik sebelum serangan berkembang.
Integrasi Data Log	Log yang dikumpulkan harus terlindungi dari manipulasi. Keputusan mitigasi berbasis *anomaly detection* hanya akurat jika integritas log terjamin.
Kontrol Akses (Access Control)	Hanya Wazuh Manager yang boleh mengeksekusi *Active Response* pada gateway. Diperlukan pengaturan hak akses ketat untuk mencegah eksekusi skrip mitigasi oleh pihak tak berwenang.
Validasi Sumber Log	Sistem harus memverifikasi bahwa log hanya berasal dari Wazuh Agent yang terdaftar, sehingga mencegah *injected log* yang dapat menyebabkan false positive/negative.

3.3 Model Perancangan Sistem

Pada subbab ini dibahas perancangan sistem yang mencakup diagram arsitektur dan alur proses sistem secara keseluruhan. Model perancangan ini dirancang untuk menggambarkan bagaimana setiap komponen dalam jaringan bekerja secara terintegrasi dalam mendeteksi dan memitigasi serangan *DDoS* secara otomatis pada lingkungan *small scale network* berbasis *Virtual Machine* (VM).

Arsitektur sistem terdiri atas beberapa komponen utama, yaitu VM penyerang (*SYN Flood*, *UDP Flood*, dan *ICMP Flood*), VM *client* sah, VM server target, *Linux Gateway* sebagai titik kontrol utama, serta Wazuh Manager dan Wazuh Agent sebagai sistem monitoring dan deteksi. Trafik, baik dari *client* sah maupun dari mesin penyerang, akan melewati *Linux Gateway* sebelum mencapai server target. Pada gateway inilah seluruh proses deteksi dan mitigasi dilakukan.

Wazuh Agent yang terpasang pada *Linux Gateway* mengumpulkan log, metrik koneksi, dan aktivitas jaringan secara *real-time*. Data ini kemudian dikirimkan ke Wazuh Manager untuk dianalisis menggunakan rule-based detection dan *anomaly detection* berbasis *threshold* dinamis. Jika pola trafik menunjukkan anomali seperti lonjakan *SYN*, *UDP*, atau *ICMP* yang melebihi ambang batas yang ditentukan, Wazuh Manager akan menghasilkan alert dan memicu mekanisme *Active Response*. *Active Response* kemudian menjalankan skrip mitigasi pada *Linux Gateway* untuk menambahkan black hole route terhadap IP penyerang.

Dengan mekanisme ini, trafik berbahaya akan diarahkan ke *null route* sehingga paket langsung dibuang dan tidak pernah mencapai server target. Sementara itu, trafik dari *client* sah tetap diteruskan secara normal. Alur proses ini memungkinkan sistem menjaga ketersediaan layanan meskipun terjadi serangan, sekaligus mengurangi beban server secara signifikan.

Perancangan ini memberikan gambaran jelas mengenai bagaimana integrasi antara Wazuh, *anomaly detection*, dan *black hole routing* dapat bekerja sebagai sistem otomatisasi respons *DDoS* dalam lingkungan jaringan berskala kecil.

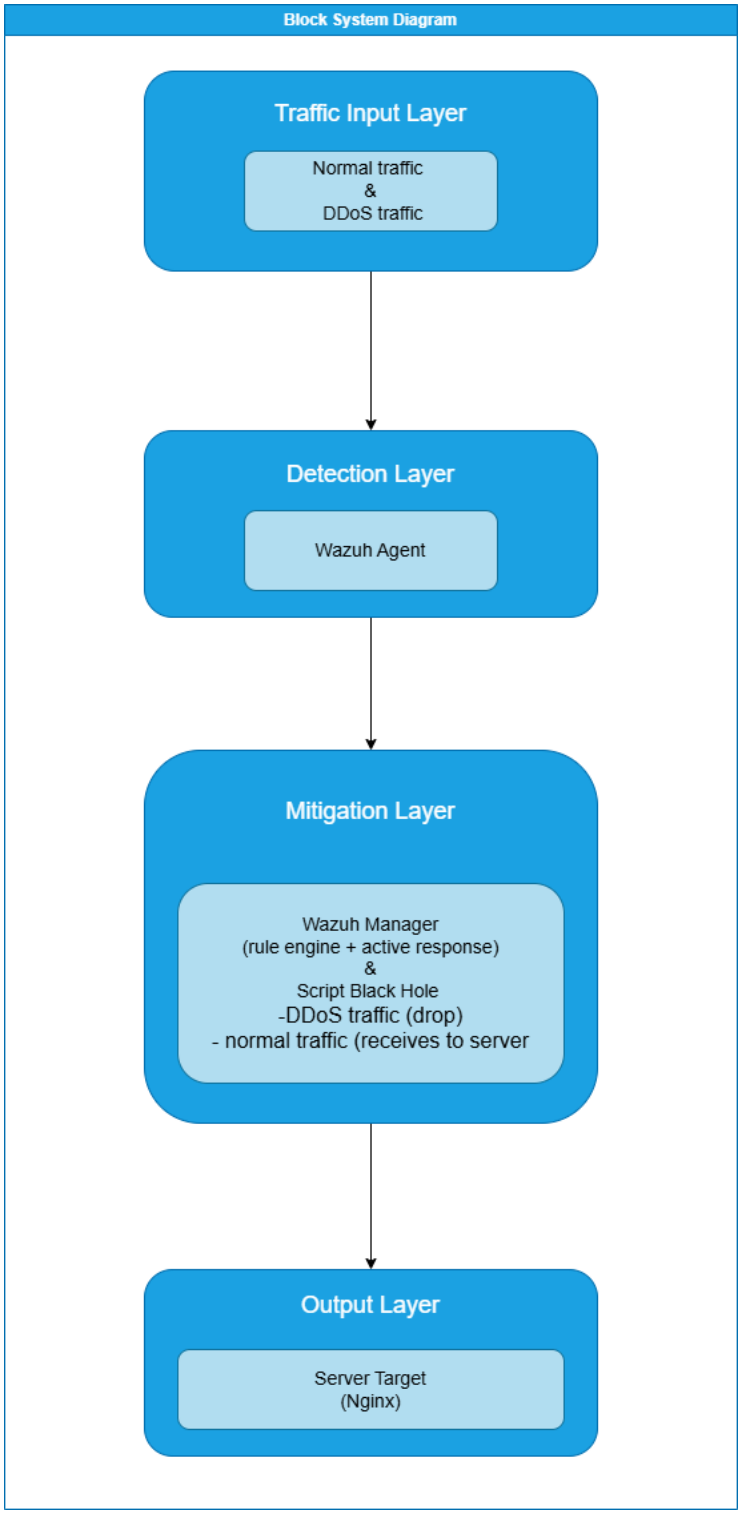
3.3.1 Diagram Blok Sistem

Diagram blok sistem ini menggambarkan alur kerja utama dari sistem deteksi dan mitigasi serangan *DDoS* yang dikembangkan. Sistem dibagi ke dalam empat lapisan (layer) fungsional yang saling berhubungan dan bekerja secara berurutan.

Pada *Traffic Input Layer*, seluruh trafik jaringan—baik trafik sah dari pengguna normal maupun trafik berbahaya dari *attacker* diterima melalui antarmuka jaringan pada Linux Gateway. Lapisan ini tidak melakukan proses analisis, tetapi hanya berfungsi sebagai pintu masuk awal yang menangkap seluruh paket tanpa melakukan filtering.

Trafik yang diterima kemudian diproses pada Detection Layer. Pada tahap ini, *Wazuh Agent* yang berjalan di dalam VM Mitigasi membaca metrik trafik seperti jumlah paket per detik (*PPS*), jumlah koneksi, serta alamat IP sumber. Informasi ini dikirimkan ke Wazuh Manager untuk dianalisis lebih lanjut. Lapisan ini berperan sebagai pusat pendeteksian anomali berbasis log yang menentukan apakah trafik menunjukkan pola normal atau terdapat indikasi serangan.

Jika Wazuh Manager mendeteksi adanya penyimpangan yang memenuhi *threshold* tertentu, proses berlanjut ke Mitigation Layer. Pada lapisan ini, Wazuh Manager mengirimkan *Active Response Command* ke VM Mitigasi untuk mengeksekusi *Blackhole Routing*. Mekanisme ini menyebabkan seluruh trafik dari IP penyerang dibuang sebelum mencapai target, sementara trafik sah tetap diteruskan. Dengan demikian, sistem mampu memisahkan dan mengeliminasi trafik berbahaya tanpa mengganggu pengguna normal.



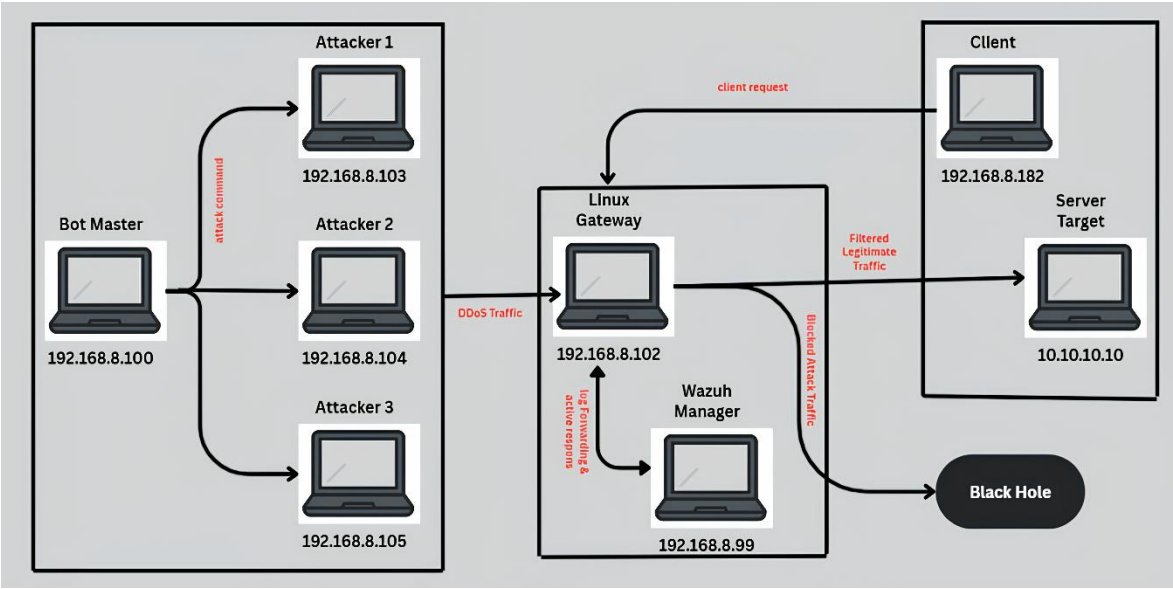
Gambar 3. 1 Diagram Blok Sistem

Tahap akhir adalah Output Layer, yaitu proses penerusan trafik sah menuju VM Target (Nginx). Pada lapisan ini hanya trafik yang telah dinyatakan aman yang dikirimkan ke server, sehingga layanan tetap tersedia dan stabil meskipun terjadi percobaan serangan.

Secara keseluruhan, diagram blok ini menunjukkan alur kerja sistem mitigasi mulai dari penerimaan trafik, deteksi anomali, implementasi *blackhole routing* terhadap IP penyerang, hingga pengiriman trafik sah ke layanan web. Struktur berlapis ini memastikan bahwa proses mitigasi berlangsung secara otomatis, efektif, dan tidak mengganggu aktivitas pengguna normal.

3.3.2 Diagram Arsitektur

Gambar berikut menunjukkan rancangan arsitektur sistem deteksi dan mitigasi serangan DDoS menggunakan Wazuh dan mekanisme blackhole routing. Pada skenario ini, beberapa mesin attacker mengirimkan berbagai jenis serangan seperti SYN Flood, UDP Flood, dan ICMP Flood menuju server target. Seluruh trafik dari attacker dan client sah akan melewati perangkat Linux Gateway yang berperan sebagai Wazuh Agent. Wazuh Agent memonitor aktivitas jaringan dan mengirimkan log secara real-time ke Wazuh Manager. Jika pola serangan terdeteksi, Wazuh Manager akan mengirimkan perintah mitigasi kembali ke Wazuh Agent untuk mengeksekusi blackhole routing. Dengan demikian, trafik berbahaya dapat dibuang sebelum mencapai server target sehingga mengurangi dampak serangan.



Gambar 3. 2 Diagram Arsitektur

Gambar tersebut menggambarkan arsitektur sistem serta alur komunikasi trafik pada penelitian ini, yang dirancang untuk memantau, mendeteksi, dan memitigasi serangan DoS/DDoS menggunakan Linux Gateway dan Wazuh sebagai sistem monitoring dan deteksi.

Pada sisi kiri, terdapat Bot Master yang berfungsi sebagai pengendali serangan. Bot Master mengirimkan perintah serangan (*attack command*) ke tiga mesin Attacker. Ketiga attacker ini kemudian menghasilkan trafik DDoS secara bersamaan menuju Linux Gateway. Trafik ini merepresentasikan serangan terdistribusi yang berasal dari banyak sumber dengan alamat IP yang berbeda.

Seluruh trafik, baik trafik serangan dari attacker maupun permintaan normal dari client, tidak langsung menuju server target, tetapi harus melewati Linux Gateway terlebih dahulu. Linux Gateway berperan sebagai titik kontrol utama yang melakukan routing, filtering, serta pengumpulan log. Trafik normal dari client (client request) akan diteruskan sebagai filtered legitimate traffic menuju server target, sehingga layanan tetap dapat diakses oleh pengguna yang sah.

Linux Gateway juga menjalankan Wazuh Agent yang mengirimkan log aktivitas sistem dan trafik ke Wazuh Manager. Wazuh Manager menganalisis log tersebut untuk mendeteksi adanya anomali atau pola serangan. Ketika serangan teridentifikasi, Wazuh dapat mengirimkan perintah active response kembali ke Linux Gateway.

Sebagai mekanisme mitigasi, Linux Gateway menerapkan blackhole routing. Trafik yang teridentifikasi sebagai serangan akan diarahkan ke Black Hole, sehingga paket berbahaya dibuang dan tidak diteruskan ke server target. Dengan mekanisme ini, server target hanya menerima trafik yang telah difilter dan dinyatakan aman.

Secara keseluruhan, gambar ini menunjukkan bagaimana sistem bekerja secara terpusat melalui Linux Gateway, dengan Wazuh sebagai komponen deteksi dan monitoring, serta blackhole routing sebagai metode mitigasi untuk melindungi server dari serangan DDoS

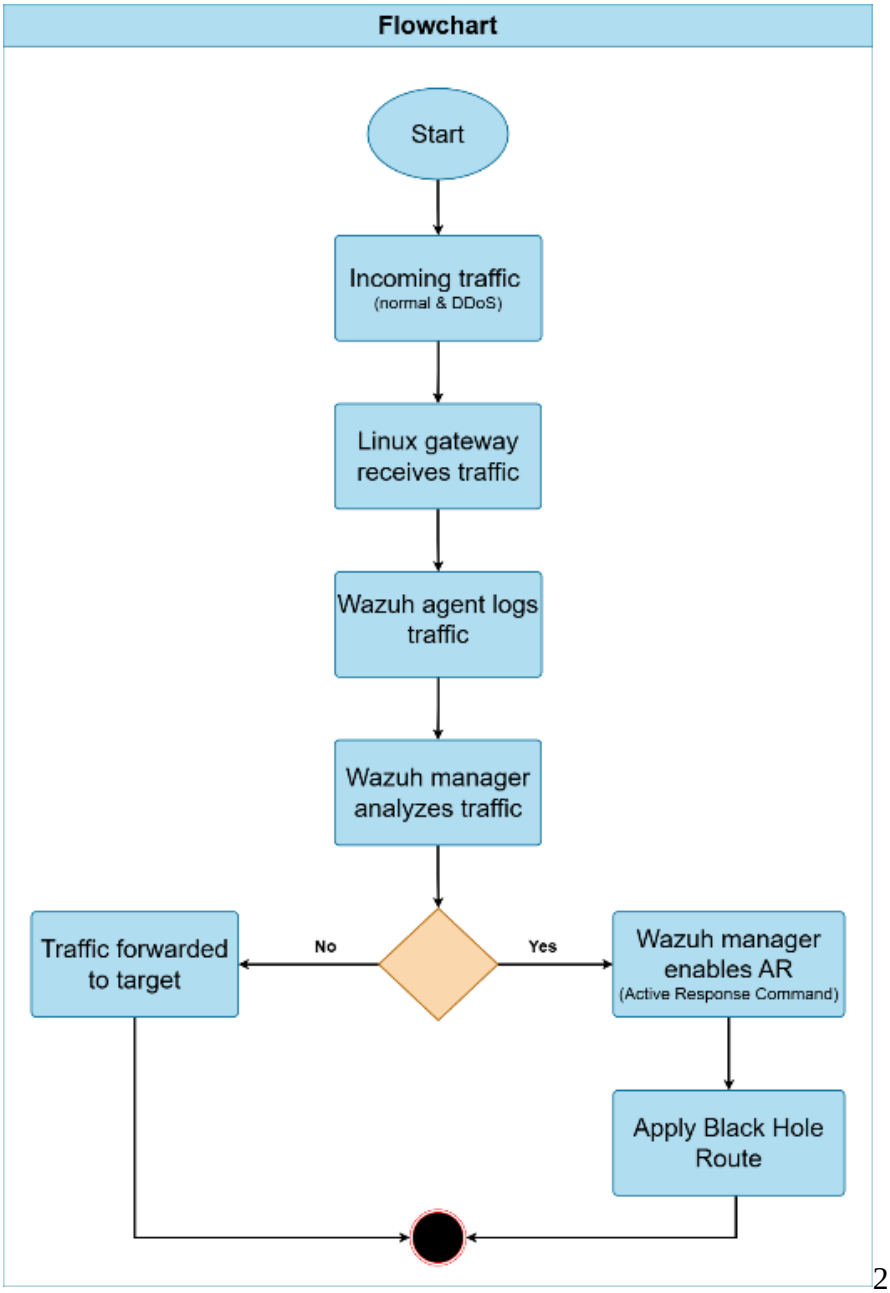
Tabel 3. 4 Arsitektur VM

Nama VM	IP Address	Adapter
VM *Attacker* 1	192.168.8.103	*Bridge*
VM *Attacker* 2	192.168.8.104	*Bridge*
VM *Attacker* 3	192.168.8.105	*Bridge*
VM Bot Master	192.168.8.100	*Bridge*
VM *Client*	192.168.8.182	*Bridge*
VM Gateway (Mitigasi)	192.168.8.102 (NIC1) 192.168.244.1 (NIC2)	*Bridge* + Host-only
VM Wazuh Server	192.168.8.99	*Bridge*
VM Target Server	192.168.244.10	Host-only

3.3.3 Flowchart

Flowchart merupakan diagram alir yang digunakan untuk menggambarkan urutan proses, aliran data, serta logika kerja dari suatu sistem secara visual dan terstruktur. Flowchart membantu memudahkan pemahaman terhadap langkah-langkah operasional yang terjadi dalam suatu proses, mulai dari input, pemrosesan, pengambilan keputusan, hingga output yang dihasilkan. Dengan menggunakan simbol-simbol standar seperti terminator, proses, decision, serta connector, flowchart mampu merepresentasikan logika sistem secara sederhana namun jelas, sehingga mempermudah proses analisis, perancangan, maupun evaluasi sistem.

Dalam konteks penelitian ini, flowchart digunakan untuk menjelaskan alur proses deteksi dan mitigasi serangan *DoS*/*DDoS* mulai dari trafik masuk ke Linux Gateway, proses logging oleh Wazuh Agent, analisis oleh Wazuh Manager, hingga keputusan apakah trafik diteruskan atau dimitigasi melalui *Active Response*. Dengan adanya flowchart, mekanisme kerja sistem dapat dipahami secara menyeluruh dan sistematis.



2

Gambar 3. 3 Flowchart

Flowchart pada Gambar 3.3.4 menjelaskan alur kerja sistem deteksi dan mitigasi serangan *DoS*/*DDoS* yang dibangun dalam penelitian ini. Proses dimulai ketika sistem aktif dan menerima berbagai jenis trafik yang masuk, baik trafik normal yang dikirim oleh *Client* maupun trafik berbahaya yang berasal dari VM *Attacker*. Seluruh trafik tersebut pertama kali tiba di *Linux Gateway* sebagai titik penerima dan pengontrol lalu lintas utama. Setiap paket yang masuk dicatat oleh Wazuh Agent yang berjalan pada Linux Gateway, di mana agent tersebut bertugas melakukan pengumpulan log dan mengirimkannya secara *real-time* ke Wazuh Manager.

Setelah menerima log dari agent, Wazuh Manager melakukan analisis menggunakan ruleset yang telah dikonfigurasi. Pada tahap ini, Wazuh Manager mengidentifikasi apakah pola trafik yang terdeteksi sesuai dengan indikator serangan seperti *SYN Flood*, *UDP Flood*, atau *ICMP Flood*. Jika trafik yang dianalisis tidak menunjukkan anomali, maka *Linux Gateway* akan meneruskan trafik tersebut menuju Target Server sehingga layanan tetap dapat diakses oleh pengguna yang sah. Namun, apabila Wazuh Manager menemukan pola serangan berdasarkan rule tertentu, maka sistem secara otomatis mengaktifkan mekanisme *Active Response*.

Active Response yang dikirim oleh Wazuh Manager memberikan instruksi kepada *Linux Gateway* untuk menjalankan tindakan mitigasi. Dalam penelitian ini, bentuk mitigasi yang digunakan adalah *blackhole routing*, yaitu teknik pemblokiran yang membuang semua trafik dari sumber serangan dengan menambahkan route khusus pada Linux Gateway. Dengan aktifnya blackhole route, trafik dari penyerang tidak lagi diteruskan ke Target Server dan langsung dihentikan pada gateway. Setelah proses mitigasi dilakukan, sistem kembali ke kondisi menunggu trafik berikutnya dan alur ini berjalan secara berulang selama sistem beroperasi.

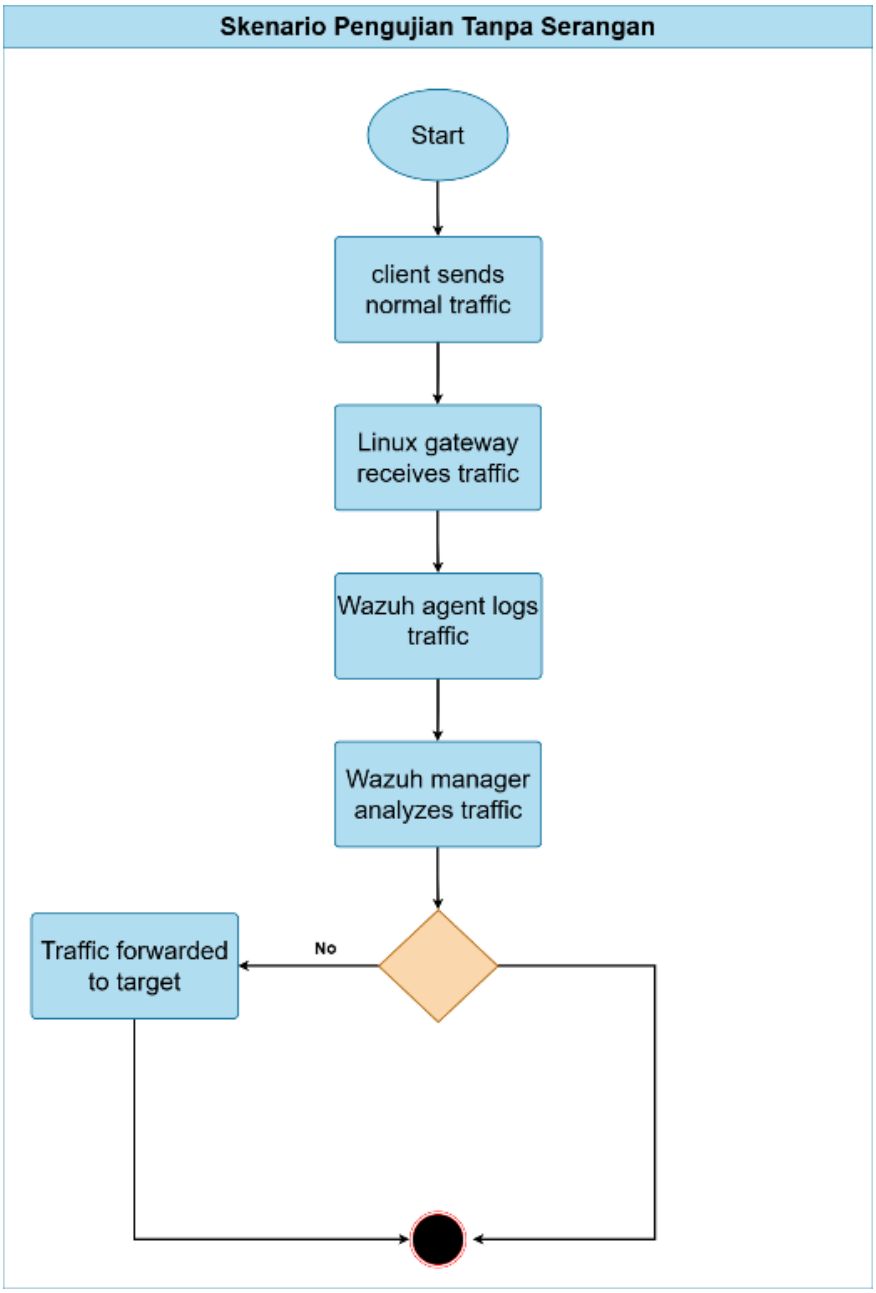
3.4 Skenario Pengujian

Pada bagian ini dijelaskan rancangan skenario pengujian yang digunakan untuk mengevaluasi kinerja sistem dalam mendeteksi dan memitigasi serangan *DoS* dan *DDoS*. Setiap skenario dirancang untuk merepresentasikan kondisi nyata yang kemungkinan terjadi pada sebuah sistem jaringan, mulai dari kondisi normal tanpa adanya serangan hingga kondisi ketika serangan dilakukan secara intensif dari berbagai sumber. Pengujian ini dilakukan secara bertahap untuk memastikan bahwa sistem mampu membedakan antara trafik normal dan trafik berbahaya, serta dapat mengeksekusi mekanisme mitigasi secara otomatis melalui *Linux Gateway* dan Wazuh Manager. Dengan adanya skenario pengujian yang terstruktur, proses evaluasi menjadi lebih terukur, sistematis, dan dapat memberikan gambaran menyeluruh mengenai efektivitas solusi yang diimplementasikan.

3.4.1 Skenario Kondisi Awal (Pengujian Tanpa Serangan)

Pada skenario pengujian awal, sistem diuji tanpa adanya aktivitas serangan untuk memastikan bahwa seluruh komponen bekerja secara normal ketika hanya menerima trafik sah dari *client*. Pada kondisi ini, *client* mengirimkan trafik normal menuju server target melalui Linux Gateway. Gateway berfungsi sebagai titik pemrosesan utama, tempat Wazuh

Agent mencatat setiap aktivitas jaringan seperti jumlah paket, jenis protokol, dan pola trafik yang masuk.



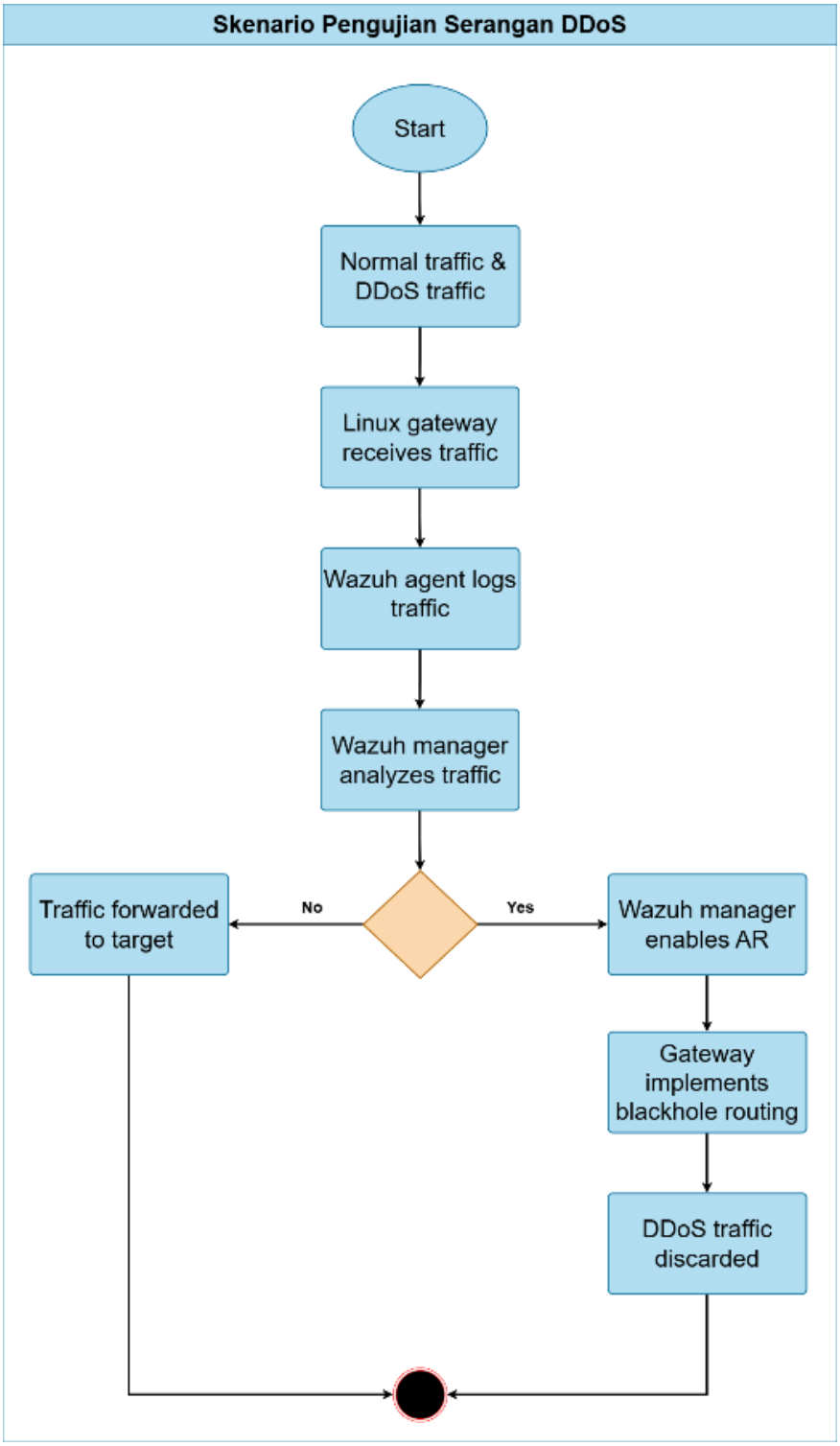
Gambar 3. 4 Skenario Pengujian Tanpa Serangan

Trafik normal tersebut diteruskan ke Wazuh Manager, yang kemudian menganalisis log menggunakan mekanisme rule engine dan *thresholding*. Pada tahap ini tidak ditemukan indikasi anomali, karena seluruh request berada dalam batas ambang yang telah ditentukan. Oleh karena itu, gateway tidak mengaktifkan skema mitigasi apa pun, dan seluruh trafik normal diteruskan ke server target tanpa hambatan.

Selama proses berlangsung, Wazuh Manager tetap melakukan monitoring secara *real-time* untuk memastikan bahwa tidak ada peningkatan trafik mencurigakan, pola flood, atau lonjakan paket yang tidak wajar. Administrator dapat melihat status agent, event log, serta informasi trafik melalui dashboard Wazuh. Pengujian ini sangat penting untuk memverifikasi bahwa sistem tidak hanya mampu memitigasi serangan, tetapi juga dapat bekerja secara stabil dan efisien ketika menangani trafik sah tanpa menimbulkan false positive ataupun pemblokiran yang tidak diperlukan.

3.4.2 Skenario Pengujian Serangan *DDoS*

Pada pengujian ini, sistem dievaluasi dalam kondisi ekstrem di mana beberapa *attacker* secara bersamaan menghasilkan trafik serangan berskala besar (*multi-attacker, multi-vector*). Jenis serangan yang digunakan meliputi *SYN Flood*, *UDP Flood*, dan *ICMP Flood*, yang dikirimkan secara paralel untuk mensimulasikan karakteristik khas serangan *DDoS*. Bersamaan dengan itu, trafik sah dari *client* tetap dikirimkan agar sistem dapat diuji dalam kondisi campuran antara trafik normal dan trafik berbahaya.



Gambar 3. 5 Skenario Pengujian Serangan DDoS

Seluruh trafik tersebut masuk melalui Linux Gateway, yang berfungsi sebagai titik pusat analisis. Di dalam gateway, Wazuh Agent mencatat seluruh paket masuk, termasuk volume paket per detik, jenis protokol, serta pola trafik yang muncul. Informasi ini dikirimkan ke Wazuh Manager, yang kemudian menjalankan analisis berbasis *rule engine* dan *threshold*

untuk mendeteksi adanya anomali. Jika trafik masih berada dalam batas ambang, paket diteruskan ke server target dan proses berjalan seperti kondisi normal.

Namun, ketika Wazuh Manager mendeteksi bahwa lonjakan paket dari satu atau lebih sumber telah melebihi ambang batas yang ditetapkan, sistem segera mengaktifkan *Active Response*. Mekanisme ini memerintahkan *Linux Gateway* untuk menjalankan skrip *blackhole routing*, yaitu menambahkan *null route* terhadap IP penyerang. Dengan *null route* tersebut, seluruh paket dari alamat IP yang teridentifikasi sebagai sumber serangan langsung dibuang (*discarded*) di gateway dan tidak pernah mencapai server target.

3.4.3 Kriteria Keberhasilan Pengujian

Subbab ini menjelaskan kriteria yang digunakan untuk menentukan apakah sistem yang dibangun telah berjalan sesuai dengan tujuan penelitian. Kriteria keberhasilan ditetapkan sebagai acuan dalam mengevaluasi hasil pengujian, baik pada kondisi trafik normal maupun saat terjadi serangan DoS/DDoS. Dengan adanya kriteria ini, proses penilaian kinerja sistem dapat dilakukan secara terukur dan objektif.

Tabel 3. 5 Kriteria Keberhasilan Pengujian

Parameter Pengujian	Kriteria Keberhasilan	Keterangan
Ketersediaan layanan server	*Client* masih dapat mengakses server (ping) selama dan setelah serangan	Jika server tetap accessible maka sistem berhasil menjaga availability
Aktivasi black hole routing	IP *attacker* otomatis diarahkan ke *null route* pada Linux Gateway	Dicek dengan: ip route show atau log *Active Response*
Deteksi serangan oleh Wazuh	Wazuh menghasilkan alert sesuai *threshold* (severity sesuai rule)	Alert tampil pada Dashboard dan berada pada rule yang relevan
Eksekusi *Active Response*	Wazuh menjalankan skrip pemblokiran otomatis ketika *threshold* terlewati	Dapat dilihat dari log: /var/ossec/logs/active-responses.log
Trafik serangan terbuang	*PPS* dari *attacker* tidak lagi mencapai server target setelah mitigasi	Dibuktikan dengan tcpdump/wireshark pada server target
False Positive	Tidak ada trafik sah yang ikut terblokir	Jika *client* tetap bisa akses server maka tidak terjadi false positive

False Negative	Sistem tidak meloloskan serangan yang sudah melewati *threshold*	Jika serangan masih menyebabkan server down maka terjadi false negative

BAB IV
IMPLEMENTASI DAN HASIL PENGUJIAN

Bab ini berisi penjelasan rinci mengenai tahapan implementasi setiap tools yang digunakan dalam pengerjaan tugas akhir. Uraian yang disajikan meliputi tahap instalasi yaitu mencakup instalasi software yang relevan, konfigurasi masing masing komponen agar dapat berfungsi sesuai dengan kebutuhan sistem. Kemudian pengujian sistem untuk memastikan bahwa sistem berjalan dengan baik dan mampu memenuhi kebutuhan sesuai dengan skenario pengujian yang akan membantu mengidentifikasi potensi masalah atau kelemahan yang mungkin terjadi dalam sistem, sehingga solusi perbaikan dapat segera diterapkan jika diperlukan.

4.1 Instalasi dan Konfigurasi

Pada bagian ini dijelaskan proses instalasi dan konfigurasi perangkat lunak pada empat Ubuntu Server dan tiga Kali Linux yang digunakan dalam topologi penelitian. Setiap server dikembangkan untuk memenuhi kebutuhan dalam simulasi monitoring dan mitigasi serangan DDoS terhadap layanan web berbasis Nginx. Tahapan implementasi dimulai dari instalasi sistem operasi Ubuntu Server pada masing-masing node, kemudian dilanjutkan dengan instalasi tools, layanan, serta komponen keamanan yang sesuai dengan peran tiap mesin. Node Ubuntu Server digunakan sebagai Web Server, Linux Gateway, dan Wazuh Server, sedangkan Kali Linux berperan sebagai mesin penyerang yang menghasilkan trafik berlebih untuk menguji efektivitas Wazuh dan mekanisme mitigasi. Dengan struktur ini, penelitian mampu mensimulasikan kondisi jaringan nyata dalam skala kecil secara terisolasi dan aman.

4.1.1 Instalasi dan Konfigurasi pada *Virtual Machine* I (Web Server)

Pada subbab ini dijelaskan tahapan instalasi dan konfigurasi yang dilakukan pada Virtual Machine I yang bertindak sebagai Web Server. Server ini menggunakan Ubuntu Server 22.04 sebagai sistem operasi dan berfungsi sebagai target utama dalam skenario serangan DDoS. Tahapan konfigurasi dimulai dari pemasangan layanan web Nginx sebagai penyedia layanan HTTP, dilanjutkan dengan penyesuaian struktur direktori, pengaturan firewall dasar, serta aktivasi Wazuh Agent untuk mengirimkan log aktivitas sistem dan trafik ke Wazuh Manager. Seluruh konfigurasi dilakukan untuk memastikan Web Server berjalan

stabil, responsif, dan mampu menghasilkan log yang diperlukan untuk keperluan monitoring, analisis, serta deteksi serangan.

4.1.1.1 Instalasi dan Konfigurasi Nginx

Nginx digunakan sebagai layanan web server pada server target dalam pengujian ini. Nginx dipilih karena memiliki arsitektur event-driven yang ringan cepat, dan mampu menangani banyak koneksi secara bersamaan, sehingga sesuai dijadikan target dalam skenario serangan *DDoS* berbasis trafik. Selain itu, konfigurasi Nginx relative sederhana dan stabil untuk lingkungan uji berbasis *Virtual Machine* (VM).

1. **Instalasi Web Server Nginx**

Proses instalasi layanan web Nginx sebagai penyedia layanan HTTP pada server dilakukan pada sistem operasi Linux (Ubuntu) menggunakan perintah berikut:

```
sudo apt update
sudo apt install nginx -y
```

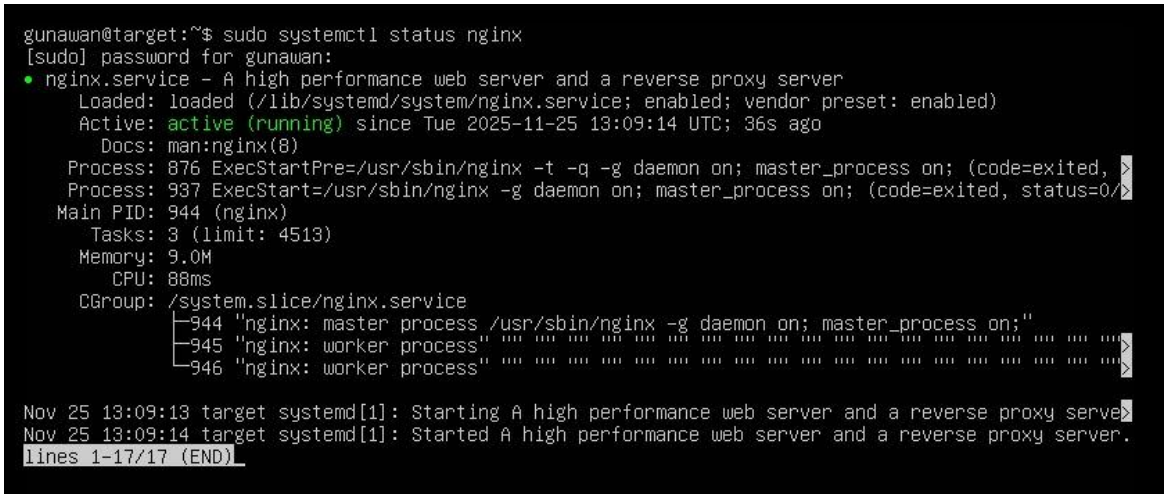
Setelah proses instalasi selesai, status layanan diperiksa untuk memastikan bahwa Nginx berjalan dengan baik. Jika layanan belum aktif, maka dilakukan pengaktifan manual menggunakan:

```
sudo systemctl start nginx
sudo systemctl enable nginx
```

2. **Analisis Hasil Instalasi Nginx**

Untuk memastikan server web berjalan, dilakukan pengecekan status:

```
sudo systemctl status nginx
```



Gambar 4. 1 Status layanan Nginx

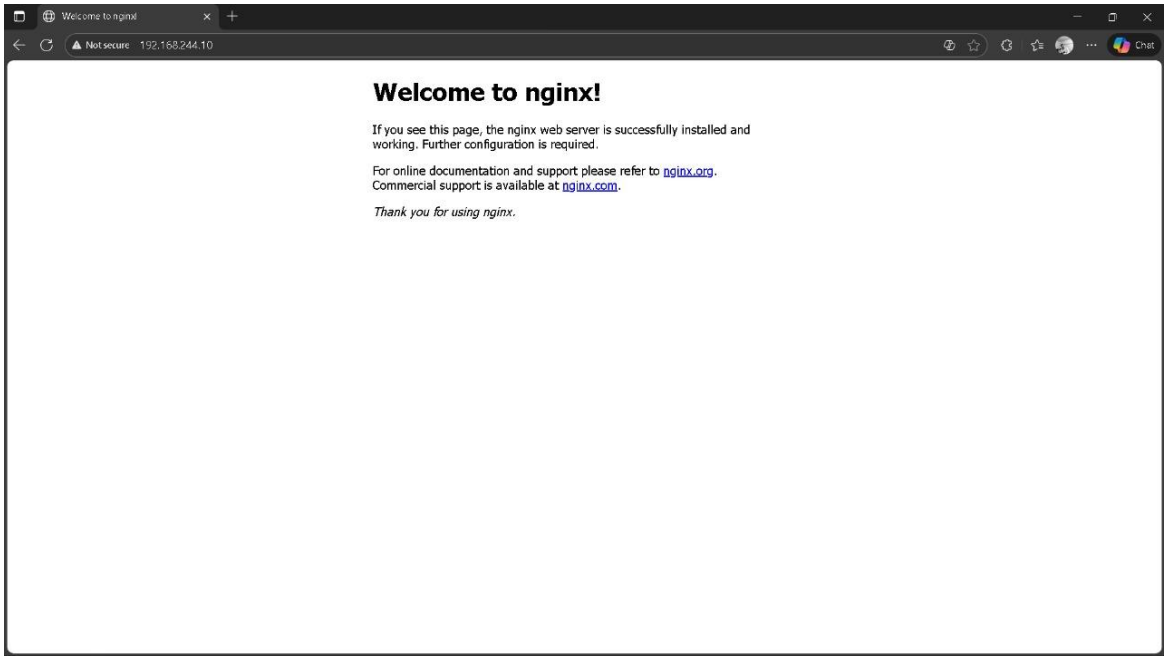
Perintah sudo systemctl status Nginx digunakan untuk memeriksa status layanan nginx. Hasil gambar diatas menunjukkan bahwa layanan dalam kondisi active (running), yang menandakan bahwa Nginx telah berhasil di install dan berjalan dengan baik. Informasi tambahan seperti waktu aktif, PID, dan log ringkas juga mendukung bahwa layanan berjalan stabil tanpa error.

3. Verifikasi Instalasi Nginx

Setelah proses instalasi selesai, layanan Nginx dijalankan secara otomatis. Untuk memverifikasi bahwa Nginx telah berjalan dengan baik, pengguna dapat membuka browser dan mengakses alamat IP dari server Nginx melalui:

```
curl http://<alamat-ip-server>
```

Jika proses instalasi berhasil, maka sistem akan menampilkan halaman default dari Nginx melalui browser. Halaman ini berfungsi sebagai indikator bahwa layanan web server telah berjalan dengan baik dan siap untuk digunakan dalam proses selanjutnya, seperti konfigurasi situs, implementasi modul keamanan, atau pengujian sistem



Gambar 4. 2 Halaman default Nginx

Tampilan di atas menunjukkan hasil akses terhadap alamat IP server, yaitu 192.162.244.10, yang digunakan dalam penelitian ini. Web server Nginx telah berhasil diinstal dan berjalan dengan baik, ditandai dengan munculnya halaman default Nginx yang menampilkan pesan “Welcome to nginx”. Halaman ini merupakan indikasi bahwa layanan Nginx telah aktif dan siap digunakan untuk proses konfigurasi lebih lanjut.

4.1.1.2 Instalasi dan Konfigurasi Wazuh Agent

Pada tahap ini dilakukan instalasi dan konfigurasi Wazuh Agent pada Web Server agar seluruh aktivitas sistem dapat dikirimkan ke Wazuh Manager untuk dianalisis. Wazuh Agent berfungsi sebagai komponen utama pada sisi host yang akan mengumpulkan log sistem, log aplikasi, serta data keamanan lainnya. Proses instalasi dilakukan melalui repository resmi Wazuh dan disesuaikan agar agent dapat terhubung dengan Wazuh Manager pada jaringan penelitian.

Langkah-langkah instalasi dan konfigurasi Wazuh Agent:

1. **Menambahkan Repository Wazuh ke Sistem**

Repository Wazuh perlu ditambahkan terlebih dahulu agar paket Wazuh Agent dapat di unduh secara resmi.

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo apt-key
add -
echo "deb https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee
/etc/apt/sources.list.d/wazuh.list
sudo apt update
```

2. **Menginstal Paket wazuh Agent**

Setelah repository berhasil ditambahkan, isntalasi Wazuh Agent dilakukan dengan perintah berikut:

```
sudo apt install wazuh-agent -y
```

3. **Konfigurasi file ossec.conf**

Agar Agent terhubung dengan Wazuh Manager, konfigurasi dilakukan pada file:

```
sudo nano /var/ossec/etc/ossec.conf
```

Pada bagian client, dilakukan penyesuain dengan menambahkan alamat IP dari Wazuh Manager sebagai berikut:

```
<ossec_config>
  <client>
    <server>
      <address>192.168.8.99</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
```

Gambar 4. 3 Konfigurasi ossec.conf pada Wazuh Agent

Konfigurasi ini menentukan bahwa Wazuh Agent akan mengirim log ke Wazuh Manager melalui alamat 192.168.8.99 menggunakan protokol TCP pada port 1514.

4. **Mengaktifkan dan Menjalankan Layanan Wazuh Agent**

Setelah konfigurasi selesai, layanan Wazuh Agent dijalankan dan di buat otomatis aktif setiap kali server menyala:

```
sudo systemctl start wazuh-agent
sudo systemctl enable wazuh-agent
```

5. **Verifikasi Konektivitas Agent**

Status Agent diperiksa menggunakan:

```
sudo systemctl status wazuh-agent
```

Jika koneksi berhasil, agent akan muncul sebagai active di Wazuh Dashboard pada sisi server. Hal ini menandakan bahwa Web Server telah terhubung secara penuh dengan sistem monitoring.

```
client@client:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
     Active: active (running) since Wed 2025-12-10 11:11:03 UTC; 3h 22min ago
      Tasks: 33 (limit: 9436)
     Memory: 313.1M (peak: 341.7M)
        CPU: 1min 48.676s
     CGroup: /system.slice/wazuh-agent.service
             ├─883 /var/ossec/bin/wazuh-execd
             ├─894 /var/ossec/bin/wazuh-agentd
             ├─907 /var/ossec/bin/wazuh-syscheckd
             ├─920 /var/ossec/bin/wazuh-logcollector
             └─938 /var/ossec/bin/wazuh-modulesd
```

Gambar 4. 4 Status Layanan Wazuh Agent

4.1.2 Instalasi dan Konfigurasi pada *Virtual Machine* II (Linux Gateway)

Pada sub bab ini dijelaskan proses instalasi dan konfigurasi pada *Virtual Machine* yang berfungsi sebagai Linux Gateway. *Linux Gateway* merupakan komponen utama dalam arsitektur penelitian ini karena seluruh trafik, baik trafik normal maupun trafik serangan, akan melewati perangkat ini sebelum menuju Target Server. Oleh karena itu, *Linux Gateway* harus dikonfigurasi dengan benar agar dapat menjalankan fungsi routing, *firewall*, logging, serta eksekusi *active response* melalui integrasi dengan Wazuh Agent. Tahapan pada bagian ini mencakup instalasi sistem dasar, konfigurasi jaringan, pemasangan Wazuh Agent, serta penerapan mekanisme mitigasi seperti *iptables* dan *blackhole routing*. Seluruh konfigurasi dilakukan untuk memastikan *Linux Gateway* dapat beroperasi sebagai titik kontrol dan mitigasi yang efektif terhadap serangan *DDoS*.

4.1.2.1 Konfigurasi Antarmuka Jaringan (Network Interface)

Linux Gateway memiliki dua network interface:

Tabel 4. 1 Konfigurasi Network Interface

Interface	Mode	Fungsi
ens33	*Bridge Adapter*	Terhubung ke jaringan utama (192.168.8.0/24)
ens37	*Host-Only*	Terhubung ke jaringan internal server target

Konfigurasi statis ditulis pada file:

```
sudo nano /etc/netplan/01-mitigasi.yaml
```

```
GNU nano 6.2                    /etc/netplan/01-mitigasi.yaml
network:
  version: 2
  renderer: networkd

  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.8.102/24
      routes:
        - to: default
          via: 192.168.8.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 1.1.1.1

    ens37:
      dhcp4: no
      addresses:
        - 10.10.10.1/24
```

Gambar 4. 5 Konfigurasi IP statis

Gambar diatas menunjukkan konfigurasi jaringan pada Linux Gateway yang di atur melalui file Netplan. Pada konfigurasi ini, gateway memiliki 2 interface yang amsing-masing menjalankan fungsi berbeda. Interface pertama, yaitu ens33 dikonfigurasi tanpa DHCP dan diberikan alamat IP statis 192.168.8.102/24 untuk terhubung dengan jaringan eksternal. Interface ini juga memiliki rute default yang diarahkan ke 192.168.8.1, sehingga seluruh koneksi menuju jaringan luar akan melalui gateway tersebut. Selain itu, nameserver menggunakan DNS publik 8.8.8.8 dan 1.1.1.1 untuk kebutuhan resolusi domain. Interface kedua, yaitu ens37, berfungsi sebagai koneksi ke jaringan internal Host-Only dan diberikan alamat IP statis 10.10.10.1/24. Melalui antarmuka ini, Linux Gateway menghubungkan server target yang berada dalam jaringan internal, sekaligus menjadi jalur lalu lintas antara

attacker, agent, dan server target. Secara keseluruhan, konfigurasi ini mempersiapkan Linux Gateway untuk menjalankan fungsi routing, meneruskan trafik, serta mendukung mekanisme mitigasi seperti blackhole routing dalam lingkungan pengujian.

Setelah konfigurasi selesai, terapkan dengan:

```
sudo netplan apply
```

```
gunawan@mitigasi:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:76:71:67 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.8.102/24 brd 192.168.8.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe76:7167/64 scope link
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:76:71:71 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 10.10.10.1/24 brd 10.10.10.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe76:7171/64 scope link
       valid_lft forever preferred_lft forever
gunawan@mitigasi:~$
```

Gambar 4. 6 Hasil konfigurasi netplan

4.1.2.3 Mengaktifkan IP Forwarding

Agar Linux Gateway dapat meneruskan paket antar jaringan, fitur IP forwarding perlu diaktifkan. Pengaturan dilakukan dengan membuka:

```
sudo nano /etc/sysctl.conf
```

```
###################################################################
# Magic system request Key
# 0=disable, 1=enable all, >1 bitmask of sysrq functions
# See https://www.kernel.org/doc/html/latest/admin-guide/sysrq.html
# for what other values do
#kernel.sysrq=438

net.ipv4.ip_forward=1
```

Gambar 4. 7 Aktivasi IP Forwarding

Gambar diatas menujukkan konfiguarsi pada file /etc/sysctl.conf yang digunakan untuk mengaktifkan fitur forwarding secara default berada dalam kondisi nonaktif untuk mencegah perangkat bertindak sebagai ro uter tanpa izin. Agar Linux Gateway dapat

meneruskan paket dari jaringan internal menuju jaringan eksternal, parameter ini harus diaktifkan secara manual.

Baris konfigurasi net.ipv4.ip_forward=1 yang terlihat pada gambar merupakan instruksi untuk mengaktifkan fungsi penerusan paket IPv4. Dengan nilai 1, kernel Linux mengizinkan setiap paket yang masuk melalui satu antarmuka jaringan untuk diteruskan ke antarmuka lainnya. Pengaturan ini sangat penting dalam penelitian ini karena Linux Gateway berperan sebagai perantara antara Web Server, Mail Server, dan koneksi internet melalui modem Telkomsel Orbit. Tanpa pengaktifan parameter ini, gateway tidak akan mampu melakukan routing dan seluruh trafik yang melewati server akan terblokir secara otomatis.

4.1.2.4 Konfigurasi Firewall dan NAT menggunakan iptables

Gateway harus menerapkan NAT agar jaringan internal dapat mengakses internet dan trafik bisa melewati gateway.

NAT Masquerade:

```
sudo iptables -t nat -A POSTROUTING -o ens33 -j MASQUERADE
```

Forwarding Trafik antar jaringan:

```
sudo iptables -A FORWARD -i ens33 -o ens37 -j ACCEPT
sudo iptables -A FORWARD -i ens37 -o ens33 -j ACCEPT
```

4.1.2.2 Instalasi dan Registrasi Wazuh Agent

Untuk memungkinkan monitoring aktivitas jaringan dan pengaktifan active response, Wazuh Agent diinstall pada Linux Gateway. Instalasi dimulai dengan menambahkan repository Wazuh, memperbarui repositori, kemudian menginstal paket agent:

Langkah Instalasi:

1. **Download packet agent**

```
Wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuhagent/wazuh-agent_4.14.1-1_amd64.deb
```

2. **Instal dan daftarkan Agent ke Wazuh server**

```
sudo apt install wazuh-agent -y
sudo WAZUH_MANAGER='192.168.8.99' WAZUH_AGENT_NAME='gateway' dpkg -i ./wazuh-agent_4.14.1-1_amd64.deb
```

3. **Konfigurasi agent:**

```
sudo nano /var/ossec/etc/ossec.conf
```

```
<ossec_config>
  <client>
    <server>
      <address>192.168.8.99</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
```

Gambar 4. 8 Konfigurasi ossec.conf pada Wazuh Agent

Konfigurasi ini menentukan bahwa Wazuh Agent akan mengirim log ke Wazuh Manager melalui alamat 192.168.8.99 menggunakan protokol TCP pada port 1514.

4. **Reload dan aktifkan service**

Setelah seluruh konfigurasi Wazuh Agent selesai dilakukan, Langkah selanjutnya adalah melalukan reload konfigurasi sistem dan mengaktifkan layanan Wazuh Agent. Proses ini bertujuan agar perubahan konfigurasi yang telah dibuta dapat diterapkan dengan benar serta memastikan Wazuh Agent berjalan otomatis saat sistem dinyalakan dan aktif melakukan pengiriman log ke Wazuh Manager.

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

5. **Validasi Instalasi Wazuh Agent pada VM Mitigasi**

Pemeriksaan status layanan bertujuan untuk memverifikasi bahwa Wazuh Agent aktif, tidak mengalami error, serta siap mengirimkan log dan event ke Wazuh Manager.

```
sudo systemctl status wazuh-agent
```

```
gunawan@mitigasi: ~
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2025-11-30 13:27:26 UTC; 37min ago
      Tasks: 28 (limit: 4515)
     Memory: 95.4M
        CPU: 3.901s
     CGroup: /system.slice/wazuh-agent.service
             ├─1033 /var/ossec/bin/wazuh-execd
             ├─1057 /var/ossec/bin/wazuh-agentd
             ├─1092 /var/ossec/bin/wazuh-syscheckd
             ├─1209 /var/ossec/bin/wazuh-logcollector
             └─1256 /var/ossec/bin/wazuh-modulesd

Nov 30 13:27:19 mitigasi env[935]: Deleting PID file '/var/ossec/var/run/wazuh-syscheckd-1098.pid' not used...
Nov 30 13:27:19 mitigasi env[935]: Deleting PID file '/var/ossec/var/run/wazuh-agentd-1061.pid' not used...
Nov 30 13:27:19 mitigasi env[935]: Deleting PID file '/var/ossec/var/run/wazuh-execd-1037.pid' not used...
Nov 30 13:27:20 mitigasi env[935]: Started wazuh-execd...
Nov 30 13:27:21 mitigasi env[935]: Started wazuh-agentd...
Nov 30 13:27:22 mitigasi env[935]: Started wazuh-syscheckd...
Nov 30 13:27:23 mitigasi env[935]: Started wazuh-logcollector...
Nov 30 13:27:24 mitigasi env[935]: Started wazuh-modulesd...
Nov 30 13:27:26 mitigasi env[935]: Completed.
Nov 30 13:27:26 mitigasi systemd[1]: Started Wazuh agent.
~
~
```

Gambar 4. 9 Status layanan Wazuh Agent

4.1.2.3 Pembuatan Skrip Mitigasi

Skrip blackhole routing di:

```
#!/bin/bash
IP="$1"
if [[ $IP =~ ^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$ ]]; then
  sudo /sbin/ip route add blackhole "$IP" 2>/dev/null
  echo "$(date) BLOCKED $IP" >> /var/log/ddos-block.log
fi
```

4.1.2.4 Verifikasi Layanan Gateway

Pada tahap ini dilakukan verifikasi konfigurasi jaringan pada Linux Gateway unutk memastikan seluruh antarmuka jaringan dan tabel routing telah dikonfigurasi dengan benar.

```
gunawan@mitigasi:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:76:71:67 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.8.102/24 brd 192.168.8.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe76:7167/64 scope link
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default ql
en 1000
    link/ether 00:0c:29:76:71:71 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 10.10.10.1/24 brd 10.10.10.255 scope global ens37
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe76:7171/64 scope link
       valid_lft forever preferred_lft forever
gunawan@mitigasi:~$ ip r
default via 192.168.8.1 dev ens33 proto static
10.10.10.0/24 dev ens37 proto kernel scope link src 10.10.10.1
blackhole 10.10.10.99
192.168.8.0/24 dev ens33 proto kernel scope link src 192.168.8.102
gunawan@mitigasi:~$
```

Gambar 4. 10 Verifikasi Konfigurasi pada Linux Gateway

Hasil pemeriksaan menunjukkan bahwa Linux Gateway memiliki dua antarmuka aktif, yaitu ens33 yang terhubung ke jaringan eksternal (192.168.8.0/24) dan ens37 yang terhubung ke jaringan internal target (10.10.10.0/24). Selain itu, tabel routing menunjukkan bahwa mekanisme blackhole routing telah diterapkan.

4.1.3 Konfigurasi pada *Virtual Machine* III (Wazuh Server)

Pada sub bab ini dijelaskan proses instalasi dan konfigurasi seluruh komponen utama yang berjalan pada Wazuh server. VM ini menjalankan paket Wazuh All-in-One**,** sehingga seluruh komponen seperti Wazuh Manager, Wazuh Indexer (OpenSearch), Wazuh Dashboard, Filebeat, serta API Wazuh terpasang secara otomatis.

4.1.3.1 Instalasi Sistem Dasar Wazuh Server

Sub bab ini berisi instalasi awal sistem operasi dan paket pendukung sebelum pemasangan Wazuh.

Perintah dasar update OS:

```
sudo apt update && sudo apt upgrade -y
```

Instalasi dependencies wajib untuk Wazuh:

```
sudo apt install curl apt-transport-https lsb-release gnupg unzip -y
```

4.1.3.2 Penambahan Repository wazuh

Agar sistem dapat mengunduh paket resmi Wazuh, repository Wazuh ditambahkan ke dalam sistem menggunakan perintah berikut:

Perintah:

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo apt-key add
-
echo "deb https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee
/etc/apt/sources.list.d/wazuh.list
sudo apt update
chmod +x wazuh-install.sh
sudo bash wazuh-install.sh --all-in-one
```

Dengan penambahan repositori ini, semua komponen Wazuh dapat diinstal langsung melalui manajer paket Ubuntu.

4.1.3.3 Instalasi Wazuh Indexer

Wazuh Indexer adalah mesin penyimpanan dan pencarian berbasis OpenSearch yang menjadi fondasi penyimpanan log dan alert. Instalasi dilakukan dengan menjalankan:

```
sudo apt install wazuh-indexer -y
```

Setelah instalasi selesai, layanan dijalankan otomatis. Statusnya dapat diperiksa melalui:

```
sudo systemctl status wazuh-indexer
```

```
server@server:~$ sudo systemctl status wazuh-indexer
● wazuh-indexer.service - wazuh-indexer
     Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
     Active: active (running) since Tue 2025-12-02 01:47:17 UTC; 35min ago
       Docs: https://documentation.wazuh.com
   Main PID: 5687 (java)
      Tasks: 93 (limit: 9205)
     Memory: 1.5G (peak: 1.5G)
        CPU: 1min 43.791s
     CGroup: /system.slice/wazuh-indexer.service
             └─5687 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networka>

Dec 02 01:46:57 server systemd-entrypoint[5687]: WARNING: System::setSecurityManager has bee>
Dec 02 01:46:57 server systemd-entrypoint[5687]: WARNING: Please consider reporting this to >
Dec 02 01:46:57 server systemd-entrypoint[5687]: WARNING: System::setSecurityManager will be>
Dec 02 01:46:58 server systemd-entrypoint[5687]: Dec 02, 2025 1:46:58 AM sun.util.locale.pro>
Dec 02 01:46:58 server systemd-entrypoint[5687]: WARNING: COMPAT locale provider will be rem>
Dec 02 01:46:59 server systemd-entrypoint[5687]: WARNING: A terminally deprecated method in >
Dec 02 01:46:59 server systemd-entrypoint[5687]: WARNING: System::setSecurityManager has bee>
Dec 02 01:46:59 server systemd-entrypoint[5687]: WARNING: Please consider reporting this to >
Dec 02 01:46:59 server systemd-entrypoint[5687]: WARNING: System::setSecurityManager will be>
Dec 02 01:47:17 server systemd[1]: Started wazuh-indexer.service - wazuh-indexer.
```

Gambar 4. 11 Status layanan Wazuh-Indexer

4.1.3.4 Instalasi Wazuh-Manager

Wazuh Manager berfungsi untuk menganalisis log, menjalankan rule deteksi, serta mengoordinasikan seluruh agent. Instalasinya dilakukan dengan perintah:

```
sudo apt install wazuh-manager -y
```

Setelah terpasang, layanan diuji dengan:

```
sudo systemctl status wazuh-manager
```

```
server@server:~$ sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
     Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
     Active: deactivating (stop) since Tue 2025-12-02 02:21:35 UTC; 6s ago
    Process: 11883 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, s>
  Cntrl PID: 20336 (wazuh-control)
      Tasks: 100 (limit: 9205)
     Memory: 2.4G (peak: 4.7G)
        CPU: 21min 45.723s
     CGroup: /system.slice/wazuh-manager.service
             ├─12261 /var/ossec/bin/wazuh-modulesd
             ├─20336 /bin/sh /var/ossec/bin/wazuh-control stop
             └─20413 sleep 1

Dec 02 02:21:35 server env[20336]: Killing wazuh-analysisd...
Dec 02 02:21:35 server env[20336]: wazuh-maild not running...
Dec 02 02:21:35 server env[20336]: Killing wazuh-execd...
Dec 02 02:21:35 server env[20336]: Killing wazuh-db...
Dec 02 02:21:35 server env[20336]: Killing wazuh-authd...
Dec 02 02:21:35 server env[20336]: wazuh-agentlessd not running...
Dec 02 02:21:35 server env[20336]: wazuh-integratord not running...
Dec 02 02:21:35 server env[20336]: wazuh-dbd not running...
Dec 02 02:21:35 server env[20336]: wazuh-csyslogd not running...
Dec 02 02:21:35 server env[20336]: Killing wazuh-apid...
```

Gambar 4. 12 Status layanan Wazuh-Manager

4.1.3.5 Instalasi Wazuh Dashboard

Wazuh Dasboard digunakan untuk menampilkan visualisasi log, status agent, alert keamanan, serta hasil deteksi serangan. Instalasi dilakukan dengan:

```
sudo apt install wazuh-dashboard -y
```

Dashboard secara otomatis berjalan sebagai layanan terpisah. Statusnya diperiksa melalui:

```
sudo systemctl status wazuh-dashboard
```

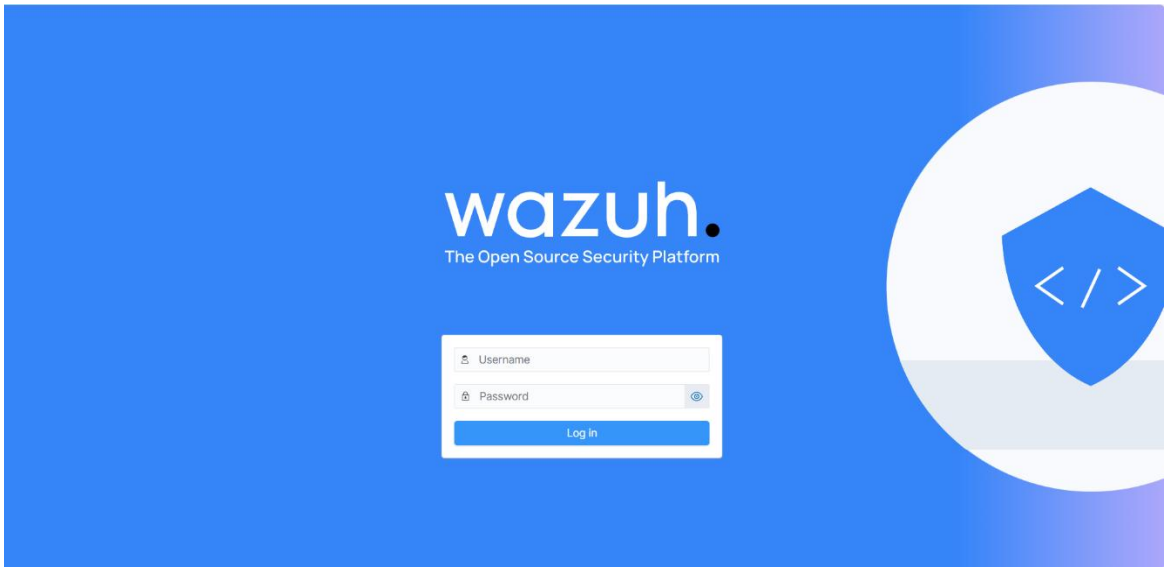
```
server@server:~$ sudo systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
     Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
     Active: active (running) since Tue 2025-12-02 02:16:44 UTC; 7min ago
   Main PID: 20130 (node)
      Tasks: 11 (limit: 9205)
     Memory: 199.5M (peak: 341.8M)
        CPU: 13.188s
     CGroup: /system.slice/wazuh-dashboard.service
             └─20130 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-heade>

Dec 02 02:17:33 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:33 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:33 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:33 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:33 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:33 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:33 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:34 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:34 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
Dec 02 02:17:34 server opensearch-dashboards[20130]: {"type":"response","@timestamp":"2025-1>
```

Gambar 4. 13 Status layanan Wazuh-Dasboard

Akses dashboard dapat dilakukan melalui browser dengan mengunjungi:

```
https://<IP-Wazuh-Server>:5601
```



Gambar 4. 14 Dashboard Platform Wazuh

4.1.3.6 Instalasi Filebeat

Pada tahap ini dilakukan proses instalasi Filebeat sebagai komponen tambahan untuk mendukung pengiriman log dari Wazuh Manager ke Wazuh Indexer. Filebeat berfungsi sebagai *lightweight log shipper* yang bertugas mengekspor log dari berbagai sumber pada Wazuh Server dan mengirimkannya ke Indexer untuk dilakukan pengindeksan dan pencarian. Meskipun Wazuh telah memiliki mekanisme internal untuk mengelola log, penggunaan Filebeat memberikan efisiensi yang lebih baik dalam proses pengiriman log

dalam jumlah besar, terutama selama simulasi serangan DDoS yang menghasilkan trafik dan alert dengan intensitas tinggi. Oleh karena itu, instalasi Filebeat diperlukan agar proses distribusi log berjalan optimal, stabil, dan terstruktur. Bagian berikut menjelaskan langkah-langkah instalasi dan konfigurasi Filebeat pada Wazuh Server.

```
server@server:~$ sudo systemctl status filebeat
● filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
     Loaded: loaded (/usr/lib/systemd/system/filebeat.service; enabled; preset: enabled)
     Active: active (running) since Tue 2025-12-02 01:17:47 UTC; 1h 7min ago
       Docs: https://www.elastic.co/products/beats/filebeat
   Main PID: 940 (filebeat)
      Tasks: 9 (limit: 9205)
     Memory: 52.2M (peak: 52.7M)
        CPU: 1.367s
     CGroup: /system.slice/filebeat.service
             └─940 /usr/share/filebeat/bin/filebeat --environment systemd -c /etc/filebeat/f>
```

Gambar 4. 15 Status layanan Filebeat

4.1.3.7 Integrasi Wazuh Manager, Indexer, dan Dashboard

Setelah seluruh komponen terinstal, dilakukan integrasi melalui skrip bawaan Wazuh untuk menghasilkan sertifikat, menghubungkan komponen, serta memastikan proses komunikasi berjalan aman. Integrasi dilakukan melalui:

```
sudo /usr/share/wazuh-indexer/opensearch-tar-installation.sh
sudo /usr/share/wazuh-dashboard/configure-dashboard.sh
```

4.1.4 Instalasi dan Konfigurasi pada *Virtual Machine* IV (*Client*)

Virtual Machine (VM) *Client* digunakan sebagai sumber trafik normal (*legitimate traffic*) dalam skenario pengujian. VM ini berfungsi untuk mengakses layanan web server secara berkala sehingga dapat dijadikan pembanding dengan trafik berbahaya yang dikirim oleh VM *Attacker*. Pada VM ini dilakukan proses instalasi sistem dasar, konfigurasi jaringan, serta pengujian konektivitas untuk memastikan bahwa *Client* dapat mengakses server target tanpa hambatan.

4.1.4.1 Instalasi Sistem Dasar pada Client

Tahap ini meliputi instalasi sistem operasi dan paket perangkat lunak pendukung untuk kebutuhan pengujian. Sistem operasi yang digunakan adalah Ubuntu Server, yang kemudian diperbarui agar berada dalam kondisi stabil.

Perintah:

```
sudo apt update && sudo apt upgrade -y
sudo apt install curl net-tools iputils-ping -y
```

Perintah di atas memastikan VM *Client* telah memiliki tool dasar seperti ping, curl, dan utilitas jaringan lainnya untuk melakukan pengujian konektivitas.

4.1.4.2 Konfigurasi Jaringan Client

Pengaturan network adapter dilakukan agar VM *Client* dapat berada pada segmen jaringan yang sama dengan Linux Gateway, Wazuh Server, dan VM *Attacker*

Konfigurasi dilakukan melalui file:

```
/etc/netplan/01-network-manager-all.yaml
```

```
  GNU nano 7.2                /etc/netplan/01-network-manager-all.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.8.182/24
      routes:
        - to: 0.0.0.0/0
          via: 192.168.8.102
        - to: 10.10.10.0/24
          via: 192.168.8.102
      nameservers:
        addresses:
          - 8.8.8.8
          - 1.1.1.1
```

Gambar 4. 16 Konfigurasi Jaringan Client

Pada tahap ini dilakukan konfigurasi jaringan pada Virtual Machine Client menggunakan Neplan untuk memastikan client dapat berkomunikasi dengan Linux Gateway dan Server Target. Client dikonfigurasikan menggunakan alamat IP statis 192.168.8.182/24 pada interface enp0s3. Seluruh trafik keluar serta trafik menuju jaringan target diarahkan melalui Linux Gateway dengan alamat IP 192.168.8.102. konfigurasi ini memastikan bahwa seluruh akses client ke target selalu melewati gateway, sehingga trafik dapat dimonitor dan dikontrol sesuai rancangan sistem.

4.1.4.3 Konfigurasi Wazuh Agent

Pada tahap ini dilakukan instalasi dan konfigurasi Wazuh Agent pada Virtual Machine Client agar seluruh aktivitas sistem client dapat dimonitor dan dikirmkan ke Wazuh Server. Dengan terpasangnya Wazuh Agent, log sistem seperti login, aktivitas layanan, serta event kemanan dasar dapat dikumpulkan secara terpusat.

1. **Menambahkan repository Wazuh:**

Repositori Wazuh ditambahkan agar sistem dapat mengunduh paket Wazuh Agent secara resmi

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo apt-key add
-echo "deb https://packages.wazuh.com/4.x/apt stable main" | sudo tee
/etc/apt/sources.list.d/wazuh.listsudo apt update
```

2. **Instal wazuh Agent**

Setelah repositori berhasil ditambahkan, Wazuh Agent diinstal menggunakan perintah berikut.

```
sudo apt install wazuh-agent -y
```

3. **Konfigurasi Agent**

File konfigurasi utama Wazuh Agent diedit untuk menentukan alamat Wazuh Manager yang akan menerima log.

```
sudo nano /var/ossec/etc/ossec.conf
```

```
<ossec_config>
  <client>
    <server>
      <address>192.168.8.99</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
```

Gambar 4. 17 Konfigurasi ossec.conf pada Wazuh Agent

Pada file ini, alamat IP Wazuh Manager disesuaikan dengan topologi jaringan yang digunakan.

4. **Mengaktifkan Agent**

Setelah konfigurasi selesai, layanan Wazuh Agent diaktifkan dan dijalankan agar dapat beroperasi secara otomatis saat sistem menyala.

```
sudo systemctl start wazuh-agent
sudo systemctl enable wazuh-agent
```

```
client@client:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-11 15:48:57 UTC; 10h ago
    Process: 856 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, sta>
      Tasks: 33 (limit: 9436)
     Memory: 329.3M (peak: 350.0M)
        CPU: 4min 26.461s
     CGroup: /system.slice/wazuh-agent.service
             ├─898 /var/ossec/bin/wazuh-execd
             ├─906 /var/ossec/bin/wazuh-agentd
             ├─919 /var/ossec/bin/wazuh-syscheckd
             ├─929 /var/ossec/bin/wazuh-logcollector
             └─946 /var/ossec/bin/wazuh-modulesd
```

Gambar 4. 18 Status layanan Wazuh Agent

4.1.5 Instalasi dan Konfigurasi pada *Virtual Machine* IV (*Attacker*)

Proses instalasi dilakukan pada tiga virtual machine terpisah dalam VMware Workstation. Masing-masing VM dialokasikan sumber daya minimal 2 vCPU, 2 GB RAM, dan 20 GB penyimpanan karena Kali Linux memerlukan kapasitas lebih besar dibandingkan Ubuntu Server. Instalasi dilanjutkan dengan pemilihan file ISO Kali Linux, pengaturan hostname yang berbeda untuk setiap attacker (misalnya *attacker-01*, *attacker-02*, *attacker-03*), pembuatan akun pengguna, dan aktivasi *OpenSSH Server* untuk mempermudah akses jarak jauh. Setelah instalasi selesai, masing-masing mesin di-*reboot* untuk melanjutkan konfigurasi berikutnya.

4.1.5.1 Instalasi Sistem Dasar

Sebelum menjalankan serangan, VM *Attacker* perlu dipersiapkan dengan sistem operasi Kali Linux serta paket pendukung jaringan.

Perintah:

```
sudo apt update && sudo apt upgrade -y
sudo apt install curl net-tools iputils-ping -y
```

Paket ini digunakan untuk memastikan VM *Attacker* dapat melakukan ping, cek routing, dan konektivitas jaringan dasar.

4.1.5.2 Konfigurasi Jaringan pada Mesin Attacker

Setiap attacker dikonfigurasi agar berada pada jaringan internal penelitian yang sama dengan Web Server dan Linux Gateway. Pengaturan alamat IP dilakukan melalui:

```
sudo nano /etc/netplan/01-network-manager-all.yaml
```

Tabel 4. 2 Arsitektur dan Pembagian VM Attacker

Nama VM	IP Address
Attacker-SYN	192.168.8.103
Attacker-ICMP	192.168.8.104
Attacker-UDP	192.168.8.105

4.1.5.3 Instalasi Tools Serangan

hping3 merupakan tool utama yang digunakan untuk menghasilkan serangan DoS dan *DDoS*. Tool ini tersedia pada repository Kali Linux.

Perintah:

```
sudo apt install hping3 -y
```

Cek versi *hping3*:

Gambar 4. 19 Verifikasi instalasi hping3

hping3 mendukung berbagai protokol seperti *TCP*, *UDP*, dan *ICMP* sehingga cocok untuk pengujian *SYN Flood*, *UDP Flood*, dan *ICMP Flood*.

4.2 Pengujian Sistem

Pada tahap ini dilakukan serangkaian pengujian untuk memastikan bahwa sistem yang dibangun mampu berfungsi sesuai tujuan, baik dalam kondisi normal maupun ketika menerima serangan. Pengujian ini dilakukan secara bertahap untuk melihat performa layanan Web Server, efektivitas monitoring oleh Wazuh, serta kemampuan Linux Gateway dalam mendistribusikan trafik dan menjalankan mekanisme mitigasi. Tahapan pengujian terdiri dari pengujian trafik normal, pengujian serangan DoS dan DDoS, analisis alert pada Wazuh, pengujian mitigasi blackhole routing, serta evaluasi kinerja sistem setelah mitigasi. Pengujian ini bertujuan untuk mendapatkan gambaran menyeluruh mengani perilaku sistem dalam berbagai skenario dan untuk menilai efektivitas implementasi Wazuh serta Linux Gateway dalam menghadapi serangan.

4.2.1 Pengujian Trafik Normal dan Konektivitas Sistem

Pengujian trafik normal dilakukan untuk memastikan bahwa seluruh komponen jaringan, termausk Web Server, Linux Gateway, dan sistem monitoring Wazuh, berfungsi dengan baik sebelum dilakukan serangan. Pengujian ini menjadi baseline yang digunakan sebagai pembanding dengan kondisi sistem saat menerima trafik berlebih. Melalui pengujian ini, dapat dilihat bagaimana perilaku server dan dashboard Wazuh ketika sistem berada dalam kondisi stabil tanpa adanya aktivitas berbahaya.

1. **Pengujian Konektivitas Client ke Gateway**

Pengujian pertama dilakukan untuk memastikan bahwa Client dapat terhubung dengan Linux Gateway sebagai perantara uatama seluruh trafik menuju target.

```
client@client:~$ ping 192.168.8.102
PING 192.168.8.102 (192.168.8.102) 56(84) bytes of data.
64 bytes from 192.168.8.102: icmp_seq=1 ttl=64 time=7.62 ms
64 bytes from 192.168.8.102: icmp_seq=2 ttl=64 time=5.65 ms
64 bytes from 192.168.8.102: icmp_seq=3 ttl=64 time=13.0 ms
64 bytes from 192.168.8.102: icmp_seq=4 ttl=64 time=10.1 ms
64 bytes from 192.168.8.102: icmp_seq=5 ttl=64 time=15.1 ms
64 bytes from 192.168.8.102: icmp_seq=6 ttl=64 time=5.40 ms
64 bytes from 192.168.8.102: icmp_seq=7 ttl=64 time=5.49 ms
64 bytes from 192.168.8.102: icmp_seq=8 ttl=64 time=4.06 ms
64 bytes from 192.168.8.102: icmp_seq=9 ttl=64 time=7.04 ms
64 bytes from 192.168.8.102: icmp_seq=10 ttl=64 time=3.28 ms
64 bytes from 192.168.8.102: icmp_seq=11 ttl=64 time=4.22 ms
64 bytes from 192.168.8.102: icmp_seq=12 ttl=64 time=20.8 ms
64 bytes from 192.168.8.102: icmp_seq=13 ttl=64 time=3.50 ms
64 bytes from 192.168.8.102: icmp_seq=14 ttl=64 time=11.2 ms
```

Gambar 4. 20 Hasil Pengujian Konektivitas Client ke Gateway

Hasil ping menunjukkan bahwa Client dapat berkomunikasi dengan Gateway tanpa packet loss. Hal ini memastikan bahwa jalur awal pengiriman tarfik bekerja dengan baik.

2. **Pengujian Konektivitas Gateway ke Web Server**

Selanjutnya, konektivitas diperiksa dari Gateway ke Web Server untuk memastikan bahwa konfigurasi routing pada kedua mesin berjalan sesuai rencana.

```
gunawan@mitigasi:~$ ping 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=4.30 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.605 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.938 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.602 ms
64 bytes from 10.10.10.10: icmp_seq=5 ttl=64 time=1.35 ms
64 bytes from 10.10.10.10: icmp_seq=6 ttl=64 time=0.594 ms
64 bytes from 10.10.10.10: icmp_seq=7 ttl=64 time=0.571 ms
64 bytes from 10.10.10.10: icmp_seq=8 ttl=64 time=0.682 ms
64 bytes from 10.10.10.10: icmp_seq=9 ttl=64 time=0.545 ms
64 bytes from 10.10.10.10: icmp_seq=10 ttl=64 time=0.588 ms
```

Gambar 4. 21 Pengujian Konektivitas Gateway ke Web Server

Respons ping menunjukkan bahwa interface internal Gateway (ens37) dapat menjangkau Web Server. Artinya kkonfigurasi subnet 10.10.10.10/24 telah berhasil diterapkan.

3. Pengujian Konektivitas Client ke Web Server

Ini merupakan pengujian konektivitas utama untuk memastikan routing end-to-end: client→gateway→web server.

```
client@client:~$ ping 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=63 time=23.9 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=63 time=11.0 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=63 time=7.06 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=63 time=1460 ms
64 bytes from 10.10.10.10: icmp_seq=5 ttl=63 time=316 ms
64 bytes from 10.10.10.10: icmp_seq=6 ttl=63 time=22.2 ms
```

Gambar 4. 22 Pengujian Konektivitas Client ke Web Server

4. Pengujian Akses HTTP (curl) dari Client ke Web Server

Setelah ping berhasil, dilakukan pengujian akses HTTP untuk memastikan bahwa layanan Nginx berjalan dan dapat memberikan respons.

```
client@client:~$ curl http://10.10.10.10
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
client@client:~$
```

Gambar 4. 23 Hasil pengujian akses server target

Berdasarkan hasil yang ditampilkan pada gambar, client berhasil menerima seluruh konten halaman default Nginx berupa kode HTML lengkap. Tampilan HTML tersebut berisi judul "Welcome to nginx!" serta informasi bahwa instalasi web server telah berjalan dengan benar. Keberhasilan curl menampilkan halaman ini membuktikan bahwa jalur komunikasi antara client, gateway, server target sudah berjalan tanpa hambatan. Selain itu, layanan HTTP pada server target berfungsi dengan normal dan dapat diakses dari jaringan lain melalui routing yang telah dikonfigurasi.

Dengan demikian, pengujian akses web ini memperkuat hasil sebelumnya bahwa komunikasi jaringan bekerja optimal dan sistem telah siap untuk dilakukan pengujian lanjutan, seperti pendeteksian serangan serta mitigasi menggunakan Wazuh.

5. Pengamatan Performa Web Server

Untuk memastikan Web Server beroperasi dalam kondisi stabil, dilakukan pemantauan performa menggunakan perintah:

```
top - 08:34:00 up 26 min,  2 users,  load average: 0.00, 0.00, 0.00
Tasks: 216 total,   1 running, 215 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.2 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3875.8 total,   3183.5 free,    304.9 used,    387.3 buff/cache
MiB Swap:   2830.0 total,   2830.0 free,      0.0 used.   3337.2 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    725 root      20   0  241048   8552   7236 S   0.3   0.2   0:03.43 vmtoolsd
      1 root      20   0  100920  11448   8100 S   0.0   0.3   0:07.12 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.05 kthreadd
      3 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_gp
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_par_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 slub_flushwq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 netns
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 mm_percpu_wq
     11 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_rude_
     12 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_trace
     13 root      20   0       0      0      0 S   0.0   0.0   0:00.08 ksoftirqd/0
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.46 rcu_sched
     15 root      rt   0       0      0      0 S   0.0   0.0   0:00.03 migration/0
     16 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     17 root      20   0       0      0      0 I   0.0   0.0   0:00.03 kworker/0:1-events
     18 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     20 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.93 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.05 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     25 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kdevtmpfs
     26 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 inet_frag_wq
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kauditd
     28 root      20   0       0      0      0 I   0.0   0.0   0:01.59 kworker/0:2-events
     29 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     30 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
     31 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 writeback
     32 root      20   0       0      0      0 S   0.0   0.0   0:00.12 kcompactd0
```

Gambar 4. 24 Performa Web Server

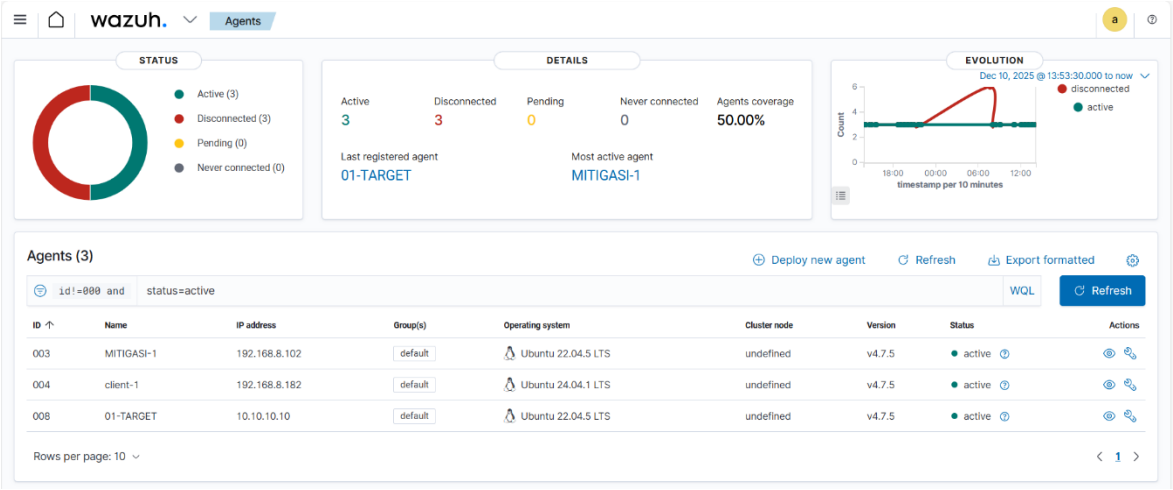
Gambar tersebut menunjukkan hasil pemantauan sistem pada server target menggunakan perintah top saat pengujian trafik normal dilakukan. Pada kondisi ini, client mengirimkan permintaan ping secara terus-menerus ke server dengan IP 10.10.10.10 untuk menguji konektivitas dan kestabilan jaringan melalui Linux Gateway.

Dari tampilan top, terlihat bahwa penggunaan CPU berada pada angka 0% idle 99–100%, menandakan bahwa aktivitas ping tidak memberikan beban signifikan terhadap server. Seluruh proses yang terlihat pada daftar merupakan proses sistem standar seperti systemd, kthreadd, rcu_sched, dan kernel worker lainnya. Tidak ada proses yang menunjukkan lonjakan konsumsi CPU atau memori, sehingga trafik dianggap normal dan tidak mengganggu performa server.

Hasil ini menunjukkan bahwa server target mampu menerima dan merespons permintaan ICMP (ping) dari client dengan baik tanpa menyebabkan overload. Pengamatan ini menjadi baseline yang penting sebelum pengujian serangan dilakukan, karena memastikan bahwa kondisi server dalam keadaan stabil dan berfungsi normal sebelum disimulasikan adanya lonjakan trafik anomali.

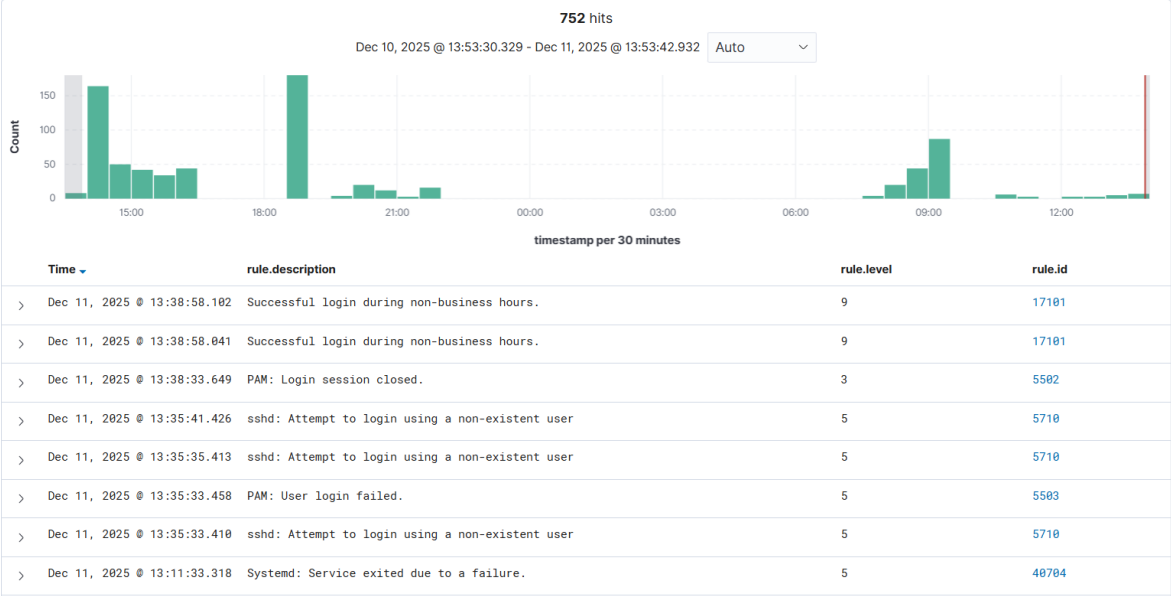
6. Verifikasi Log Normal pada Wazuh dashboard

Pada tahap ini dilakukan penegcekan bahwa selurh agent pada Client, Linux Gateway, dan Web Server telah berhasil terhubung ke Wazuh Server dan mengirimkan log secara normal. Hal ini penting untuk memastikan bahwa proses monitoring sudah berjalan sebelum pengujian serangan dilakukan.



Gambar 4. 25 Status Agent pada Menu Agents

Gambar diatas memperlihatkan halaman Agents pada Wazuh dashboard, yang menampilkan status seluruh node yang terhubung ke Wazuh Manager. Pada gambar ini agent dalam kondisi axtive, yang menandakan seluruh mesin berhasil terhubung dan mengirimkan log secara real-time ke Wazuh server. Status active mengonfirmasi bahwa koneksi antara agent dan manager berjalan normal, tidak ada error komunikasi, dan tidak ada agent yang mengalami connection timeout. Hasil ini menunjukkan bahwa tahap instalasi dan konfigurasi agent pada masing-masing VM telah dilakukan dengan benar sehingga Wazuh siap melakukan monitoring penuh, termasuk mendeteksi aktivitas abnormal saat pengujian DDoS dimulai.



Time	rule.description	rule.level	rule.id
Dec 11, 2025 @ 13:38:58.102	Successful login during non-business hours.	9	17101
Dec 11, 2025 @ 13:38:58.041	Successful login during non-business hours.	9	17101
Dec 11, 2025 @ 13:38:33.649	PAM: Login session closed.	3	5502
Dec 11, 2025 @ 13:35:41.426	sshd: Attempt to login using a non-existent user	5	5710
Dec 11, 2025 @ 13:35:35.413	sshd: Attempt to login using a non-existent user	5	5710
Dec 11, 2025 @ 13:35:33.458	PAM: User login failed.	5	5503
Dec 11, 2025 @ 13:35:33.410	sshd: Attempt to login using a non-existent user	5	5710
Dec 11, 2025 @ 13:11:33.318	Systemd: Service exited due to a failure.	5	40704

Gambar 4. 26 Log Event dari Agent Client

Gambar ini menampilkan daftar log yang dikirimkan khusus oleh agent Client pada Wazuh Dashboard. Event yang terlihat mencakup aktivitas login, sesi SSH, proses system, hingga percobaan login yang gagal. Semua log ini merupakan bagian dari system event monitoring yang dilakukan Wazuh Agent. Pada tahap ini, tidak terdapat log berkaitan dengan serangan atau trafik abnormal, sehingga seluruh event dikategorikan sebagai aktivitas normal sistem.

Grafik yang muncul menujukkan distirbusi jumlah log yang dikirimkan dalam rentang waktu tertentu, yang menandakan bahwa agent Client aktif, stabil, dan mampu mengirimkan log secara real-time ke Wazuh Manager. Hasil ini membuktikan bahwa instalasi dan konfigurasi Wazuh Agent pada Client telah berhasil, serta jalur komunikasi agent-manager berjalan. Dengan dmeikian, sistem monitoring dasar telah siap untuk digunakan pada tahap pengujian berikutnya.

4.2.2 Pengujian Serangan DDoS (*Multi Attacker*)

Pengujian ini dilakukan untuk mensimulasikan serangan Distributed Denial of Service (DDoS) yang berasal dari banyak sumber secara bersamaan. Tidak seperti serangan DoS yang hanya berasal dari saru mesin penyerang, serangan DDoS melibatkan beberapa attacker untuk menghasilkan beban trafik yang jauh lebih besar dan lebih sulit ditangani oleh Web Server. Pada penelitian ini digunakan tiga mesin attacker berbasis Kali Linux untuk menjalankan serangan secara simultan, sehingga Web server menerima tekanan trafik dari berbagai alamat IP. Tujuan pengujian ini adalah mengetahui dampak serangan DDoS

terhadap kinerja Web server serta mengevaluasi kemampuan Wazuh dalam mendeteksi pola serangan terdistribusi.

1. **Eksekusi Serangan DDoS dari Tiga Mesin Attacker secara Bersamaan**

Setiap attacker menjalankan serangan menggunakan hping3 secara bersamaan, sehingga Web Server dibanjiri oleh TCP SYN tiga sumber berbeda.

```
bot@bot:~$ ./bot_master.sh
[*] Menjalankan serangan di semua attacker...
 - Attack started on attacker1@192.168.8.103
 - Attack started on attacker2@192.168.8.104
 - Attack started on attacker3@192.168.8.105
[*] Serangan berjalan. Tekan CTRL+C untuk menghentikan semua attacker.
```

Gambar 4. 27 Eksekusi serangan DDoS

Pada pengujian serangan DDoS ini, tiga mesin attacker dijalankan secara bersamaan dengan jenis serangan yang berbeda: ICMP Flood dari mesin pertama, UDP Flood dari mesin kedua, dan SYN Flood dari mesin ketiga. Ketiga serangan ini dikirim menuju alamat IP server target 10.10.10.10 secara terus-menerus dengan intensitas tinggi. ICMP flood memanfaatkan perintah ping -f untuk mengirim paket ICMP dalam jumlah besar, sedangkan UDP dan SYN flood dijalankan menggunakan *hping3* dengan mode flood. Ketika ketiga serangan ini aktif secara simultan, trafik menuju server meningkat drastis hingga menyebabkan resource server menjadi penuh dan koneksi layanan web terganggu. Serangan terkoordinasi dari beberapa sumber ini menunjukkan bagaimana DDoS dapat membuat layanan tidak dapat diakses meskipun sebelumnya server dalam kondisi normal.

2. **Observasi Dampak Serangan pada Web Server**

Saat serangan berlangsung, Wazuh mencatat lonjakan trafik dan menghasilkan sejumlah alert dengan tingkat keparahan tinggi. Alert muncul karena Wazuh Agent mendeteksi pola anomaly

```
top - 09:36:01 up  1:20,  1 user,  load average: 0.00, 0.00, 0.00
Tasks: 214 total,   1 running, 213 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.2 sy,  0.0 ni, 98.7 id,  0.0 wa,  0.0 hi,  1.1 si,  0.0 st
MiB Mem :   3875.8 total,   3131.6 free,    319.0 used,    425.3 buff/cache
MiB Swap:   2830.0 total,   2830.0 free,      0.0 used.   3322.4 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
     13 root      20   0       0      0      0 S   2.3   0.0   0:11.95 ksoftirqd/0
     14 root      20   0       0      0      0 I   0.3   0.0   0:01.39 rcu_sched
    523 root      19  -1   64720  23296  22128 S   0.3   0.6   0:01.35 systemd-journal
    725 root      20   0  241048   8552   7236 S   0.3   0.2   0:12.41 vmtoolsd
   1714 syslog    20   0  222404   5112   4552 S   0.3   0.1   0:00.47 rsyslogd
      1 root      20   0  100920  11468   8100 S   0.0   0.3   0:07.64 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.06 kthreadd
      3 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_gp
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_par_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 slub_flushwq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 netns
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 mm_percpu_wq
     11 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_rude_
     12 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_trace
     15 root      rt   0       0      0      0 S   0.0   0.0   0:00.09 migration/0
     16 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     18 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     20 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:01.00 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:02.45 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     25 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kdevtmpfs
     26 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 inet_frag_wq
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kauditd
     29 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     30 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
     31 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 writeback
     32 root      20   0       0      0      0 S   0.0   0.0   0:00.38 kcompactd0
```

Gambar 4. 28 Dampak Serangan pada Web Server

Pada saat serangan DDoS berlangsung, dilakukan pemantauan langsung pada server target menggunakan perintah top. Berdasarkan tampilan pada gambar, terlihat bahwa beberapa proses kernel seperti ksoftirqd, rcu_sched, inet_frag_wq, dan proses dengan banyak interrupt mengalami peningkatan aktivitas. Proses-proses ini bertanggung jawab untuk menangani paket jaringan dan interrupt yang masuk.

Meskipun nilai CPU usage keseluruhan terlihat kecil, hal yang sebenarnya terjadi adalah server mengalami overload pada level interrupt (IRQ). Ketika tiga attacker mengirimkan paket secara bersamaan (ICMP flood, UDP flood, dan SYN flood), kernel menerima jumlah paket yang sangat besar dan harus memprosesnya secara terus menerus. Hal ini menyebabkan server memasuki kondisi soft lockup, yaitu server terlalu sibuk menangani interrupt sehingga tidak bisa merespons permintaan normal.

Akibatnya:

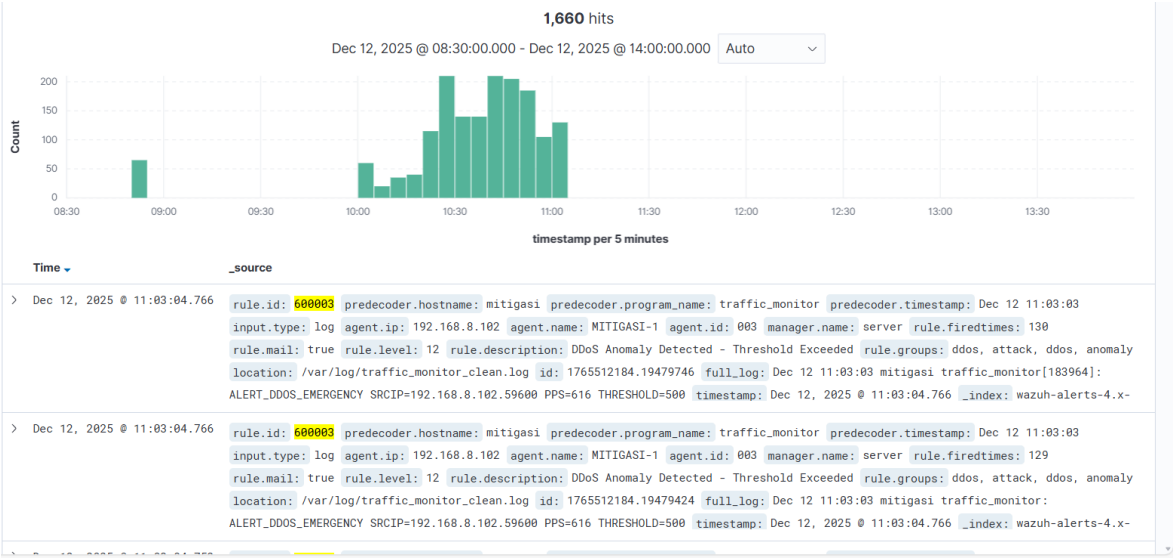
- Proses layanan seperti Nginx tidak mendapat jatah CPU.
- Permintaan HTTP/cURL dari client gagal dijawab ("Empty reply from server").

- Server terlihat hidup, tetapi fungsi aplikasinya mati secara logis (logical DoS).
- Load average sempat meningkat walaupun tidak sampai terlihat tinggi, karena beban terjadi pada jalur jaringan (network stack), bukan CPU userland.

Dengan demikian, serangan DDoS berhasil membuat server tidak responsif, sekalipun penggunaan CPU tampak rendah pada lapisan user. Kondisi ini merupakan karakteristik umum dari serangan flood yang memaksa kernel bekerja secara berlebihan untuk menangani trafik yang datang dari banyak sumber.

3. Deteksi Anomali Secara Otomatis oleh wazuh

Pada tahap ini dilakukan pengamatan terhadap Wazuh dalam mendeteksi anomali trafik secara otomatis ketika terjadi lonjakan paket yang tidak normal. Deteksi anomali dilakukan berdasarkan analisis jumlah paket per detik (Packets Per Second / PPS) yang melewati Linux Gateway. Ketika nilai PPS melebihi ambang batas (threshold) yang telah ditentukan, Wazuh akan menghasilkan alert sebagai indikasi adanya potensi serangan DoS atau DDoS. Visualisasi hasil deteksi anomali tersebut ditampilkan melalui Wazuh Dashboard pada menu Security Events.



Gambar 4. 29 Deteksi Anomali Trafik oleh Wazuh berdasarkan Threshold PPS

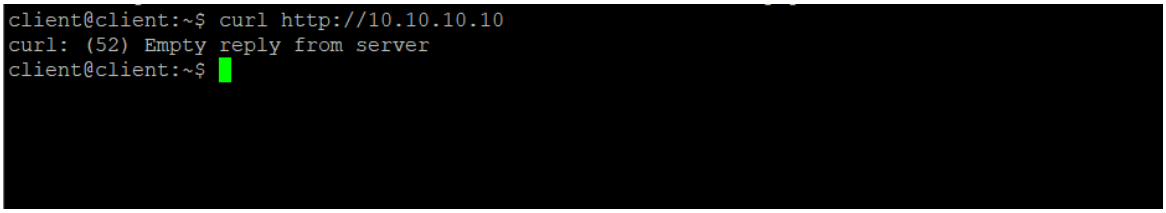
Berdasarkan tampilan pada Wazuh Dashboard, terlihat adanya peningkatan jumlah event yang signifikan dalam rentang waktu tertentu, yang ditunjukkan oleh grafik histogram pada bagian atas. Event tersebut berasal dari agent MITIGASI-1 yang berjalan pada Linux Gateway, dengan rule ID 600003 dan level alert 12, yang menandakan tingkat keparahan tinggi. Deskripsi rule menunjukkan bahwa sistem berhasil mendeteksi kondisi DoS

Anomaly Detected – Threshold Exceeded, di mana nilai PPS yang tercatat (misalnya PPS=616) telah melampaui ambang batas yang ditetapkan (THRESHOLD=500).

Log yang dihasilkan berasal dari proses traffic_monitor yang mencatat aktivitas jaringan secara periodik dan mengirimkannya ke Wazuh Manager. Hal ini membuktikan bahwa mekanisme deteksi anomali berbasis threshold telah berjalan secara otomatis dan mampu mengidentifikasi pola trafik yang tidak normal tanpa intervensi manual. Dengan demikian, Wazuh berfungsi sebagai sistem peringatan dini (early warning system) terhadap potensi serangan DoS/DDoS pada jaringan yang diuji.

4. **Pengujian Akses Web Server dari Client Selama Serangan**

   Untuk mengetahui bagaimana kondisi layanana saat serangan, Client mencoba mengakses Web server menggunakan perintah:



Gambar 4. 30 Pengujian akses Web Server

   Pada saat serangan DDoS sedang berlangsung, client mencoba mengakses web server menggunakan perintah curl http://10.10.10.10. Namun, server memberikan respons berupa pesan “Empty reply from server”, yang menunjukkan bahwa koneksi berhasil dibuat tetapi server tidak mampu mengirimkan data balasan. Kondisi ini terjadi karena resource server sedang sepenuhnya digunakan untuk menangani paket serangan yang bertubi-tubi dari beberapa attacker, sehingga proses layanan HTTP gagal memproses permintaan normal dari client. Hasil ini menegaskan bahwa serangan DDoS menyebabkan web server tidak dapat memberikan respon meskipun koneksi masih dapat terbentuk, sehingga layanan menjadi tidak tersedia bagi pengguna sah.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Hasil pengujian menunjukkan bahwa Wazuh telah mampu mendeteksi anomali trafik jaringan yang mengindikasikan serangan DDoS, seperti peningkatan packet per second (PPS) yang melebihi nilai ambang batas (threshold). Deteksi ini ditampilkan dalam bentuk alert pada Wazuh Dashboard, sehingga sistem dapat berfungsi sebagai early warning mechanism terhadap serangan jaringan. Selain itu, Wazuh juga berhasil menerima dan menampilkan log sistem dasar seperti aktivitas login, systemd, dan status layanan dari agent yang terpasang.

Namun demikian, mekanisme mitigasi otomatis belum berjalan secara penuh. Meskipun skrip mitigasi berbasis blackhole routing telah disiapkan dan dapat dijalankan secara manual, proses eksekusi mitigasi secara otomatis melalui fitur active response belum dapat dilakukan secara konsisten. Akibatnya, pada beberapa skenario pengujian, server target masih dapat menerima trafik serangan meskipun proses deteksi telah terjadi.

Secara keseluruhan, sistem yang dibangun telah berhasil mencapai fungsi deteksi serangan secara otomatis, namun tujuan mitigasi otomatis masih memerlukan penyempurnaan lebih lanjut. Sistem saat ini dapat dianggap sebagai prototipe yang telah mampu mendeteksi serangan dan memberikan peringatan dini, tetapi belum sepenuhnya mampu melakukan respons otomatis terhadap serangan DDoS.

5.2 Saran

Untuk meningkatkan efektivitas sistem, diperlukan beberapa pengembangan lanjutan. Pertama, integrasi antara hasil deteksi anomali dan mekanisme active response perlu diperkuat, sehingga setiap alert serangan dapat langsung memicu eksekusi skrip mitigasi secara otomatis. Kedua, pengelolaan dan validasi skrip mitigasi perlu diperbaiki, termasuk pengaturan izin eksekusi dan parameter input agar blackhole routing dapat diterapkan secara tepat sasaran.

Selain itu, penyesuaian threshold dan parameter deteksi anomali perlu dilakukan agar sistem mampu membedakan trafik normal dan trafik serangan dengan lebih akurat serta meminimalkan kesalahan deteksi. Dengan penyempurnaan pada aspek tersebut, sistem diharapkan dapat berkembang menjadi solusi deteksi dan mitigasi serangan jaringan yang lebih andal dan siap diterapkan pada skala yang lebih luas.

DAFTAR PUSTAKA

[1] A. Abdelhamid, M. S. Elsayed, A. D. Jurcut, and M. A. Azer, "A Lightweight Anomaly Detection System for Black Hole Attack," *Electron.*, vol. 12, no. 6, 2023, doi: 10.3390/electronics12061294.

[2] A. Apostu *et al.*, "Detecting and Mitigating DDoS Attacks with AI: A Survey," 2025, [Online]. Available: http://arxiv.org/abs/2503.17867

[3] Y. Yigit, B. Bal, A. Karameseoglu, T. Q. Duong, and B. Canberk, "Digital Twin-Enabled Intelligent DDoS Detection Mechanism for Autonomous Core Networks," *IEEE Commun. Stand. Mag.*, vol. 6, no. 3, pp. 38–44, 2022, doi: 10.1109/MCOMSTD.0001.2100022.

[4] I. Ko, D. Chambers, and E. Barrett, "Adaptable feature-selecting and threshold-moving complete autoencoder for DDoS flood attack mitigation," *J. Inf. Secur. Appl.*, vol. 55, no. October, 2020, doi: 10.1016/j.jisa.2020.102647.

[5] S. P. Thirimanne, L. Jayawardana, L. Yasakethu, P. Liyanaarachchi, and C. Hewage, "Deep Neural Network Based Real-Time Intrusion Detection System," *SN Comput. Sci.*, vol. 3, no. 2, pp. 1–12, 2022, doi: 10.1007/s42979-022-01031-1.

[6] A. Ahmed, S. Hameed, M. Rafi, and Q. A. Mirza, "An Intelligent and Time-Efficient DDoS Identification Framework for Real-Time Enterprise Networks SAD-F: Spark Based Anomaly Detection Framework," *IEEE Access*, 2020, doi: 10.1109/ACCESS.2020.3042905.

[7] A. Amalia, Y. Pramitarini, R. H. Y. Perdana, K. Shim, and B. An, "A Deep-Learning-Based Secure Routing Protocol to Avoid Blackhole Attacks in VANETs †," *Sensors*, vol. 23, no. 19, pp. 1–28, 2023, doi: 10.3390/s23198224.

[8] F. M. Salem, H. Youssef, I. Ali, and A. Haggag, "A variable-trust threshold-based approach for DDOS attack mitigation in software defined networks," *PLoS One*, vol. 17, no. 8 August, pp. 1–19, 2022, doi: 10.1371/journal.pone.0273681.

[9] N. A. Stamatelatos, "A measurement study of BGP Blackhole routing performance," no. September 2006, p. 111, 2006, [Online]. Available: http://hdl.handle.net/10945/2648

[10] "AN ENHANCED MECHANISM TO DETECT," 2023.

[11] D. Firdaus, F. Fahira, and R. Rianti, "Deteksi Anomali Dan Serangan Low Rate Ddos Dalam Lalu Lintas Jaringan Menggunakan Naive Bayes," *Naratif J. Nas. Riset, Apl.*

dan Tek. Inform., vol. 5, no. 2, pp. 140–148, 2023, doi: 10.53580/naratif.v5i2.208.

[12] Fitri Nova, M. D. Pratama, and D. Prayama, "Wazuh sebagai Log Event Management dan Deteksi Celah Keamanan pada Server dari Serangan Dos," *JITSI J. Ilm. Teknol. Sist. Inf.*, vol. 3, no. 1, pp. 1–7, 2022, doi: 10.62527/jitsi.3.1.59.

[13] Y. Wei, J. Jang-Jaccard, F. Sabrina, W. Xu, S. Camtepe, and A. Dunmore, "Reconstruction-based LSTM-Autoencoder for Anomaly-based DDoS Attack Detection over Multivariate Time-Series Data," vol. 14, no. 8, pp. 1–13, 2023, [Online]. Available: http://arxiv.org/abs/2305.09475

[14] L. H. De Melo, G. de C. Bertoli, M. Nogueira, A. L. Dos Santos, and L. A. Pereira, "Anomaly-Flow: A Multi-domain Federated Generative Adversarial Network for Distributed Denial-of-Service Detection," *IEEE Netw.*, pp. 1–8, 2025, doi: 10.1109/MNET.2025.3567251.

[15] T. Y. Paskah, D. W. Sudiharto, and M. A. Nugroho, "Analisis restorasi layanan jaringan menggunakan StopIt dan RTBH dalam perbandingan. Studi kasus: kontra serangan DDoS," vol. 5, no. 3, pp. 7578–7585, 2018.

[16] H. A. Damanik and M. Anggraeni, "Analisis dan Mitigasi Kerentanan DDoS pada Infrastuktur Jaringan dengan Teknik Hierarchical Clustering dan Firewall IPTables," *J. Pekommas*, vol. 10, no. 1, pp. 29–38, 2025, doi: 10.56873/jpkm.v9i1.5551.

[17] J. Hansen and T. Sutabri, "Mendesain Cyber Security Untuk Mencegah Serangan DDoS Pada Website Menggunakan Metode Captcha," *Digit. Transform. Technol.*, vol. 3, no. 1, pp. 289–298, 2023.

[18] "No Title", [Online]. Available: https://www.cloudflare.com/img/learning/ddos/what-is-a-ddos-attack/ntp-amplification-botnet-ddos-attack.png)

[19] L. B. Gardea, "MOBILE FIREWALL SYSTEM FOR DISTRIBUTED DENIAL OF SERVICE," no. June, 2023.

[20] Septian Geges1) and Waskitho Wibisono2), "Pengembangan Pencegahan Serangan Distributed Denial of Service (Ddos) Pada Sumber Daya Jaringan Dengan Integrasi Network Behavior Analysis Dan Client Puzzle," *JUTI J. Ilm. Teknol. Inf.*, vol. 13, no. 1, pp. 53–67, 2015.

[21] I. M. W. Bhaskara, I. P. G. H. Suputra, I. M. Widiartha, I. G. A. G. A. Kadyanan, I. G. N. A. C. Putra, and I. B. G. Dwidasmara, "Klasifikasi Serangan Distributed Denial of Service (DDoS) Menggunakan Random Forest Dengan CFS," *JELIKU (Jurnal*

Elektron. Ilmu Komput. Udayana), vol. 11, no. 2, p. 215, 2022, doi: 10.24843/jlk.2022.v11.i02.p01.

[22] S. Neciri and N. Chaib, "INTELLIGENT SYSTEMS AND APPLICATIONS IN ENGINEERING Hybrid Deep Learning for Anomaly Detection in FANETs : A Defense Against DDoS Attacks," vol. 12, no. 4, pp. 3799–3809, 2024.

[23] C. A. Poveda, "Naval Postgraduate," *Security*, no. June, 2007.

[24] R. Abubakar *et al.*, "An Effective Mechanism to Mitigate Real-Time DDoS Attack," *IEEE Access*, vol. 8, pp. 126215–126227, 2020, doi: 10.1109/ACCESS.2020.2995820.

[25] G. Kalifaur, Khairtati, "Cendikia pendidikan," *Cendekia Pendidik.*, vol. 4, no. 4, pp. 50–54, 2024, doi: 10.9644/sindoro.v3i9.252.

[26] "No Title", [Online]. Available: Wazuh agent - Components · Wazuh documentation)

[27] "No Title", [Online]. Available: vmware-workstation-pro · GitHub Topics

[28] "No Title", [Online]. Available: https://brandlogo.org/wp-content/uploads/2024/09/Oracle-VirtualBox-Logo-2024.png

LAMPIRAN

Code traffic_monitor.sh
/usr/local/bin/traffic_monitor.sh

```
#!/bin/bash
THRESHOLD=500
LOG="/var/log/traffic_monitor_clean.log"

while true; do
  # Hitung koneksi per IP
  ss -tn state syn-recv | awk 'NR>1 {print $5}' | cut -d: -f1 | sort |
uniq -c | sort -nr > /tmp/ip_counts

  # Baca IP dengan koneksi tertinggi
  read count ip < /tmp/ip_counts

  if [[ -n "$ip" && "$count" -gt "$THRESHOLD" ]]; then
    echo "DDOS_PPS_ANOMALY srcip=$ip pps=$count" | sudo tee -a "$LOG"
    logger "DDOS_PPS_ANOMALY srcip=$ip pps=$count"
  fi

  sleep 1
done
```

Code ddos_block.sh

```
#!/bin/bash
IP="$1"
if [[ $IP =~ ^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$ ]]; then
  sudo /sbin/ip route add blackhole "$IP" 2>/dev/null
  echo "$(date) BLOCKED $IP" >> /var/log/ddos-block.log
fi
```